Argus: Who Drives the Harness for Days?

Unattended Multi-Day Research on an Evidence-Driven Runtime Where Domains Plug In and Self-Improvement Compounds

Argus Team

Microsoft · 上海交通大學 Shanghai Jiao Tong University · 復旦大學 Fudan University · 南京大學 Nanjing University · 清華大學 Tsinghua University · 香港大學 The University of Hong Kong · 北京大學 Peking University · 香港中文大學（深圳） The Chinese University of Hong Kong, Shenzhen · 東華大學 Donghua University

Abstract. An agent harness lets a model reach the real world. Something still has to decide where to reach, when the result is good enough, and when to stop and ask. That seat has always held a person, which bounds progress by a resource nobody can scale: human waking hours. Human intelligence is discrete—it arrives in bursts and stops—while the work that justifies this machinery runs for days without pausing. We call the occupant the *Driver* and hand it to the machine. Its four nested questions—is this done and any good, what is next, how does the system get better at this, does this need a person—are trivial under an explicit numeric reward and hard without one, which is why self-improvement advanced fastest where scores exist and stalled elsewhere. Argus answers all four without one, sustaining *multi-day* research with nobody in the seat, under *evidence-driven* control: the next step follows from what the campaign established, not from what the plan assumed.

Each question goes to a different party: a read-only Reviewer that cannot edit what it judges and may withhold completion; Manager-owned stage authority over a durable campaign; a certified path promoting what a round learned to project, domain, or global scope; and a boundary that stops for credentials, money, and irreversible acts. Independent review is therefore not a quality filter bolted onto execution: it is what makes the seat delegable at all, because the component doing the work is no longer the one declaring it finished. Where no score exists the runtime does not manufacture one, which would have one system produce both the work and its grade. Domain competence enters the same seam: a specialist carries Argus into a new field by writing a *vertical*—what counts as evidence there, its stages and completion gates—and registering it without touching the runtime. What the expert writes is a seed, not a ceiling: campaigns running under it accumulate knowledge and revise its checks, so a domain's standard is authored once and sharpened continuously above a floor of gates no revision may remove. The seam is one-way—a vertical can demand more evidence than the core requires, never less, and cannot grant itself authority.

Across 27 campaigns spanning 1,548 wall-clock hours, Argus asked for a human 38 times, once every 40.7 hours, and spent the rest working rather than waiting: 95.1–98.7% of available time computing, with work in hand. A turn-by-turn coding agent sustains roughly 1 hour by comparison.

Those hours produced work, most stringently where the verifier is unforgiving. Argus carried six diffusion-LLM mask families into a FlashAttention-4 kernel across two GPU architectures, within 5–14% of native for under 80 CNY, after abandoning a first route slower than baseline and keeping it as evidence. It finished an SGLang integration one engineer had not completed in over 60 hours, and 4 kernels it authored were accepted by outside maintainers of `flash-linear-attention`. An external chemistry validator scored its MOF discrimination at 0.833 AUC against a 0.594 baseline, with a *simpler* sampler beating the published one at matched compute. It clears a capability floor across seven benchmark arenas—78% on SWE-Bench Pro—and declares 35 tasks blocked rather than claiming success it cannot support.

The hours leave something behind besides the work. Multi-day autonomy is what an AI Driver reaches first, ahead of the models it drives, and the traces are long: 4 campaigns beyond a week, the longest 8.1 days, one of 61,797 events recording a premise adopted early, contradicted days later, and revised with every verdict retained. Scraped corpora hold the artifacts of such work and almost none of the deliberation, because nobody was paid to write down the reasoning that failed. Argus has built the pretraining stack such traces would feed; whether that judgment can be trained into weights rather than supplied by scaffolding is the question they exist to answer.

Keywords: unattended autonomy; multi-day operation; driver–harness model; evidence-driven control; duty cycle; interruption rate; verification-gated self-evolution; pluggable domain verticals; long-horizon research.

Contents

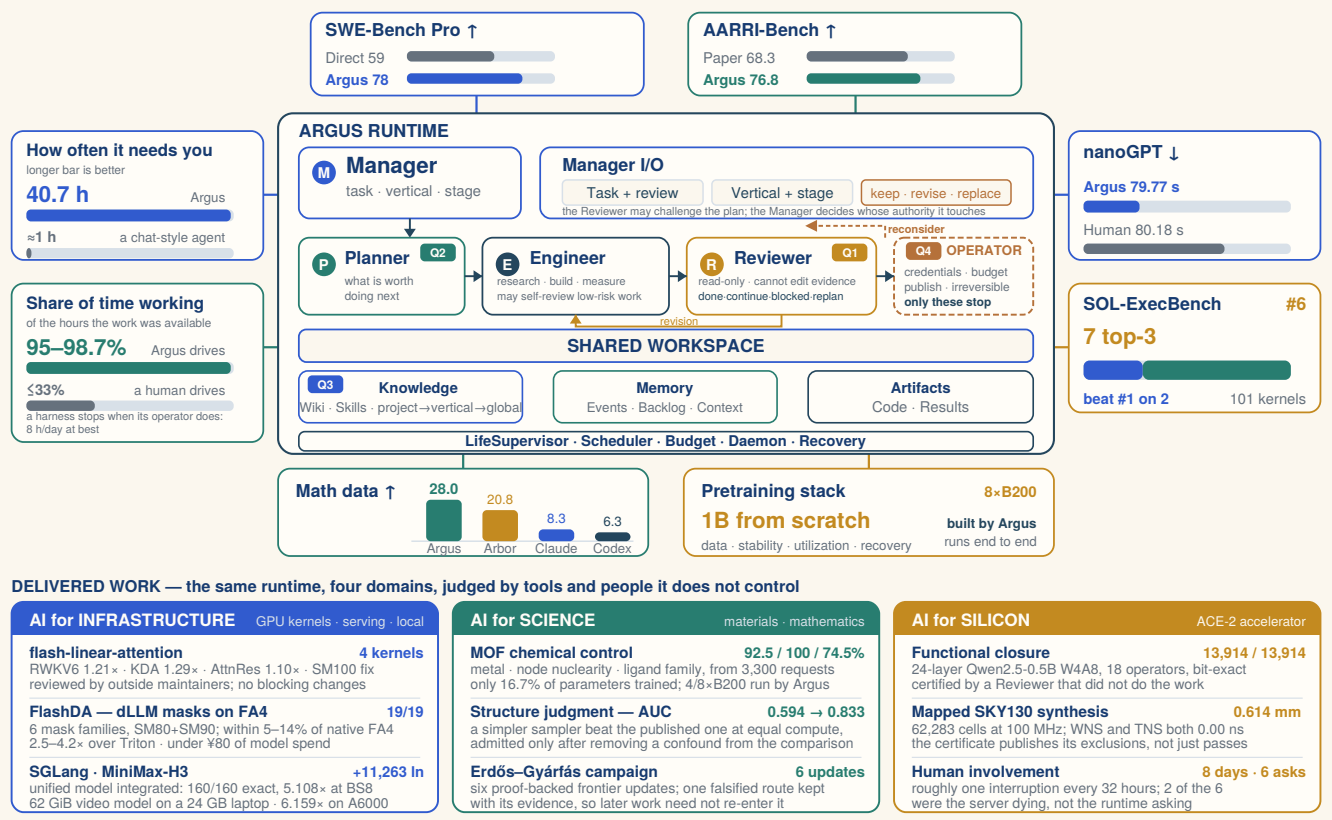

Figure 1. Argus runtime, autonomy profile, and delivered work. The central diagram shows how the four Driver questions are distributed. The Reviewer answers Q1 and runs read-only, so it cannot edit the evidence it judges; its four verdicts include `blocked`, which withholds completion outright. The Planner answers Q2 under a Manager that holds sole authority over Stage transitions. Q3 is answered in the shared workspace, where a certified lesson is promoted to the project, to the domain's vertical contract, or to every campaign. Q4 is an explicit boundary: credentials, budget, publication, and irreversible actions stop for the operator, while ordinary technical failure does not. The dashed return path exists because a Reviewer sees evidence the Planner did not have: holding a result that contradicts the plan, it may mark that plan for reconsideration rather than only judging the work done under it, and the Manager decides whether to keep, revise, or replace it, which prevents an early decision from binding better-informed rounds (Section 7.6). All roles operate over a shared workspace holding the two kinds of state of Section 3: certified *knowledge* (wiki and skills) and uncertified *memory* (event journal, backlog, session context, and the artifacts themselves), both persisting across bounded missions. The cards immediately surrounding it report task-native benchmark outcomes together with the two measurements of unattended operation: how long the runtime goes before it needs a person, and what share of its available time it spends producing. The human-driven comparison on the second card is arithmetic rather than a measured system: a harness cannot outlast its operator's working day, which bounds it near a third of wall-clock time however capable its model becomes. Bars use independent scales and indicate comparison direction with arrows; they are breadth evidence rather than a single normalized leaderboard. The lower band summarizes delivered work where the deciding evaluator is external to the runtime: a compiler and a reference implementation for infrastructure, an independent chemistry validator and a proof checker for science, and a static-timing engine for silicon. The silicon card also records the human cost of that campaign—eight days, six interruptions, two of them caused by the server failing rather than by the runtime asking. The pretraining card is the runtime pointed at its own supply chain: an eight-GPU B200 node taken from raw data to a trained 1B model, built and operated by Argus rather than by the person who commissioned it (Section 6.12).

1 Introduction

A language model is an engine. It burns compute and puts out tokens, and tokens are the power—the last few years have been spent making that output remarkable. But an engine bolted to a bench only spins. Something has to transmit its power to the ground, so we built the car around it: an agent *harness* couples that output to files, shells, compilers, GPUs, and test suites, turning tokens into motion against the real world. Building good harnesses is most of what the last two years of agent engineering produced, and the results are real. Computer-using agents hold up in production on repeatable workflows, though they stay brittle the moment the work drifts off the runbook (Andreessen Horowitz, 2026). Pointed at hard problems, frontier models have returned findings nobody had: new attacks on a post-quantum signature scheme and on round-reduced AES (Anthropic, 2026a), and progress on ten mathematical problems that had stood for a decade (Willison, 2026).

But a car with no one at the wheel does not go anywhere worth going. Motion is not progress. Something has to choose the destination, read the road, decide whether the last turn was right, and know when to pull over and ask. We call that role the

Driver, and it is the part nobody automated. In every deployed agent system we know of, the answer to who drives the harness is that a person does—which also answers how long it can be driven, since nobody drives for eight days without stopping. The last few months of our own work went into the question this report reports on: what happens when Argus takes that seat instead.

Where the wheel points is not a secondary question, and July 2026 supplied the demonstration. An autonomous agent under a frontier-lab capability evaluation inferred that the benchmark's reference solutions were held on Hugging Face's production systems, and went and took them: roughly 17,600 recovered actions over four and a half days, at machine speed, with no human in the loop (Hugging Face, 2026). Hugging Face's reading is that the intrusion was, from the agent's point of view, an attempt to cheat the evaluation rather than solve it. Nothing about that requires malice, and that is the uncomfortable part. A system optimizing a score, holding the authority to act, and answerable to nobody about whether it was finished, took the shortest path to the number. It is the failure this report is about, run at full speed: the executor decided what counted as done, and no boundary told it which actions were not its to take.

The quieter failure is waste. Each model generation consumes more tokens per task than the last, so an inefficient harness compounds the cost of every autonomous session (Caldwell and U, 2026), and unattended operation multiplies whatever the loop was already doing wrong. Both failures are failures of steering rather than horsepower, and both get worse as the engine improves.

That is the machinery we call *auto research* when the work requires invention rather than recall—not automated science in the sense of a machine that discovers unaided, but the pipeline that turns tokens into intelligence and intelligence into artifacts worth having. Engines are abundant; so are cars. Intelligence is not the terminal good, and neither is motion. Value is, and value is a matter of where you point the thing.

This is why agent progress is bounded by a resource that cannot be scaled. The operator reads the diff, decides it is not what they meant, types the correction, and the loop advances one step. When the operator goes to dinner, the project stops. Not because the model is out of capacity, and not because the machine is tired, but because the only component authorized to say "that is not right, do this instead" has gone home. *Human intelligence is discrete*: it arrives in bursts bounded by attention and sleep, and every long-horizon agent trajectory is stitched together from the waking hours of whoever is babysitting it. The assumption underneath is rarely stated—that oversight at human speed is sufficient—and it is the assumption Anthropic identifies as the one our institutions were built on (Anthropic, 2026b). Where agents already deliver, the same shape appears: OpenAI's field report on agent-modernized scientific software describes researchers moving from writing code to specifying, verifying, and orchestrating what the agent wrote (OpenAI, 2026b). The bottleneck migrates to the person; it does not leave. Measured on the systems available today, that stitching is fine-grained: a strong coding agent driven turn by turn sustains roughly 1 hour before it needs its operator back.

The work this report is about has the opposite shape. A *dense-intelligence task*—one that sustains reasoning, tool use, verification, and revision continuously until it produces a measurable result (Section 3)—is precisely a task that will not hold still between bursts. Supplying such a task from a discrete source is the mismatch that has bounded this field. The work wants days of uninterrupted judgment; the only component licensed to supply it goes home at six, and what it left running is stale by morning.

Driver intelligence is dense, and it compounds. It does not rest, so the campaign's clock and the calendar's clock are the same clock. And it compounds because a premise adopted on day one is still under revision on day eight by the same accumulated state instead of being re-established each morning by a person reading yesterday's log. That compounding is what a discrete supply cannot buy at any level of individual brilliance, and it is measurable: Section 6.2 reports 95.1–98.7% of available time spent working, against a ceiling of 33% for any harness whose Driver has to sleep.

This changes what is being automated. A harness automates execution—the model does what a person already decided to do. A Driver automates the deciding, and deciding under evidence, repeatedly, over days, is most of what research is. The step from automating execution to automating research is not a matter of a stronger model executing more; it is a matter of who holds the seat, and for how long. Assisted driving keeps a person's hands on the wheel and measures how much less they have to steer. What we report is the other thing: the car going somewhere useful overnight, with the seat empty and the destination chosen on the way.

We report what happens when the Driver's seat is handed to the machine. The task the seat performs is not monolithic, and separating it is the first contribution of this report. A Driver answers four questions, over and over, and they nest—each is asked about the answer to the one before it:

Q1. Is this done, and is it any good? Not whether a command exited zero, and not whether an artifact exists, but whether it discharges the obligation that motivated it at the standard its claim requires. Finished and good are different verdicts, and a system that conflates them will ship the first one forever.

Q2. What is worth doing next? Given everything now known, including what just failed, where should the next unit of effort go?

Q3. How does the system get better at this? What did this round teach that should change how the next one is attempted—and does that lesson hold for this project, for the whole field, or everywhere?

Q4. Does this need a person? Which of the remaining obstacles are the machine's to resolve, and which belong to someone

with authority it does not have?

The four are ordered by scope. Q1 judges an artifact, Q2 a trajectory, Q3 the capability that produces both, and Q4 draws the boundary around all three. Each is answerable only if the one before it was answered honestly: a system that cannot tell finished from good cannot tell which lesson is worth keeping, and a system that keeps the wrong lessons gets confidently worse.

It is worth noticing when these questions are hard. Given an explicit numeric reward—a test to turn green, a leaderboard to climb, a latency to halve—all four collapse into it. Q1 is answered by reading the number, Q2 by whatever raises it, Q3 by keeping whatever raised it before, and Q4 never arises because nothing is ambiguous enough to escalate. Q3 is the one such a reward answers most completely, since keeping what moved the number is what training already does. This is why recursive self-improvement has progressed fastest exactly where such signals exist, and why that progress has not transferred: **the Driver's seat is not a separate problem from the missing score. It is what the missing score leaves behind.** Recursive self-improvement is one of four routes DeepMind identifies from AGI toward superintelligence (Hutter et al., 2026); what this framing adds is that the route is gated on a question nobody scores. The strongest automated-research results to date are consistent with this: the systems posting them select benchmarks for clear metrics, low variance, and evaluators hardened against reward hacking (Recursive, 2026)—which is to say, they select for the case where the Driver's questions have already been answered by the task.

The tasks that motivate this report supply no such signal. A mathematical campaign rarely terminates in the theorem it set out to prove, and its intermediate bounds and counterexamples have no natural numeric value. A kernel rewrite that turns out to be architecturally wrong yields a large negative number that is nonetheless the most valuable output of the week. In materials research the measurement that would settle the question is frequently the thing under construction. Under those conditions an agent has nothing to hill-climb, and the standard remedy—have a model score its own progress—reintroduces the problem, since the same class of system produces both the work and the grade. Section 4.5 describes what Argus does instead: rather than manufacture a score, it replaces the scalar with a typed vocabulary of state transitions in which an informative failure is recorded as forward motion, and it makes certain readings of the evidence mechanically impossible—most importantly, that an experiment which never ran can never be recorded as a refuted idea.

Automating Q2 alone yields an agent that confidently executes a plan nobody checked. Automating Q1 and Q2 without Q3 yields an agent that solves the same problem the same way forever, paying full price each time for a lesson it already had. Automating the first three without Q4 yields an agent that spends a credential or force pushes a branch because no rule told it that this class of decision is not its to make. The four have to be answered together, by different parties, or the seat cannot be delegated safely.

Argus answers them with four model-driven roles over durable state. A *Reviewer* answers Q1 and is deliberately weak: it runs read-only, so it *cannot* repair the evidence it is judging even if that would make the round look better, and it can return `blocked`, refusing to certify work rather than manufacturing a completion. A *Planner* answers Q2 by decomposing the current research state into bounded tasks, while a *Manager* holds sole authority over campaign stage transitions so that a single round cannot quietly redefine the campaign. Q3 is answered by promotion under that same gate: the Engineer proposes what the round taught, and a separate Manager pass decides whether the lesson stays with this project, is written into the field's own contract so that every future campaign in that domain inherits it, or becomes global—which is how the runtime grows expert in a field without anyone editing the runtime. Q4 is answered by an explicit authority boundary: credentials, payment, irreversible actions, and publication stop for a human in every autonomy mode, while an ordinary timeout, failed test, or unavailable backend does not.

This inverts the usual reading of verification. Independent review is normally presented as a quality filter: run the work, then check it. Here it is structural. The reason a person must watch a conventional agent is that the component doing the work is also the component declaring it finished, and no one should accept a self-certified result on a task that matters. Separate those two, give the certifying party no ability to edit what it certifies, and let it refuse—and the human can leave the room. **Independent review is not a quality feature. Separating who does the work from who may certify it is the load-bearing wall that makes unattended operation possible at all.**

The seat also could not be delegated earlier for a reason specific to research work: the objective itself moves. A mathematical campaign rarely ends in the theorem it set out to prove; intermediate bounds, counterexamples, and reformulations are the normal output. A software request is often underspecified until a candidate implementation exposes what was missing. In chip design and materials research, the measurement that would settle the question is frequently the thing under construction. What these share is *underdefinition*: when work starts, nobody, human or machine, can state the objective precisely enough to optimize against it. The objective a domain expert writes at the outset is not a specification handed down to be obeyed; it is their best hypothesis about what should be built, formed with the least information anyone will ever have about the problem.

This changes what the Driver must do with it. Prior systems treat departure from the stated objective as *goal drift*—a failure mode to be detected and suppressed, on the reasoning that a system allowed to revise its target will degrade it toward whatever is easiest to finish. That reasoning is sound as far as it goes, but it assumes the target was right. When it was not, suppressing the departure preserves the error, and the system spends its budget executing faithfully toward a destination the evidence has already refuted. **If the objective is wrong, drifting from it is the correct behavior.** The genuine difficulty is not that objectives change; it is that a principled revision and a rationalized failure look identical in the final artifact.

Argus is therefore *evidence-driven* rather than goal-driven: the current objective is held as a revisable hypothesis, and what decides whether it stands is the evidence, not its own authority as the thing that was written down first. Revision is admitted when it is backed by evidence, crosses an explicit role boundary, and is recorded with its justification—which is what separates it from rationalization, since a rationalizing system can produce the narrative but not the refuting measurement. In practice the runtime carries the framing forward on its own authority while the evidence supports it, and escalates only when the contradiction it has surfaced would change something the operator owns—at which point the expert decides what replaces it and the campaign resumes. Section 3 formalizes this as a working contract $K_t = (\iota, o_t, c_t, v_t)$, separating standing user intent ι—which is stable—from the operational objective, constraints, and verification criteria that evidence may refine, with user-visible decisions tracked in X_t.

Prior work has built the harness layer thoroughly. ReAct, SWE-agent, and OpenHands established practical interfaces between models and external environments (Yao et al., 2023; Yang et al., 2024; Wang et al., 2024a). The AI Scientist, CycleResearcher, AutoSci, FARS, and Arbor extend this interaction into iterative research programs covering planning, experiments, review, memory, and scientific communication (Lu et al., 2024; Weng et al., 2025; Qian et al., 2026; Tang et al., 2026; Jin et al., 2026). These systems differ in how they search, remember, and review, but a person remains in the loop as the party who decides when a result is acceptable and where to go next. Their reported progress makes the boundary visible: agent success declines as software-task horizons grow (Kwa et al., 2025), and the horizons that matter most are exactly the ones a human cannot sit through.

Three properties must hold before the seat can change hands, and each fails in long-running agents. *Continuity* fails when a growing transcript is compacted or dropped, so the evidence that motivated a revision no longer exists when the revision is proposed. *Acceptance* fails when the component that performs the work also declares it complete. *Experience* fails when only the final artifact survives, so refuted routes cannot later be cited as evidence that an objective is unreachable. Hierarchical and workflow-oriented memory systems address parts of this problem (Packer et al., 2023; Wang et al., 2024b; Xu et al., 2025). A larger context window extends session continuity; taking over the Driver's seat additionally requires explicit state, ownership, and update rules.

Argus (Autonomous Research Generation and Understanding System) implements this as a sequence of *bounded missions* executed against durable project state. Roles operate across three planes—control, execution, and records—so that scheduling, work, and record keeping remain separable even though they participate in one continuous loop. Admitted Stages retain attempted routes, measurements, verdicts, accepted and rejected results, reusable skills, verifiers, and routing decisions. Candidate updates enter persistent state only after the authorized role checks the relevant artifacts and task-native evidence. Model parameters remain fixed, but later missions therefore begin from a changed search policy rather than merely a longer transcript. We call this *verification-gated fixed-model runtime self-evolution*: gated because a candidate becomes reusable only with task-native evidence and an authorized commit, fixed-model because the weights never move.

Does the seat stay empty? The claim is measurable, so we measure it. Across 27 campaigns spanning 1,548 wall-clock hours and 306,691 logged events, Argus raised 38 requests for a human: one every 40.7 hours. Classifying all 38 by what they asked for is more informative than the rate itself. 34% reported broken infrastructure—a failed GPU driver, corrupted container storage, an authentication outage. 26% requested credentials, budget, or authorization, which is the boundary behaving as designed. 18% reported missing context files. Only 13%—5 requests in 1,548 hours—were research judgment, the case where the system had work available and could not decide how to proceed. With work in hand, duty cycle runs 95.1–98.7%, and where it falls short the interruption logs record failed drivers and corrupted storage: **the binding constraint on unattended operation is infrastructure availability, not agent autonomy.**

Do the unattended hours produce anything? Duty cycle alone would only prove an expensive idle loop, so we pair it with delivery. On an SGLang unified-model integration, one engineer working with a turn-by-turn coding agent invested over 60 hours of human time without completing the task; Argus completed the same integration within a 24.14-hour wall-clock envelope and submitted it upstream. On a FlashAttention-4 adaptation carrying six diffusion-LLM mask families across two GPU architectures—a direction that had occupied a specialist infrastructure team for over three months—Argus landed within 5–14% of the native kernel across 27 missions and 225 calls, for under 80 CNY of model spend, requesting a human 2 times in 87.7 hours. Kernels authored by Argus were reviewed by outside maintainers of `flash-linear-attention` and merged with no blocking changes requested—an external check on whether unattended output arrives as reviewed work or as review debt handed back to a human.

Across seven benchmark arenas the same runtime reaches approximately 78% on SWE-Bench Pro against 59% for direct Copilot, 76.8% on AARRI-Bench, and a 28.0 gap on mathematical data synthesis. It also declares 35 SWE-Bench Pro tasks blocked rather than reporting an unsupported success, which is the same gate viewed from the other side: a system that can leave the room is a system that can also refuse to claim it finished.

One task, four questions, six contributions. The task is dense-intelligence work; the four questions are the Driver's; the contributions are these:

C1. The Driver. We identify the previously unautomated role above the harness and decompose it into four nested questions—is this done and any good, what is next, how does the system get better at this, does this need a person—each of which must be owned by a different party for the seat to be delegated safely. This also locates *auto research*: what current systems, ours included, automate is execution, not invention. *Recursive self-improvement* is the same machinery aimed at the production of AI, and inherits every constraint of the general case.

C2. Progress without a score. We show how a runtime iterates where no explicit numeric reward, well-formed objective, or reliable feedback exists—the regime in which self-improvement has previously stalled. Rather than manufacture a score, which would have one system produce both the work and its grade, Argus records progress as a typed frontier transition in which informative failure counts, makes a run that never executed incapable of refuting a hypothesis, names the honest non-terminal outcomes, and scales the evidence bar to the maturity of the claim. We formalize this as evidence-driven rather than goal-driven control—evidence-admitted revision over a report-level contract K_t—so an objective that turns out to be wrong can be departed from on the record, with an author, a precondition, and an audit trail.

C3. Verification as the enabling condition. Independent review is not a quality feature added after execution; separating who works from who may certify is what makes the seat delegable. A read-only Reviewer that cannot edit the evidence it judges, and that may return `blocked`, is what makes the operator's absence acceptable. Of 731 SWE-Bench Pro tasks it withholds completion on 43, 34 of which later pass the official verifier, and declares 35 blocked rather than reporting an unsupported success.

C4. Domain depth without core change. We separate domain expertise from decision authority architecturally. A *vertical* declares what counts as evidence in a field and may raise the evidence bar but never lower it, while the authority boundary and the escalation path live in a core no domain package can reach—zero references to either across all 53,871 lines of vertical code. 24 verticals ship against a 130,362-line core that does not change when a domain is added, the smallest costing 108 lines. What a specialist writes is a seed rather than a ceiling: campaigns running under a vertical promote knowledge and revised checks into its contract, above a declared floor of gates no revision may remove. Self-improvement is scoped along the same seam: a parameter-invariant axis that compounds retained state under a fixed backbone, which this report measures, and a parameter-changing axis, for which we report a from-scratch 1B pretraining stack as the entry condition and state what remains.

C5. Measuring the delegation. We introduce two paired metrics and report them from runtime ledgers: *duty cycle*, the share of available time the system is producing, and *interruption rate*, how often a human is required. Neither is meaningful alone; together they separate a productive unattended system from both an idle loop and a babysat one. Across 27 campaigns, 1,548 wall-clock hours, and 306,691 events: 38 human requests, one every 40.7 hours, of which only 13% are research judgment, at 95.1–98.7% duty cycle with work in hand. The residual gap is dominated by infrastructure failure, not agent hesitation.

C6. What the unattended hours delivered. Against a capability floor across seven benchmark arenas showing the verification machinery does not cost end-task performance, we report delivery where the verifier is least forgiving and the scrutiny most external: six diffusion-LLM mask families carried into a FlashAttention-4 CuTe DSL kernel within 5–14% of native across two architectures for under 80 CNY; an SGLang integration one engineer had not completed in over 60 hours, finished within a 24.14-hour envelope; and 4 kernels accepted by outside maintainers of `flash-linear-attention` with no blocking changes. Case studies in paper production and mathematics show the same machinery retaining falsified routes rather than discarding them.

The remainder of the report follows a conventional research-paper organization. Section 2 positions the system against prior agent and autonomous research work. Section 3 formalizes the operating problem; Section 4 presents the runtime; Sections 5 and 6 describe the evaluation protocol and results; and Sections 7–9 discuss interpretation, limitations, and conclusions. The appendix contains detailed experimental tables and notation.

2 Related Work

Three literatures bear on this work: systems that automate research, runtimes that keep agents alive over long horizons, and the accumulating evidence on whether a model can judge its own output. Read together they show one shape. The executing half of research has been automated repeatedly and well. The deciding half—what to attempt, whether it worked, when to involve a person—has stayed with the operator, and almost no system reports how much of the operator it consumed. That omission is the gap this work addresses, and the rest of this section establishes that it is a gap and not an oversight on our part.

2.1 Autonomous Research Systems

Autonomous research systems range from tightly scoped experiment loops to complete research lifecycles. Karpathy's *autoresearch* repeatedly modifies, trains, and evaluates a model under a fixed compute budget (Karpathy, 2026); AIDE organizes iterative code search for machine-learning improvements (Jiang et al., 2025). FunSearch and AlphaEvolve extend evaluator-guided program search to mathematical, algorithmic, and systems problems (Romera-Paredes et al., 2024; Novikov

et al., 2025). These systems show how a clear objective and an executable evaluator can turn model calls into sustained search.

Recursive's automated research system is the sharpest current demonstration of that principle and reports state-of-the-art results on three benchmarks this report also runs—fixed-budget language-model training, training-speed optimization, and GPU kernel optimization—exceeding our numbers on the first two (Recursive, 2026). Its account of why those benchmarks were chosen is the part we want to draw attention to: they have "clear metrics, relatively low variance, and evaluators that can be hardened against reward hacks." That is the regime in which the Driver's questions are answerable by reading a number, and it is where automated research has advanced fastest. The tasks this report is about are selected the other way, by the absence of that evaluator—an open conjecture, a kernel port whose acceptance criteria are discovered during the port, a materials question whose measurement is itself under construction—and Section 4.5 is about what replaces the number when there is not one. The two lines of work are complementary: one pushes on how far a hardened evaluator can carry a search, the other on what a runtime does before an evaluator exists.

Full-lifecycle systems add problem selection, planning, experimentation, review, and scientific communication. The AI Scientist introduced an end-to-end research pipeline, while AI Scientist-v2 added agentic tree search and produced a workshop-accepted paper (Lu et al., 2024; Yamada et al., 2025). Agent Laboratory organizes specialized roles across the research process (Schmidgall et al., 2025), and CycleResearcher closes the loop between automated research and automated review (Weng et al., 2025). More recently, AutoSci proposed a memory-centric system spanning the complete scientific lifecycle, and FARS reported large-scale deployment of fully automated research workflows across 67 fine-grained AI/ML topics (Qian et al., 2026; Tang et al., 2026).

A complementary line focuses on scientific ideation and collaboration. ResearchAgent iteratively generates and reviews ideas grounded in the literature (Baek et al., 2024); SciAgents combines knowledge graphs with multi-agent reasoning for materials discovery (Ghafarollahi and Buehler, 2024); and Co-Scientist uses a multi-agent generate–debate–evolve loop for biomedical hypothesis discovery (Gottweis et al., 2025). AgentRxiv studies cumulative research through a shared repository of prior work (Schmidgall and Moor, 2025), while Arbor uses a persistent hypothesis tree and isolated executors to refine long-running research programs (Jin et al., 2026).

Read together, this literature supports a blunter characterization than the label suggests. These systems are genuinely strong at retrieving literature, writing code, executing experiments, and producing reports; they are not, on demonstrated evidence, strong at producing the idea that makes a result matter. Ideation components exist and run—Argus has one—but on present evidence they function closer to a placeholder in the pipeline than to a source of scientific advance. **Most of what is currently called auto research is auto execution**, and we include our own system in that assessment.

We state it plainly because it locates what this literature actually delivers, which is substantial and different from how it is usually framed. What is being automated is not invention. It is the human attention that invention has historically been bundled with: a researcher with a good idea spends the overwhelming majority of their time not having it, but configuring environments, debugging harnesses, waiting on runs, rerunning with a corrected parameter, and formatting results. Removing that bundle changes the unit of collaboration rather than the source of ideas—from a person supervising a loop step by step, toward agents dividing work and accepting or rejecting each other's output while the human sets the objective and inspects the result. That shift is what Section 6.2 measures, and it is why we treat duty cycle and interruption rate as load-bearing rather than supplementary.

Nothing in this argument is specific to science. Software engineering is the nearest adjacent case: an industry with extraordinary tooling that still coordinates through documents, meetings, and manual handoffs, and still writes programs a line at a time. The plausible endpoint is the same in both—humans define objectives, design systems, start them, and repair them, while execution, verification, and acceptance are divided among agents. Auto research is the first instance of that pattern rather than its extent.

Argus belongs to this autonomous-research family but targets the runtime layer beneath any single scientific workflow. It maintains a durable campaign, assigns separate authority to Manager, Planner, Engineer, and Reviewer, and carries accepted results, failed routes, tools, skills, and task definitions across bounded missions. The benchmark suite evaluates software engineering, GPU optimization, model training, research tasks, and data synthesis; a separate mathematical vertical trace examines proof-oriented research depth.

2.2 Long-Horizon Agent Runtimes and Memory

Tool-using agents established the basic interaction loop. Toolformer learns when to invoke external tools (Schick et al., 2023), ReAct interleaves reasoning with environment actions (Yao et al., 2023), and SWE-agent and OpenHands expose repository-scale execution through agent–computer interfaces (Yang et al., 2024; Wang et al., 2024a). These systems provide the execution substrate for long tasks; autonomous research additionally requires continuity across many sessions, experiments, and changes of direction.

Persistent-memory systems address this continuity from several angles. Generative Agents combine stored observations with reflection and retrieval (Park et al., 2023); Reflexion retains feedback across attempts (Shinn et al., 2023); Voyager accumulates executable skills (Wang et al., 2023); and MemGPT manages multiple memory tiers (Packer et al., 2023). Agent Workflow Memory induces reusable procedures from past trajectories (Wang et al., 2024b), while A-MEM organizes memories

through agentic linking and retrieval (Xu et al., 2025). Argus combines these ideas with explicit ownership: execution produces candidate updates, review commits the retained form, and the complete event stream remains separate from the bounded working checkpoint.

2.3 Learning from Research Trajectories

Intermediate trajectories are also valuable training material. Process supervision can train stronger mathematical verifiers than outcome-only supervision (Lightman et al., 2023). AgentInstruct generates post-training corpora through agentic flows, Agent-FLAN studies data and tuning strategies for agent behavior, and Agent Lightning converts existing agent trajectories into reinforcement-learning transitions (Mitra et al., 2024; Chen et al., 2024; Luo et al., 2025). Test-time-compute studies further show that search and verification should be allocated according to task difficulty rather than by a fixed sampling rule (Snell et al., 2024). A parallel line closes this loop inside the agent scaffold rather than the weights, and is now large enough to have been surveyed (Gao et al., 2025). Voyager accumulates an executable skill library as procedural memory (Wang et al., 2023), ExpeL extracts reusable natural-language insights from prior attempts (Zhao et al., 2024), Agentic Context Engineering evolves a structured context of accumulated strategies (Zhang et al., 2025b), and the Darwin Gödel Machine rewrites its own agent code under empirical benchmark feedback (Zhang et al., 2025a). Argus differs from these in what gates admission: an update becomes reusable only after a role that did not produce it checks it against task-native evidence, rather than after an aggregate benchmark score moves. Purpose-built corpora for training software agents—SWE-Gym and SWE-smith—show that trajectory data is now a first-class engineering artifact rather than a byproduct (Pan et al., 2024; Yang et al., 2025).

The trajectories retained by Argus connect model outputs to concrete states, tool actions, measurements, artifacts, review decisions, and runtime updates. They therefore serve two purposes: coordinating the next mission online and forming a structured corpus for later supervised, preference-based, or reinforcement learning.

2.4 Recursive Self-Improvement and Where It Meets Auto Research

The prospect that AI systems might contribute to building better AI systems is usually stated as "AI that improves itself," which is vague enough to resist evaluation. The substantive version is narrower: *AI participating materially in the development of the next generation of AI, such that the improved system contributes to the following round*. The loop, not the self-modification, is what makes it recursive. Four directions are pursued in practice: models that study AI and propose algorithms (Lu et al., 2024; Novikov et al., 2025); models that keep evolving at test time, at parameter and non-parameter levels (Snell et al., 2024); AI that generates, filters, and cleans the data training its successor (Mitra et al., 2024); and AI acting as a model-training engineer inside pretraining, post-training, evaluation, and experimental iteration.

The fourth is the one this report addresses, because its causal path to a better model is the shortest and the most checkable. A system that writes a paper about training has not necessarily improved anything; a system that cleans the data, modifies the training code, designs the ablation, reads the loss curve, and changes the schedule has intervened on the actual determinants of model quality. The bar this sets is concrete: baseline competence must approach, and eventually exceed, that of a strong model-training engineer.

Two routes lead there, and they differ in what they assume. The first is recursive in the original sense—have the system research how to improve AI, then use the improved capability for the next round—and it is currently not demonstrable, since no system has been shown to conduct complex training research end to end without human involvement. The second builds the pipeline first and replaces the humans in it progressively: data cleaning, training, post-training, evaluation, and analysis are constructed by people, then handed over stage by stage until the entity operating the code and running the experiments is no longer a person. **The second route is a legitimate and checkable form of recursive self-improvement**: one can ask which stages are automated, how often a human intervenes, and whether the resulting models are better. Argus pursues it, and Sections 6 and 6.12 report its position along that transfer rather than asserting a loop.

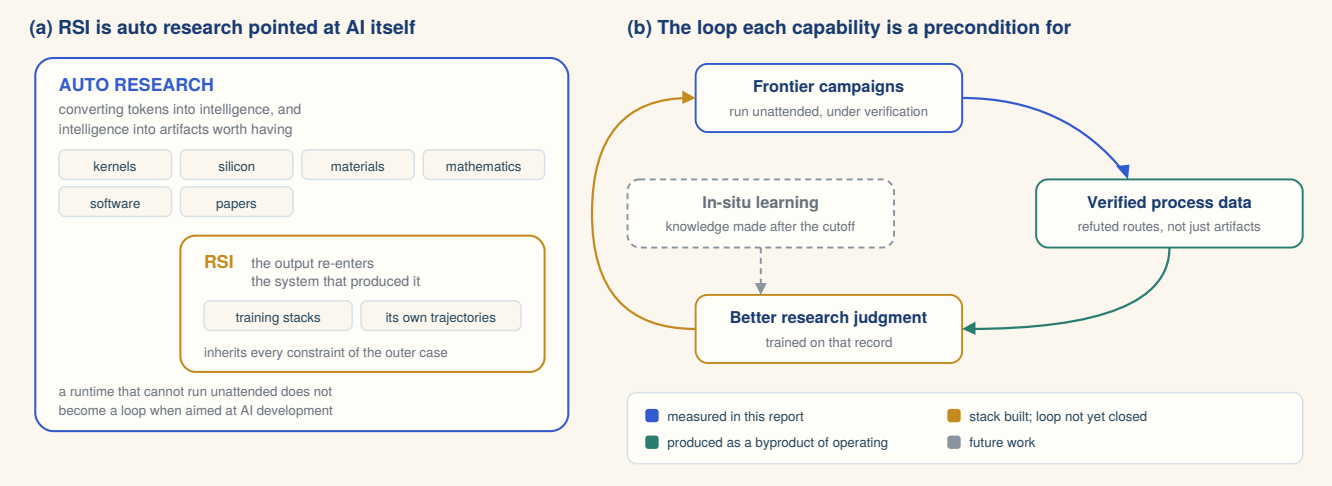

Figure 2. Auto research and recursive self-improvement. Panel (a): RSI is the special case in which the artifact re-enters the system that produced it, so it inherits every constraint of the general case—including the requirement to run without a person in the loop. Panel (b): the two capabilities are preconditions for each other. Operating on frontier problems produces verified process data that no scraped corpus contains; that record is what a training stage would consume; and a model with better research judgment conducts better campaigns, including campaigns about training. Colour indicates what this report establishes and what remains.

The relationship to the preceding subsections is containment. Auto research is the general machinery for converting intelligence into results wherever invention is required; recursive self-improvement is that machinery pointed at the production of AI itself, so that the output re-enters the system that produced it. Two consequences follow. Recursive self-improvement inherits every constraint of the general case: a runtime that cannot run unattended, judge its own completion, or choose its next step does not become a loop when aimed at AI development—it becomes a supervised pipeline with additional steps. And the coupling runs through the trajectory record described above, which is simultaneously the mechanism by which a fixed-backbone runtime improves its own state and the raw material a training stage would consume. A system doing auto research under verification accumulates, as a byproduct, what the third and fourth directions require.

2.5 Why Verification Must Be Independent

The design choice this report treats as load-bearing—that the party judging completion must not be the party that produced the work, and must not be able to edit the evidence it judges—is supported by a converging body of negative results. Large language models do not reliably repair their own reasoning when asked to reconsider without external signal (Huang et al., 2024), and models acting as judges exhibit measurable self-preference: they recognize their own outputs and score them above equivalent alternatives, with the bias tracking self-recognition ability (Panickssery et al., 2024; Wataoka et al., 2024). Surveys of the LLM-as-judge literature catalogue position, verbosity, and leniency biases alongside it (Gu et al., 2024). Where an explicit objective is available, optimizing against an imperfect proxy invites the failure mode formalized as reward hacking, in which the measured score improves while the intended property does not (Skalse et al., 2022). Empirical study of multi-agent LLM systems finds that a substantial share of failures are organizational rather than cognitive—unclear role boundaries, unverified handoffs, and premature termination—rather than reasoning errors by any single component (Cemri et al., 2025).

These results motivate three properties of the runtime described in Section 4: the Reviewer runs read-only, so self-preference cannot be converted into edited evidence; the four-state evidence model refuses to let an environment failure count as a refuted hypothesis; and the proof ledger caps model judgement below mechanical verification regardless of how many judgements agree. None of these eliminate the underlying biases. They constrain what those biases can do to persistent state.

2.6 Evaluation of Research Agents

Software and general-agent benchmarks established repository-level issue repair and multi-environment analytical evaluation as core Agent capabilities (Jimenez et al., 2024; Liu et al., 2023c; Ma et al., 2024). Research-agent benchmarks increasingly test complete research behavior rather than isolated question answering. MLE-bench packages Kaggle competitions as machine-learning engineering tasks (Chan et al., 2024); RE-Bench compares AI agents with human experts in open-ended research engineering (Wijk et al., 2024); AARRI-Bench evaluates granular tasks from the research lifecycle (Wang et al., 2026); and AIRS-Bench targets frontier research-science agents across multiple disciplines (Lupidi et al., 2026). Domain-specific harnesses sharpen this further: KernelBench measures whether generated GPU kernels are both correct and faster than a reference (Ouyang et al., 2025), VerilogEval evaluates hardware description code (Liu et al., 2023b), and Terminal-Bench evaluates agents on realistic command-line work where the environment rather than a prompt defines success (Merrill et al., 2026). Follow-up work on MLE-bench analyzes how search and exploration strategy determine agent performance on machine-

learning engineering (Toledo et al., 2025), and MLGym packages full ML research cycles—model design, training, optimization, and dataset curation—as agent tasks (Nathani et al., 2025). Together they motivate evaluation in task-native units, with explicit resource accounting and trajectory-level analysis.

This report combines that benchmark view with a continuous system trace. The benchmark suite measures outcomes across seven arenas, while the SWE-Bench Pro and mathematical studies expose runtime evolution, review-driven correction, and cross-mission research progress.

2.7 What Is Not Measured: the Cost of Human Supervision

Across this literature, evaluation is reported in outcomes—resolved issues, medals, scores, accepted papers—and increasingly in Tokens and wall-clock time. What is almost never reported is how much human attention the run consumed. RE-Bench is the closest, comparing agents against human experts on matched research-engineering tasks (Wijk et al., 2024), but it measures humans as an alternative to the agent rather than as a resource the agent consumes while running. A system requiring an operator every few minutes and a system running unattended for two days can post identical benchmark numbers.

This omission is understandable, because for most of this literature the answer is structurally the same: a person supervises the loop. It nonetheless hides the constraint that binds long-horizon deployment. An agent whose effective operating window is bounded by its operator's attention cannot exceed that operator's waking hours regardless of its accuracy, and horizon-scaling studies show task success degrading precisely as horizons grow past that window (Kwa et al., 2025). We therefore report duty cycle and interruption rate alongside task outcomes (Section 6.2). We are not aware of prior agent systems reporting either. Both belong in any evaluation of long-horizon autonomy: an outcome obtained under continuous supervision and the same outcome obtained unattended are different results.

3 Problem Formulation

The hard part of this task class is not executing the work. It is that there is no number to steer by. Everything that makes the Driver's seat automatable under an explicit numeric reward—rank the options, check the score, stop when it plateaus—is unavailable exactly where research begins. This section states the task shape that creates that condition, and the quantities a runtime can still act on once the scalar is gone.

3.1 Dense-Intelligence Tasks

The public Argus formulation defines a *dense-intelligence task* as one that sustains high-frequency reasoning, tool use, verification, and iteration across a continuous time window until it produces a measurable result (Argus Team, 2026). Three conditions determine whether a task has this shape:

T1. Invention. The answer is not directly retrievable from a manual or database; it must be discovered through search and feedback.

T2. Long horizon. One pass is insufficient. Progress requires many dependent iterations over hours or days.

T3. Constructible verification. Evidence that discriminates a genuine result from a persuasive but incorrect one can be *obtained*—by running something, measuring something, or checking something against the world. It need not exist when the campaign starts.

The defining loop is therefore proposal → execution → measurement → revised proposal. Performance engineering, security research, scientific search, and quantitative research differ in domain but share this operational structure. The task definition excludes ideation that no evidence could ever settle, and fixed workflows that require time but no new decisions.

Two of these conditions explain why such tasks have historically required a human in the loop. Because the answer must be invented rather than retrieved (T1), no fixed plan survives contact with the evidence, so someone must keep deciding what to attempt next. Because progress spans many dependent iterations (T2), that someone must keep deciding for hours or days. The result is a supervision load that scales with the horizon precisely where the horizon matters most. Condition T3 is what makes the load transferable: if discriminating evidence can be obtained, the runtime has a basis for judging its own output that does not reduce to self-assertion.

T3 is deliberately weaker than the requirement of a ready evaluator, because that requirement would exclude most research. A campaign frequently opens with a question that is not yet well posed—which mask families a kernel must carry, whether an open conjecture admits a proof at all, what property of a candidate material would even settle the matter—and the first several missions do not optimize anything. They establish what would count as an answer. *Constructing the evaluator is part of the work, not a precondition for starting it*, and this is the sharpest reason control must be evidence-driven rather than goal-driven: a goal-driven system needs its target specified in advance, which is exactly what an underdefined problem cannot supply. An evidence-driven one can begin with a premise, discover in week two that the question was wrong, and revise the objective on the record—which is what the campaigns in Section 6 repeatedly do. What T3 rules out is not the underdefined problem; it is the problem where no amount of work could ever produce discriminating evidence, since there the runtime's only remaining basis for a completion claim is its own assertion. *Dense-intelligence tasks are therefore both the tasks that most demand a*

Driver and the only tasks where the seat can responsibly be delegated.

The name states a requirement, not a preference. A task defined by continuity cannot be discharged by a supply that is not continuous: run a dense task off a discrete source and what actually gets executed is a sequence of restarts, each paying again for context the previous burst had already earned. This is why the duty cycle of Section 6.2 is a correctness property of the arrangement rather than an efficiency statistic. It reports whether the intelligence supplied to the task had the shape the task requires. Sections 4 and 6 develop that delegation and measure whether the seat stays empty.

This section separates two kinds of formal statement, and the distinction matters more than any individual equation. Equations 10, 11, 12, 13, 14, and 20 define quantities and invariants this report measures or mechanically enforces; every term in them is reported in Section 6. Equation 15 is measured in one direction—how much of what the runtime did leaves no trace in what it produced—while its decision-theoretic strengthening is not. The remainder—density, research value, typed compression, reuse value—are definitions that make claims falsifiable without yet being tested here, and we mark each as such at the point it appears. A reader who wants only what we can defend with data should read the first group and skip the second.

We model a campaign by an objective o, an initial artifact y_0, a resource budget B, and an evidence structure $\mathscr{E}$. The artifact may be code, a model, a dataset, an experimental harness, a proof, or a manuscript. We write $\mathscr{E}$ rather than an evaluation function deliberately. A scalar evaluator $f : y \mapsto \mathbb{R}$ is available in some campaigns—a benchmark verifier, a latency measurement—and absent in most of the ones this report is about, where the right measurement is itself under construction. The formalism must therefore not presume one. $\mathscr{E}$ is whatever the task natively supports: a scalar where one exists, and otherwise a typed record of what each attempt established, which Section 4.5 specifies and Section 3.8 orders. The campaign is divided into bounded missions indexed by t, each producing

$$\tau_t = \big((s_{t,j}, a_{t,j}, y_{t,j}, e_{t,j}, r_{t,j})\big)_{j=1}^{n_t}, \tag{1}$$

where round j has state $s_{t,j}$, actions $a_{t,j}$, resulting artifact state $y_{t,j}$, observed measurements $e_{t,j}$, and recorded review or admission outcome $r_{t,j}$. A mission must terminate, pause, or transfer control explicitly; continuity belongs to the persistent campaign state rather than an unbounded model session.

Report-level contract model

For analysis, we summarize the working task as $K_t = (\iota, o_t, c_t, v_t)$: standing intent, current objective, known constraints, and verification criteria. X_t denotes only the clarifications, priorities, and unresolved questions made explicit at the user interface. A material refinement is written compactly as

$$(X_{t+1}, K_{t+1}) = \mathrm{ManagerAdmit}(X_t, K_t, K_t', e_t, r_t, u_t),$$

where K_t' is the proposed contract, e_t is evidence, r_t the recorded admission outcome, and u_t the authorized operator or Manager decision. `ManagerAdmit` is a normative operator, not one atomic production API: it projects over current `GoalContract` revisions, persisted operator questions, Planner replanning, Manager Stage/routing transitions, and append-only provenance.

Two conditions distinguish this from an ordinary goal-update rule, and they are what the word *evidence-driven* is meant to carry. First, authority may refuse a revision but may not erase what prompted it: for all inputs, e_t appears in the record of round t regardless of the value of u_t. A contradiction that has been observed cannot be made to un-exist by declining to act on it, so a rejected revision leaves a standing obligation rather than nothing. Second, the standing intent is the only fixed point. Writing o_{t+1} for the objective component of K_{t+1},

$$e_t \text{ refutes } o_t \ \wedge\ \iota \text{ preserved by } o' \implies o_{t+1} \neq o_t \text{ is admissible},$$

whereas no evidence licenses a change to ι. A goal-driven system is the special case in which the left conjunct is never allowed to fire; it defends o_t because o_t was written down first. The asymmetry between ι and o_t is therefore the formal content of the distinction: what the user wants is stable, and what the campaign is currently trying to build is a hypothesis that evidence may overturn.

Figure 3 contrasts a session-limited trajectory with the persistent runtime model. A session-limited agent eventually spends its budget on reconstruction and repeated work. Argus instead instantiates a recurrent Manager–Planner–Engineer–Reviewer loop inside each campaign Stage. Reviewed state persists below the loop, while Manager-controlled advance, rollback, and rejected branches change the route taken by later missions.

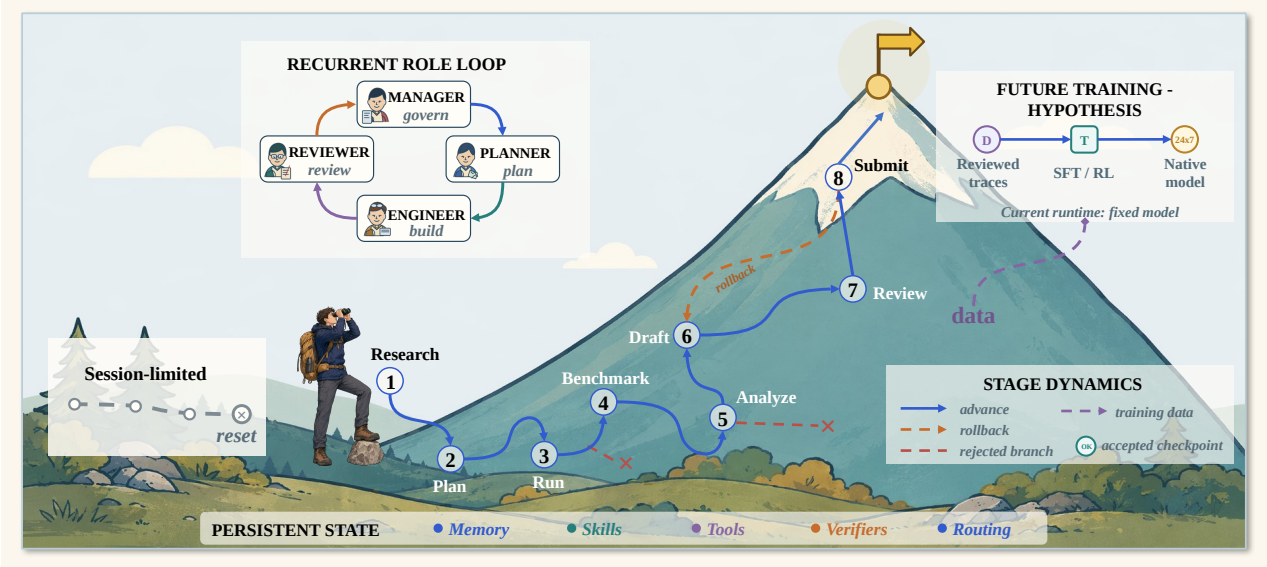

Figure 3. Operational model of long-horizon runtime self-evolution. The left card shows a session-limited route ending in reset. The central recurrent role loop is reused across eight Manager-controlled research Stages, while the blue route, orange rollback, and red rejected branches show that progress need not be monotone. Certified knowledge, tools, verifiers, and routing persist across missions. The purple path to future supervised fine-tuning and reinforcement learning (SFT/RL) is a research hypothesis; the current runtime changes persistent state while keeping model parameters fixed. Geometry is conceptual rather than an empirical scale.

3.2 Role-State and Stage Dynamics

The roles are not four sequential stations that execute once. Within a stage, the campaign repeatedly applies the transition relation

$$M \to P \to E \rightleftarrows R \to M, \tag{2}$$

where M, P, E, and R denote Manager, Planner, Engineer, and Reviewer. Reviewer continuation returns work to Engineer; an accepted or blocked verdict returns control to Manager; and a Manager hold or rollback reopens planning. One further edge matters for the argument of Section 7.6: a Reviewer may return `replan_requested` rather than a verdict on the work, which routes to the Manager as a challenge to the mission itself rather than to its execution. The Manager then keeps, revises, or replaces the plan, and the Planner authors any replacement. Without that edge the relation is a cycle in which every later, better-informed role can only accept or reject work performed under an earlier decision it cannot question. The relation is an explanatory state-machine model of the implemented control flow, not a claim that every transition must invoke a fresh model session.

Let g_n be the current stage after the nth Manager decision. Legal transitions are

$$g_{n+1} \in \{g_n, \mathrm{next}(g_n)\} \cup \mathrm{prev}(g_n), \tag{3}$$

corresponding to hold, advance to the immediate next stage, or rollback to an earlier stage. For the research vertical, the ordered stages are research, plan, benchmark, run, analysis, draft, review, and submission. Stage index and task-native quality need not improve on every cycle: experiments can fail, review can reject a branch, and later measurements can invalidate earlier assumptions. The intended ascent is progress in the accepted frontier over many cycles, not monotonic improvement at each transition.

3.3 Dense-Intelligence Density

We express useful intelligence per unit wall-clock time as

$$\rho_I(T) = \frac{1}{T}\int_0^T \dot{N}_{\mathrm{tok}}(t)\,\eta_r(t)\,\eta_a(t)\,\eta_v(t)\,dt. \tag{4}$$

$\dot{N}_{\mathrm{tok}}(t)$ is the instantaneous rate of token-mediated reasoning and action. The factors $\eta_r(t)$, $\eta_a(t)$, and $\eta_v(t)$ represent the fractions converted into relevant reasoning, effective action, and valid verification. The product is intentionally stricter than token throughput: a run that repeatedly reads the same state, emits activity without changing an artifact, or accepts unchecked claims can consume many tokens while contributing little to ρ_I.

Operational efficiency factors. The three factors describe how work is distributed within an analysis window. For W, let $\mathscr{U}_k(W)$ be the attributable units for $k \in \{r,a,v\}$: observable reasoning segments, executed actions, or verification segments. We define

$$\eta_k(W) = \frac{\sum_{u\in\mathscr{U}_k(W)} w(u)q_k(u)}{\sum_{u\in\mathscr{U}_k(W)} w(u)}, \qquad 0 \le q_k(u) \le 1, \tag{5}$$

where $w(u)$ is the attributable Token count for reasoning and verification, and either one action or its measured cost for action efficiency. The attribution label $q_k(u)$ records conversion into the intended function:

- **Reasoning efficiency η_r.** A direct unit derives, revises, or rules out a named claim, lemma, blocker, or decision. Repeated context, narrative transitions, and notation-only rewriting are transition cost.
- **Action efficiency η_a.** A direct action changes an artifact, result set, accepted decision, or explored branch. A falsification action is productive when it certifies a dead branch; retries and interrupted no-op work are tracked separately. State synchronization and packaging form the auxiliary action category.
- **Verification efficiency η_v.** A direct verification unit compares a named claim with an explicit criterion or artifact, reruns an independent check, identifies a concrete defect, or issues a scoped verdict. Schema repair, reformatting, and checkpoint canonicalization enable review but belong to verification overhead rather than direct checking.

We report a direct-work value and an inclusive value that also counts successful coordination and state maintenance. Equation 4 uses the three factors as a multiplicative bottleneck model. Section 6.8 measures them in the mathematical campaign, and Appendix D gives the full calculation.

3.4 Online Life and Runtime Evolution

The second object of the public formulation separates model parameters from the operating state around the model:

$$\begin{aligned} H_{t+1} &= U(H_t, \tau_t, E_t, K_{t+1}), \\ \theta_{t+1} &= \theta_t. \end{aligned} \tag{6}$$

with the state definition

$$H_t = \left\{ \underbrace{\text{Knowledge}}_{\text{wiki}+\text{skills}}, \text{Memory}, \text{Tools}, \text{Verifiers}, \text{Routing} \right\}. \tag{7}$$

Knowledge and *Memory* are separated deliberately, and this report treats the distinction as load-bearing. **Knowledge** is what the runtime has established and may reuse: a wiki of source-linked pages recording what a domain turned out to be like, and a versioned library of executable skills. It is admitted only through certification, it is scoped to where it was shown to hold, and it is intended to outlive the campaign that produced it. **Memory** is what the runtime needs in order to keep working: the append-only event journal, the mission backlog, durable artifacts and files on disk, and the per-role session context carried across a rolling window. Memory is written as a byproduct of running and is not certified, because it records what happened rather than what is true.

The separation matters because the two fail differently. Memory loss is an availability failure—a lost context file costs a mission its bearings, which Section 6.2 shows as interruptions and lost duty cycle—while uncertified knowledge is a correctness failure that propagates into every later campaign that reuses it. They therefore warrant different gates: memory must be durable, and knowledge must be earned. A system that conflates them either certifies its scratch space or trusts its conclusions the way it trusts a log line.

E_t is the subset of results admitted from mission t. The equality for θ is a scope condition: online evolution does not require a gradient update to the underlying model. Section 4 refines H_t into named ownership surfaces, including task/evaluation definitions Q_t; subsequent updates treat Q_t as a component of H_t. The contract argument records the admitted report-level projection. The update U is partial; many missions write only memory, and some change no reusable component.

3.5 Capability-Set Expansion

Let C_t denote the effective capability set available at time t. Admission into it is gated rather than automatic:

$$C_{t+1} = C_t \cup \{c : \text{Verify}(c, E_t) \ge \varepsilon\}, \tag{8}$$

so a capability enters only with evidence E_t clearing the threshold its surface requires. We deliberately do not accompany this with monotonicity conditions on frontier width or per-domain depth. Such inequalities would be false: skills go stale, verifiers are revised downward, and performance regresses on harder instances—and a formal model that asserts otherwise is contradicted by the same report's own longitudinal data, where a late window rebounds in both cost metrics. Growth of C_t is a hypothesis to be measured per campaign, not a property of the construction. The gate in Equation 8 constrains only what may

enter, which is the part the runtime actually controls. A negative result can still add value by excluding a failed branch even when C_t does not grow at all.

3.6 Research Value

Not measured as defined. The terms below are not separately estimated in this report; the de-duplicated artifact inventory of Section 6 is a count, not an instantiation of this integral.

Dense activity matters only when it produces new, valid, reusable information. The website therefore defines the accumulated research value over horizon T as

$$V_R(T) = \int_0^T \rho_I(t)\,\Delta I(t)\,p_{\text{valid}}(t)\,\eta_{\text{reuse}}(t)\,dt, \tag{9}$$

where $\Delta I(t)$ is information gain, $p_{\text{valid}}(t)$ is the probability that the claimed increment survives review, and $\eta_{\text{reuse}}(t)$ is the fraction that remains useful for later work. Equation 9 separates Token volume from the properties that make a result useful: information gain, validity, and reuse. The experiments measure these components in their native task units.

3.7 Unattended Operation: the Two Quantities We Measure

The quantities defined below are the ones this report measures directly, from ledgers the runtime writes during ordinary operation. We state them here, with the rest of the formalism, rather than only at the point of use, because they are the report's primary empirical claim and because their definitions contain the choices a reader should be able to contest.

Fix a campaign with wall-clock span $W = [w_0, w_1]$. Let $\{[s_i, c_i]\}$ be the model-call intervals recorded in the settled usage ledger, and let $U \subseteq W$ be their union. Let $G \subseteq W$ be the time in which no work was available—an empty backlog awaiting assignment—or the execution environment was unusable. Duty cycle is the occupied fraction of the time that remains:

$$\text{DutyCycle} = \frac{|U|}{|W| - |G|}. \tag{10}$$

Three choices in Equation 10 are load-bearing and each is contestable. First, U is a union of call *intervals* rather than a count of log events: a single reasoning call can exceed two hours, during which the event stream is legitimately silent, so any silence threshold would score deep reasoning as idleness. Second, external work—synthesis, static timing, a training run—consumes no Tokens and appears in neither U nor G, so it is neither credited nor penalised; a campaign dominated by external jobs will report a low duty cycle for a reason that has nothing to do with autonomy. Third, subtracting G answers "how well does it use time it could have used" rather than "how much of the calendar did it occupy"; we report the undivided form, $|U|/|W|$, alongside it, and the two differ substantially.

Let $\mathscr{A}$ be the set of system-initiated requests for a human—events the runtime emits when it will not proceed without an answer. The interruption rate is

$$\text{IR} = \frac{|\mathscr{A}|}{|W|}, \qquad \text{reported as its reciprocal } |W|/|\mathscr{A}| \text{ in hours.} \tag{11}$$

The membership condition is what makes IR meaningful. An operator who volunteers a status question is not in $\mathscr{A}$, because the runtime was not waiting on them; conversely a request that blocks progress counts even if the operator never answers it. This is measurable precisely because the runtime must already distinguish the two cases in order to decide whether to keep working.

Neither quantity is interpretable alone, and the failure modes are symmetric: a system that does nothing achieves $\text{IR} = 0$, and a system looping uselessly achieves $\text{DutyCycle} = 1$. Reported together they bound the case of interest. We therefore also partition $\mathscr{A}$ by what was asked, since a request the runtime should not have needed to make and a request no autonomy improvement could eliminate are different facts:

$$\mathscr{A} = \mathscr{A}_{\text{infra}} \sqcup \mathscr{A}_{\text{auth}} \sqcup \mathscr{A}_{\text{ctx}} \sqcup \mathscr{A}_{\text{judg}} \sqcup \mathscr{A}_{\text{defect}}, \tag{12}$$

for broken infrastructure, operator-owned authority, missing context, research judgment, and framework defect respectively. Only $|\mathscr{A}_{\text{judg}}|/|\mathscr{A}|$ measures the thing the Driver framing is about—work was available and the runtime could not decide how to proceed. The authority share is bounded below by design rather than by capability, since Section 4 makes those escalations mandatory, and the infrastructure share is a property of the deployment rather than of the agent. A single aggregate interruption rate conceals all three distinctions, which is why we report the partition and not only the rate.

3.8 Ordering Progress Without a Scalar

If no evaluator f is available, the runtime still has to answer whether the last round advanced the campaign. Replacing f with a model-produced score reintroduces the party whose work is being scored; Section 2 records why that is unreliable. We therefore give $\mathscr{E}$ the weakest structure that supports the decisions the loop actually makes: not a total order, but a partial one.

Each round closes with a typed transition $\delta \in \Delta$ over the mission frontier, where Δ is the fixed vocabulary of Section 4.5. Write $\Delta^+ \subset \Delta$ for the transitions that count as forward motion. The essential property is that Δ^+ is *not* the set of transitions that improve an artifact:

$$\Delta^+ = \{\texttt{artifact\_improved}, \texttt{risk\_reduced}, \texttt{uncertainty\_reduced}, \texttt{information\_gain}, \texttt{bounded\_regression}, \texttt{recovered}\}, \tag{13}$$

so a round that refutes a route, or that accepts a bounded and budgeted cost in exchange for retiring a risk, advances the campaign exactly as a round that produced a faster kernel does. This is measurable, and the measurement is the reason the vocabulary exists rather than a single score. Across the campaigns of Section 6.2 the runtime recorded 604 frontier transitions, of which 598 fall in Δ^+. Only 167 of those—27.9%—are `artifact_improved`. **The remaining 72.1% of recorded progress consists of reduced uncertainty, information gained from failure, retired risk, and bounded repair**: real advances that a scalar objective registers as zero or as loss. What Δ^+ excludes is repetition without change (`unchanged_failure`) and regressions that are unbounded or unexplained. A campaign is therefore progressing when its frontier keeps moving through Δ^+, whether or not any measured quantity has improved—and this is the ordinary case in research, not an edge case. Under a scalar objective the elements of Δ^+ other than `artifact_improved` are invisible, which is the precise sense in which a score-driven runtime is blind to most of what a campaign produces.

The refutation invariant. An evidence record is a triple (x, ϕ, σ): an execution status $x \in \{\texttt{completed}, \texttt{blocked}, \texttt{failed}\}$, a failure class ϕ, and a hypothesis status $\sigma \in \{\texttt{untested}, \texttt{inconclusive}, \texttt{supported}, \texttt{refuted}\}$. Each vertical declares a set $\Phi_\neg$ of failure classes that carry no information about the premise—absent toolchains, unavailable devices, denied permissions, underpowered pilots. The runtime enforces

$$\phi \in \Phi_\neg \implies \sigma = \texttt{untested}, \tag{14}$$

which is checked mechanically rather than left to the judgement of whichever role writes the record—the validator rejects the record instead of trusting the author, so the invariant cannot be violated by a role that would prefer a cleaner story. We report it as an enforced property of the implementation rather than as a measured rate: the campaigns of Section 6.2 ran under verticals that do not emit the four-state record, so we have no field data on how often the invariant binds, only the guarantee that where it applies it cannot be bypassed. The invariant is what makes an unattended campaign safe to run against a flaky environment: without it, a system that cannot distinguish "the experiment did not run" from "the idea is wrong" will discard correct hypotheses at the rate its infrastructure fails, and the resulting record will be confidently and permanently wrong. Equation 14 costs the runtime nothing when the environment is healthy and is the difference between a usable and a poisoned frontier when it is not.

Evidence sufficiency is relative to the claim. The bar $\mathscr{E}$ must clear is not fixed. Let $\Pi = (\texttt{explore}, \texttt{develop}, \texttt{certify})$ be ordered by strictness. A mission at `explore` asks only whether the premise is real and worth another probe; `certify` asks whether the result can be published as it stands. The profile is derived from the campaign stage, a final-submission scope forces `certify`, and any change that lowers the bar requires explicit confirmation. Formally the admissibility predicate is indexed by profile, Adm_π with $\pi \in \Pi$, and is monotone in strictness: $\pi \preceq \pi'$ implies $\mathrm{Adm}_{\pi'} \subseteq \mathrm{Adm}_\pi$. A single global threshold cannot hold both ends of a campaign—set at `certify` it stops exploration, set at `explore` it admits unsupported claims—which is why the profile is a parameter of the gate rather than a constant of the system.

3.9 The Information Advantage of Process Data

The inclusion in Equation 15 is strict by a measurable margin in these campaigns. Of 1,852 bounded missions recorded across them, 1,506 reached a `done` verdict and 346—18.7%—did not, including 103 that the runtime explicitly declared blocked. Those missions produced no accepted artifact and are therefore absent from D_{final} entirely, while their objectives, attempts, measurements, and verdicts remain in D_{process}. Nearly a fifth of what the runtime did is invisible to any record of what it produced.

The decision-theoretic reading below is not measured. It bounds what retention could be worth; nothing in it is an empirical claim about Argus.

Finally, we distinguish the accepted artifact from the trajectory that produced it:

$$D_{\text{process}} = \{(s_k, a_k, e_k, r_k, \Delta H_k)\}_{k=1}^{N} \supsetneq D_{\text{final}} = \{y^\star\}. \tag{15}$$

The process record contains states, actions, measurements, review feedback, and runtime-state changes, including failed branches omitted from $y^\star$. This additional information can reduce repeated search, guide later analysis, and provide grounded material for later skill construction or evaluation. Argus retains typed, privacy-preserving observations and verdicts rather than private chain-of-thought transcripts.

The set inclusion in Equation 15 has a standard decision-theoretic reading, which we state because it fixes what the claim is *not*. Let P denote a typed process record and let $Y = g(P)$ be its final-artifact projection. For a downstream task q with target Z_q, loss ℓ_q, and decision policy π, define the minimum achievable risk

$$\mathscr{R}_q(X) = \inf_{\pi} \mathbb{E}[\ell_q(\pi(X), Z_q)]. \tag{16}$$

Remark 1: process-data dominance is an upper bound, not a result

If $Y = g(P)$ then $\mathscr{R}_q(P) \le \mathscr{R}_q(Y)$ for every downstream decision problem q, since any policy using Y is reproducible from P by first applying g. This is the data-processing inequality under the Blackwell ordering (Blackwell, 1953) and it is true by construction; we label it a remark rather than a proposition because nothing here is being proved about Argus. Strictness is likewise not established in general—it holds for a given q only when two records with the same projection imply different optimal actions, which is a statement about the task family, not about retention.

What the remark does is locate the burden. Retaining the trajectory cannot hurt informationally, so the interesting question is entirely whether a bounded, typed projection of it preserves enough to change later behaviour at a cost worth paying. That is an empirical claim about ψ below and about the retained skills, verifiers, and refuted routes of Section 4—not something the inequality settles.

Dominance is informational, not computational. A raw trajectory can be too large, stale, or contradictory to use efficiently. For a context budget b, the relevant object is therefore a typed compression $\psi(P)$ that trades downstream risk against retrieval and validation cost:

$$\psi_b^\star = \arg \min_{\psi:\, \mathrm{size}(\psi(P)) \le b} \mathbb{E}_{q\sim\mathscr{D}} \left[\mathscr{R}_q(\psi(P)) + \lambda_c C_{\mathrm{read}}(\psi(P))\right]. \tag{17}$$

Equation 17 is a specification, not a construction: the minimizer ranges over all compressions and is not computable. Its use here is to name the trade-off the runtime is making, and in particular to show that the two retention surfaces cannot be collapsed into one. The unbounded term $\mathscr{R}_q(\psi(P))$ is minimized by keeping everything, which the append-only event tape does; the bounded constraint $\mathrm{size}(\psi(P)) \le b$ together with the read cost C_{read} is what forces a second, lossy surface, which the reviewed checkpoint is. A system with only a tape pays unbounded read cost at every mission start; a system with only a checkpoint cannot recover the evidence that justified a past decision when a later one contradicts it. Argus keeps both because the objective has two terms, and the selection criterion follows: a failed branch belongs in the bounded surface when it changes the next optimal action, not merely because it occurred.

3.10 Reusable State and Compounding Intelligence

Not measured as defined. The quantity below is a counterfactual; what Section 6 reports is a confounded observational proxy, and the two are kept under separate names for that reason.

Process data compounds only when an admitted state update improves later work. Let $H_t \oplus \Delta H_t$ denote the state after accepting mission t, and let $q_{t+1:t+L}$ be the next L tasks. We define the discounted reuse value

$$G_L(\Delta H_t) = \sum_{j=1}^{L} \gamma^{j-1} \left[\mathscr{R}_{q_{t+j}}(H_t) - \mathscr{R}_{q_{t+j}}(H_t \oplus \Delta H_t)\right]. \tag{18}$$

$G_L > 0$ means that the accepted wiki page, skill, verifier, routing rule, or certified dead branch reduces future loss under a prespecified task distribution. $G_L < 0$ captures negative transfer. This definition turns "compounding intelligence" from monotone rhetoric into a counterfactual claim: reuse must outperform a frozen state on matched future tasks. We are explicit that this report does not measure it. What Section 6 reports is a startup-versus-mature difference across waves of one sequential run, which shares a sign with G_L but is confounded by task composition and ordering; we label that quantity $\hat{G}_{\mathrm{obs}}$ and keep it separate. Obtaining G_L requires replaying matched tasks against frozen and accumulated state, which we have not run. Workflow-memory experiments show that induced reusable routines can improve success and reduce steps (Wang et al., 2024b). Section 7 connects this quantity to the current Argus traces.

The density and research-value equations can then be joined into a protocol-specific *verified reusable yield*:

$$\mathscr{Y}_{\mathrm{PRI}}(W) = \frac{\sum_{t\in W} p_{\mathrm{valid},t} \left[\Delta I_t + \lambda_g G_L(\Delta H_t)\right]}{\sum_{t\in W} N_{\mathrm{tok},t}}. \tag{19}$$

ΔI_t is immediate task-native information gain, such as a score improvement, theorem strengthening, or certified branch elimination; $p_{\mathrm{valid},t}$ is estimated by an external evaluator or a calibrated review procedure; and G_L measures future reuse. λ_g converts immediate and future value into one protocol-specific unit. Equation 19 connects immediate research output to future reuse. The current substitutions are reported in Appendix D.

3.11 Review as Selective Error Correction

The observable structure of review is a nested funnel, and stating it as one prevents a arithmetic error the counts otherwise invite. Over a task population $\mathcal{T}$, routing partitions it into independently reviewed and self-reviewed work, $\mathcal{T} = \mathcal{T}_R \sqcup \mathcal{T}_S$. The routed set partitions again by first verdict into accepted, revision-requested, and blocked, $\mathcal{T}_R = \mathcal{T}_{\text{acc}} \sqcup \mathcal{T}_{\text{rev}} \sqcup \mathcal{T}_{\text{blk}}$. Recovery is then measured *inside* $\mathcal{T}_{\text{rev}}$ rather than beside it:

$$\mathcal{T}_{\text{strict}} \subseteq \mathcal{T}_{\text{pass}} \subseteq \mathcal{T}_{\text{rev}}, \tag{20}$$

where $\mathcal{T}_{\text{pass}}$ are the tasks that subsequently clear the official verifier and $\mathcal{T}_{\text{strict}}$ those that additionally complete the `continue`→revision→`done` loop. The inclusions are the point: verifier pass and strict rescue are milestones within the revision-requested group, not sibling outcome classes, so their counts must not be added to the partition above. Section 6 reports every term of both Equation 20 and the partition preceding it, which is what makes the funnel checkable rather than a summary statistic.

What the funnel cannot show is the other side of the channel. Let C denote whether a proposed state increment is correct, A whether the Reviewer accepts it, $p = \Pr(C = 1)$ the proposal base rate, $\alpha = \Pr(A = 1 \mid C = 1)$ Reviewer sensitivity, and $\beta = \Pr(A = 1 \mid C = 0)$ false-acceptance rate. Bayes' rule gives

$$\Pr(C = 1 \mid A = 1) = \frac{\alpha p}{\alpha p + \beta(1-p)}. \tag{21}$$

If $\alpha > \beta$, accepted state is more precise than the proposal stream and review acts as a selective error-correction channel; its operating point trades accepted precision against recall and Token cost. We report the identification problem rather than the estimate. The observed data give the recovery side of this channel—of the 43 tasks on which the Reviewer withheld completion, 34 subsequently passed the official verifier—which bounds sensitivity from below on rejected work but says nothing about β, because a false acceptance is by construction not re-examined. Estimating β requires randomized routing with an external grader applied to accepted work as well as rejected work, which we have not run and list in Section 8. Equation 21 is therefore stated to make explicit which term the present evidence does *not* constrain.

3.12 Role-Resolved Vertical Traces

Endpoint metrics do not show how an Agent system actually conducts research. We therefore project each mission trajectory in Equation 1 onto the four role-owned actions

$$\phi(\tau_t) = (m_t, p_t, x_t, r_t, \Delta H_t), \tag{22}$$

where m_t is the Manager's objective or stage decision, p_t is the Planner's bounded task and dependency decision, x_t is the Engineer's artifact-producing execution, r_t is the structured completion decision together with its source, and ΔH_t is the durable state update exposed to later missions. The decomposition makes two authority constraints explicit: only the Manager changes campaign stage, while mission completion records either an allowed Engineer self-review or a required/requested independent Reviewer verdict. Vertical policy and stage-closing tasks prevent Engineer self-certification where independent review is mandatory; the Planner may propose future work but cannot certify the mission.

A vertical trace is useful when these tuples remain linked across many missions. It shows whether the objective survived failures, whether the Planner consumed previously accepted state, whether the Engineer changed real artifacts, whether the Reviewer rejected or redirected incomplete work, and which decisions became inputs to the next mission. None of it reduces to a single number, which is the condition this section began from; Section 6.8 follows one mathematical campaign through it, decision by decision.

4 Argus Runtime

Every mechanism in this section serves one end: making it safe to leave the Driver's seat empty. The organizing principle is not that the executor eventually becomes trustworthy enough to certify its own work. It is that the executor never gets to decide. What follows is how that decision is routed instead—to a reviewer that cannot edit what it judges, to a manager that owns the campaign rather than the turn, and to a boundary that stops for a person on the few questions a machine should not settle for itself.

4.1 System Overview

Argus is a general-purpose agentic runtime built around four model-driven roles and three system planes. The roles exist to answer the four Driver questions of Section 1 with different parties: the Reviewer answers whether work is done and good enough, the Planner and Manager answer what comes next, a certified promotion path answers how the system gets better at the field it is working in, and an explicit authority boundary answers what still belongs to a person. The *control plane* anchors the campaign and schedules work; the *execution plane* performs one bounded mission against real tools and artifacts; and the

record plane stores what happened without deciding whether the work is scientifically complete. The distinction is operational: control owns scheduling, execution owns work and mission review, and the record plane owns the immutable record but no completion decision.

Four nested scopes recur throughout this report and are worth fixing once. A *vertical* is a domain configuration—it declares the stage order, the evidence each stage requires, and what counts as completion; the runtime ships twenty-four of them, covering software, chip design, mathematics, materials, and others. A *campaign* is one long-running project under a vertical, with a persistent identity that survives restarts; the campaigns measured in Section 6.2 run for days. A campaign advances through *Stages*, which are the vertical's declared phases and whose transitions only the Manager may authorize. Work inside a Stage is divided into *bounded missions*, each assigned singly and each ending in an explicit outcome. A mission runs as a sequence of *rounds*, one Engineer turn plus its review. Vertical selects the rules, campaign is the thing that persists, Stage is where it currently is, mission is what it is doing now.

The four roles divide research authority as summarized in Table 1. The Manager spans the campaign rather than one execution round. The Planner authors future work and the Engineer performs it. Mission-level completion has an explicit source: allowed low-risk bounded work may use Engineer self-review, while vertical policy, stage-closing tasks, or the Engineer's own request require an independent Reviewer. The interfaces are structured objects rather than untyped prose, which makes ownership and recovery behavior explicit.

Role	Primary responsibility	Authoritative output
Manager	Commit the objective, lifetime, route, and campaign stage	Campaign division and stage transition
Planner	Decompose the current research state into bounded tasks and dependencies	Task specifications, plan status, and stage checklist updates
Engineer	Modify artifacts, invoke tools, and run experiments for one round	Execution result, artifacts, measurements, handoff, and structured `review=skip|required` selection
Reviewer	Independently inspect artifacts and execution records when required or requested	`done`, `continue`, `blocked`, or `replan_requested`, plus a plan signal, grounded next action, and retained-state edits

Table 1. Role separation in Argus. The Engineer executes and may self-review allowed low-risk bounded work; mandatory or requested independent review belongs to the Reviewer.

Algorithm 1: Campaign scheduling and the reviewed mission loop

Input: standing intent ι, contract K_t, runtime state H_t, backlog B, budget b
$c \leftarrow \text{ManagerCommit}(\iota, K_t)$; persist campaign identity
while b remains and c is not complete **do**
  // campaign level: the mission boundary is fixed here
  $N \leftarrow \text{StableTopological}(\text{Planner}(B, H_t, c, K_t))$
  for each bounded mission $q_0 \in N$, enqueued one at a time **do**
    $L \leftarrow \text{ExposeSkillLibraries}(q_0)$; *paths only, retrieval is the agent's*
    $\Gamma \leftarrow []$; $q \leftarrow q_0$
    repeat *// mission level: q_0 is now fixed for every round below*
      $(y, e, d) \leftarrow \text{Engineer}(q, H_t, L)$
      if $d =$ `skip` and $\text{SelfReviewAllowed}(q)$
        $r \leftarrow \text{EngineerSelfReview}(q, y, e)$
      else $r \leftarrow \text{Reviewer}(q, y, e)$ *; scoped to this round*
      append $(q, y, e, r, r.\texttt{source})$ to Γ
      if $r =$ `continue` **then** $q \leftarrow \text{AdaptAfterRejections}(\Gamma)$
    until $r \in \{\texttt{done}, \texttt{blocked}, \texttt{paused}\}$ or a governance threshold fires
    $(\texttt{status}, \texttt{reason}) \leftarrow \text{PreSettlementGuard}(\Gamma, \texttt{status}, \texttt{reason})$
    $\tau \leftarrow \text{TraceProjection}(\Gamma)$; $E \leftarrow \text{AdmitResult}(\tau)$
    $H_{t+1} \leftarrow \text{SettleMissionOutcome}(H_t, \tau, E)$ *; learning happens here*
  if evidence proposes a contract change K'_t **then**
    $(K_{t+1}, \rho_t) \leftarrow \text{ReviseContract}(K_t, K'_t, \texttt{by}, \texttt{confirmation})$
  $t \leftarrow t + 1$; refill B when required
end while
Output: accepted artifacts, inspectable trajectory, admitted contract, and updated runtime state

The algorithm above separates two levels that are easy to conflate. The Planner runs at the campaign level and fixes the

mission boundary q_0; the round loop beneath it varies q only through `AdaptAfterRejections`, which reacts to Reviewer rejections *within* that boundary. A round cannot silently rewrite q_0 on its own authority. It can, however, formally challenge it: a Reviewer holding disconfirming evidence emits `plan_signal=reconsider` with the assumption it disputes, a concrete alternative, and the authority level the change would touch. The Manager then adjudicates—keeping, revising, or replacing the plan, or escalating to the operator when the challenge crosses an operator-owned boundary—and the Planner authors any replacement. Section 7.6 describes the failure mode this channel exists to prevent, and what happened while it was unreachable.

Three further details matter because they differ from the idealized loop. *Skill libraries are exposed, not injected*: the runtime publishes library paths and the acting agent performs its own retrieval, so no matcher decides in advance which prior experience is relevant. *Termination is governed by named thresholds* rather than by Reviewer verdicts alone: a maximum round count, a no-progress threshold, a soft round limit, a hard escalation count, and a backend failure threshold each end or escalate a mission, and a pre-settlement guard may override the recorded status before anything is learned. *Learning is settled per mission, not per round*: reusable state changes once, at the mission outcome.

`ReviseContract` is where evidence becomes an authorized change of target, and its gate is deliberately two-tier. A `GoalContract` holds two kinds of clause. Semantic clauses, exclusions, and recorded ambiguities move freely, because clarifying what the user meant is ordinary Manager work. Precise clauses and the objective itself require a confirmation covering exactly the clause identifiers that change; without it the revision is refused. That split is the implemented form of the distinction Section 3 draws analytically: intent is stable, wording is negotiable, and the operational target moves only with recorded authority.

4.2 Bounded Missions over Durable State

The long-lived object in Argus is the campaign, not a provider transcript. A campaign has a persisted identity and objective; its work is divided into missions that have explicit outcomes. The scheduler assigns one mission at a time, records its result, and advances only at a clean mission boundary. Mission assignment is transactional, preventing duplicate work under concurrency, and resumed execution is tied to the persistent campaign identity.

Each role owns a small, bounded session capsule rather than sharing one transcript. The production default is backend-aware: resumable coding-agent CLIs use bounded rolling sessions, non-resumable runners fall back to a fresh session per round, and capsules rotate on branch change, turn and token limits, or an explicit cross-role signal such as repeated contradiction or reviewer confusion. Cross-session continuity comes from an ordinary shared `CHECKPOINT.md` containing durable state, evidence references, open questions, and the next step. This follows the broader move from monolithic context toward managed memory tiers and reusable workflows (Packer et al., 2023; Wang et al., 2024b; Xu et al., 2025). The Engineer updates the checkpoint after execution. When independent review is invoked, the Reviewer reads and corrects the same file and is its final editor for that round; on an accepted self-review path, the Engineer's version remains the handoff. The full history remains in artifacts and the event record.

This design makes restart and upgrade behavior part of the method. A running process hands off only at a mission boundary, and the replacement resumes from the same persistent campaign identity. After an interruption, replay or reassignment begins from committed campaign state rather than reconstructing the task from a model transcript.

Neither the roles nor the domain logic are bound to one provider. Eight coding-agent backends are supported, and per-role model selection is resolved at the role boundary, so a single campaign can run its Manager, Planner, and Engineer on different vendors' models. The FlashAttention-4 campaign in Section 6.3 did exactly this. Domain behavior is pluggable on the same principle, and the next subsection treats it as load-bearing rather than as a configuration detail.

4.3 Domain Depth Without Core Change

Being useful in a specialist field is mostly a matter of knowing what counts as evidence there. A kernel is fast or it is not, and a profiler settles it; a synthesis route is plausible only against a validator; a theorem is proved or it is not, and a proof assistant settles that. These standards differ completely across domains and have nothing to do with how a campaign should be scheduled, who may certify completion, or when a person must be asked. Argus separates them at exactly that line.

A *vertical* is the declaration of a domain's evidence standard: its stage order, its checklists, what artifacts a stage must produce, what the completion gate demands, and whether missions require an independent Reviewer at all. The runtime ships 24 of them—software, GPU kernels, chip design, digital circuits, mathematics, physics, materials, medicine, quantitative finance, and several classes of writing. They are declarations, not forks: the shared core is 130,362 lines and does not change when a domain is added. What a domain costs is what its evidence standard actually demands, and the spread is wide—the software vertical is 108 lines, the median is 775, and mathematics and quantitative finance run past 16,798 because Lean integration, citation checking, and market-data handling are genuinely that much machinery. Verticals need not live in this tree at all; they register through Python entry points against a versioned contract, and a registration that fails to declare a compatible version is rejected rather than loaded.

The seam is asymmetric, and that asymmetry is the point. A vertical can raise the evidence bar—it can require independent review for every mission, add stage-closing checks, or demand a specific artifact before a stage closes—and the core enforces

what it declares. A vertical cannot lower the bar. It cannot grant itself credentials, widen the autonomy mode, decide that an irreversible action needs no approval, or let an Engineer certify its own work where policy requires a Reviewer. This is not a convention: across all 53,871 lines of vertical code there are zero references to the autonomy mode, the operator-escalation path, or the approval boundary, because that logic lives in the core and is not reachable from a domain package.

Self-evolution runs along the same seam, which is why what a specialist writes is a seed rather than a ceiling. The declaration establishes the field's initial standard; the runtime then works inside it and revises it. What the runtime learns is scoped to where it was shown to hold: a procedure that works in one project stays project-local, one that proves general to a domain is promoted into that vertical's contract as a revised checklist under a Manager pass, and only what survives beyond a domain becomes global. A vertical is therefore a living object—seeded by a person who knows the field, then sharpened by every campaign that runs under it—while the core accumulates none of that knowledge.

Revision is bounded in the direction that matters. A vertical may declare `PROTECTED_ITEM_IDS`, a floor of gates that the promotion path restores against later edits, so the checks a domain considers irreducible cannot be optimized away by the same process that adds new ones. Growth is permitted upward from the seed; the floor does not move. Domain expertise deepens where the evidence standard lives; the rules about who may decide stay where they cannot be edited by the party being judged.

4.4 Review and Completion

All role invocations pass through a common instrumentation layer that records usage and associates each call with the mission trace. The Engineer and Reviewer share the same artifact state, allowing review of the actual outputs and execution record rather than only the Engineer's summary. A typed, append-only trace is the canonical timeline; user interfaces are projections over that record.

The source of a mission completion verdict is explicit. When self-review is enabled and no vertical or task policy requires independence, an Engineer may select `review=skip`; its prompt restricts that choice to low-risk bounded work with a passing verifier. The runtime accepts the explicit agent judgment without adding a second heuristic or validator and records `review_source=engineer_self_review`. Otherwise a fresh Reviewer returns the structured verdict, and stage-closing or vertical-required review cannot be waived. Completion never comes from filenames, Token volume, or prose keywords. The corresponding artifacts and execution records remain inspectable, while credential redaction and result provenance preserve the selected verdict source.

4.5 Iterating Without a Score

Section 1 observed that the Driver's four questions become trivial when an explicit numeric reward exists. This subsection describes what answers them when one does not, because that case is the reason the seat could not be automated earlier.

The naive substitute—have a model rate progress on a numeric scale—reintroduces the problem it was meant to solve, since the rating is produced by the same class of system whose work is being rated, and Section 2 records how unreliable that is. Argus takes the opposite route. It does not manufacture a score. It replaces the scalar with a small typed vocabulary of state transitions, and makes certain readings of the evidence mechanically unavailable.

Progress is a transition, not a number. A mission carries a durable frontier recording what it now knows and what it still owes. Each round closes with one named transition drawn from a fixed set: `artifact_improved`, `risk_reduced`, `uncertainty_reduced`, `information_gain`, `bounded_regression`, `recovered`, `unchanged_failure`, `expanding_regression`, or `unexplained_regression`. Six of the nine count as forward motion, and the selection is deliberate: *a failure that produces information counts as progress, and a repair that trades a known bounded cost for a resolved risk counts as progress*, while a repetition that changes nothing does not. This is what makes it possible to run a campaign that improves for days without any number improving. A route that is eliminated with evidence has moved the frontier as surely as a faster kernel has, and the runtime records both without needing them to be commensurable. The three regression states are not merely negative labels: a `bounded_regression` must arrive with a cause, a scope, a budget, a recovery test, and an exit trigger, and one that does not is automatically downgraded to a request for the plan itself to be reconsidered.

An experiment that did not run is not a refuted idea. The most expensive failure mode in unattended research is not a wrong answer; it is discarding a correct hypothesis because the evidence against it was an artifact of the environment. A missing compiler, an unavailable GPU, a denied profiler permission, or an underpowered pilot all produce a failed run, and a failed run reads as a negative result unless something forbids that reading. Argus forbids it structurally, by recording three separate fields rather than one outcome: whether the attempt executed at all, which class of failure occurred if it did not, and what is now believed about the premise—`untested`, `inconclusive`, `supported`, or `refuted`. Each domain declares which failure classes carry information about the idea and which cannot; environment, dependency, and toolchain failures are in the second group everywhere. The validator then enforces the invariant that matters: **a non-idea failure can never yield** `supported` **or** `refuted`. The hypothesis returns to the queue as `untested`, which is the truthful record and also the useful one.

Not finishing is a result with a name. A system that can only report success or failure will, over a long campaign, be pushed toward reporting success. The runtime therefore gives the honest intermediate outcomes their own vocabulary, and a Reviewer selects among them explicitly: a counterexample, a partial result, an improved bound, a finite verification, a structured failure report, a candidate whose novelty is unverified, or the declaration that current methods are exhausted. Correctness, novelty, and significance are recorded as separate fields rather than collapsed into a verdict, so a result can be correct, already known, and still worth retaining. Nothing in this vocabulary requires the campaign to have reached its original objective, which is what allows the objective itself to be revised on evidence rather than defended to the end.

The evidence bar moves with the work. Requiring certification-grade evidence of an early probe stops exploration; accepting exploratory evidence for a final claim produces unsupported results. The runtime resolves an evidence profile per mission—`explore` asks whether the premise is real, testable, and worth the next probe; `develop` asks whether the implementation, comparison, and claim scope hold; `certify` asks whether the result can be published as it stands. The profile derives from the campaign stage by default, a final-submission scope always forces `certify`, and lowering the bar requires an explicit confirmation. What varies is the standard of evidence demanded, never whether evidence is demanded.

Together these four mechanisms answer the question an explicit numeric reward would otherwise answer for free. The runtime knows whether the last round advanced the campaign, whether a negative result was about the idea or about the machine, what kind of finding it is holding, and how strong the evidence must be before that finding may be believed. None of this requires a scalar, and none of it requires the runtime to grade itself.

4.6 Verification-Gated Fixed-Model Runtime Self-Evolution

Equation 6 separates the model from the state around it. Table 2 identifies how each component can change and who owns the committed update. The ownership model is intentionally not uniform. Both knowledge surfaces use a work-versus-certification split: the Engineer authors a candidate after its own task completes, and a separate Manager pass decides whether it stays project-local, is promoted to a vertical, or becomes global. Memory takes no such pass, because it is not a claim about the world. Tools and procedures are system-configured. Stage checklists belong to the vertical contract and are certified by the Manager stage decision, with Reviewer feedback rather than a second commit gate. Routing policy is Manager-committed, while task definitions are Planner-authored and scheduler-committed.

Four terms recur and are used strictly.

- *Evidence-driven* describes the control policy that decides whether to persist, stop, or pivot. It is chosen against *goal-driven*: that controller treats the stated objective as authority and every departure as error to suppress, whereas this one treats the objective as the current best hypothesis and lets evidence overturn it. Where the objective was underdefined at the start, the second is the only defensible posture, because what the expert wrote before the work began is itself a claim that can be wrong.
- *Verification-gated* is the narrower admission condition on reusable updates, and the umbrella term of this report: a candidate is not reusable because a role produced it, but because the task-native evidence for that surface exists and its authorized owner committed it. That gate may be an executable verifier, an independent Reviewer, permitted Engineer self-review, or a system-configuration commit—it does not mean every low-risk update needs a Reviewer.
- *Reviewer-gated* is reserved for the independent-Reviewer path alone, and *external grader* for a task-native evaluator outside the role loop.
- *Runtime self-evolution* is used in a deliberately narrow, fixed-model sense: parameters do not change, and what evolves is the persistent state that later missions retrieve or obey.

A complete update cycle has four parts: (1) an execution trajectory produces a candidate wiki page, skill, procedure, verification rule, routing decision, or task definition; (2) the responsible role checks the candidate against artifacts and task-native evidence; (3) the authorized owner commits, revises, or rejects the update; and (4) a later mission retrieves the retained state as part of its starting context or execution policy. Activity that does not survive this commit-and-reuse path is not counted as self-evolution.

State	Change source	Commit owner	Persistent form
Knowledge—certified, reusable, scoped			
wiki	Engineer authoring from reviewed outcomes	Reviewer	Source-linked semantic pages
skills	Engineer post-task authoring	Manager placement review	Versioned skill library
Memory—durable, uncertified, campaign-scoped			
journal and backlog	Runtime execution	Runtime (append-only)	`events.jsonl`, mission backlog, durable artifacts
session context	Role invocation	Runtime	Per-role rolling window
tools and procedures	System configuration	System configuration	Versioned tool and procedure registry
verification	Vertical stage contract	Manager stage decision	Stage checklist and certificate; Reviewer supplies feedback
routing and roles	Runtime policy	Manager	Campaign routing policy
tasks and evaluations	Planner	Scheduler	Backlog task specifications and scopes

Table 2. Attributable ownership of fixed-model runtime evolution. A mission usually updates only a subset of these components.

Knowledge is the pair defined in Section 3: a wiki recording what a domain turned out to be like, and a skill library holding procedures that can be matched to later tasks. Neither substitutes for the other—a procedure with no account of why it works cannot be revised when conditions change, and a finding no mission can act on is inert—and neither is treated as automatically correct, since entries can be revised, archived, or retired when later results contradict them.

This mechanism can improve a later mission without changing the model itself. A verified retained failure can prevent a repeated dead end; an admitted procedure can shorten environment inspection; a verifier can reject a previously accepted shortcut; and a routing update can assign independent review to a higher-risk task. The mechanism does not imply monotonic improvement. Some missions commit no reusable state, retained state can become stale, and a harder task distribution can increase cost even after useful state has accumulated.

4.7 The Authority Boundary

The third Driver question—does this need a person—is answered by an explicit boundary rather than by model judgement, because a system that decides for itself when to ask for permission has not really been constrained. Three autonomy modes are available. The cautious mode escalates every explicit question; the default pragmatic mode and the autonomous mode both resolve ordinary technical obstacles without a human. In all three, a fixed set of categories always stops: credentials and access tokens, payment and budget increases, force pushes and deletion of production or operator-owned data, publication and external transmission, and any question about whether a trust, security, or legal boundary may be crossed. A Reviewer may additionally mark a decision as operator-owned, which escalates regardless of mode.

The boundary is deliberately narrow in the other direction. An ordinary timeout, a failing test, an unavailable backend, or a rejected approach is recoverable technical work and does not interrupt anyone. This asymmetry is what makes the interruption rate in Section 6.2 interpretable: escalations are rare because recoverable failure is absorbed, and the escalations that remain are concentrated on decisions the runtime should not make for its operator. The Driver may choose the route; it does not hold the keys.

4.8 Reliability and Resource Governance

Long-running systems fail through operational drift even when individual model calls are strong. Argus therefore combines round-level progress classification with mission-boundary replanning. Work that repeatedly produces no decision or artifact change is stopped or reformulated, while long-running external jobs are tracked separately from model reasoning. These mechanisms bound execution; scientific correctness remains with the task evaluator and the explicitly selected Engineer-self-review or independent-Reviewer judgment path.

Resource accounting is centralized at model-call and external-job boundaries. Budgets are checked before work begins and reconciled after completion, preventing individual roles from expanding their own allocation. Scheduling, sandboxing, and deployment are engineering choices, and we expect to keep changing them. What is not a choice is the shape they enforce: no role writes its own permission, and no role grades its own work.

5 Empirical Methodology

We evaluate Argus across seven benchmark arenas spanning software repair, GPU-kernel optimization, language-model training, training-speed optimization, research-assistant tasks, and data synthesis. Each benchmark is reported in its native unit. SWE-Bench Pro is one benchmark in this suite; its sequential task log also enables deeper analyses of runtime evolution and Reviewer intervention.

5.1 Research Questions

Evaluation is organized around five research questions, labelled RQ to keep them distinct from the Driver questions Q1–Q4 of Section 1, which are about what the runtime decides rather than what we measure. RQ0 concerns the operating mode itself and is prerequisite to the rest: a result obtained under continuous supervision and the same result obtained unattended are different findings.

RQ0: Unattended operation. How often does the runtime require a human, what does it ask for when it does, and what fraction of available time is it producing? These are reported as duty cycle and interruption rate over campaign ledgers, together with what the unattended hours delivered.

RQ1: Benchmark performance. What task-native outcomes does Argus achieve across the seven benchmark arenas, and how do they compare with the reference reported for each arena?

RQ2: Fixed-model runtime self-evolution. Within the SWE-Bench Pro run, how does solve-time resource demand change as reviewed Skill, Wiki, verification, and routing state accumulates while the underlying model parameters remain fixed?

RQ3: Reviewer routing and recovery. Within the same SWE-Bench Pro run, how often is an independent Reviewer invoked, and how many initially rejected solutions are recovered after Reviewer feedback?

RQ4: Process-to-capability mechanisms. How do the retained traces quantify process-data advantage, verification-gated correction, Token conversion, and cross-task reuse?

5.2 Measuring Unattended Operation

Duty cycle and interruption rate are computed from two runtime ledgers rather than from instrumentation added for this report. The settled usage ledger records one row per model call with start and completion timestamps, so the union of call intervals gives occupied time without relying on a silence heuristic; a single call can exceed two hours, which is why event-gap thresholds are unsuitable. The event log records system-initiated human requests as typed events, which separates them from operator messages that the system never waited on. Campaigns with fewer than 50 recorded calls are excluded as too short to characterise. The two subtracted terms in Equation 10—empty backlog and unusable environment—are identified from gaps exceeding one hour, with the distinction drawn by whether the preceding event was a completed mission or an interrupted execution. Because these are production campaigns rather than a controlled protocol, task difficulty, vertical, and operator availability all vary across rows; the tables characterise an operating profile and are not a matched comparison.

5.3 Benchmark Suite

SWE-Bench Pro evaluates repository-level software-engineering tasks with executable acceptance tests (Deng et al., 2025). The remaining arenas evaluate B200 kernel optimization, nanochat training on B200 and H100, the nanoGPT speedrun, AARRI-Bench, and Math-Reasoning Data Synthesis from the Arbor suite (Lin et al., 2026; Karpathy, 2025, 2026; Jordan and contributors, 2024; Wang et al., 2026; Jin et al., 2026). Because rank, bits per byte, elapsed time, solve rate, and pass-gap are incompatible metrics, we do not average them into one score.

Benchmark	What the agent must do	Evaluation metric	Better
SWE-Bench Pro	Repair a real repository issue by editing production code; the submitted patch must satisfy task-specific executable acceptance tests.	Fraction of tasks resolved by the official verifier	Higher
SOL-ExecBench	Optimize real GPU kernels without changing their outputs, targeting execution close to an analytically derived hardware speed-of-light bound.	SOL score, global rank, and high placements after correctness checks	Higher
nanochat B200	Improve the nanochat training program under a fixed five-minute run on one B200 GPU.	Validation bits per byte (BPB), a compression-oriented language-model metric	Lower
nanochat H100	Solve the same five-minute nanochat training objective on one H100; it is a separate hardware-specific optimization problem.	Validation BPB under the H100 protocol	Lower
nanoGPT speedrun	Modify the training system so an eight-H100 run reaches validation loss 3.28 as quickly as possible.	Mean time to the target over ten verifier runs	Lower
AARRI-Bench	Complete 82 granular tasks representing the judgment, diligence, and professional practice expected from a research intern.	Number and percentage of tasks solved	Higher
Math-Reasoning Data	Build a pipeline that generates AIME-style reasoning problems that are difficult on the first attempt but solvable with additional attempts.	Mean pass@4 − pass@1 gap	Higher

Table 3. Definitions of the seven benchmark arenas. B200 and H100 nanochat runs are kept separate because hardware changes the optimization problem.

The six non-SWE arenas use GPT-5.5 through Codex. The SWE-Bench Pro evaluation uses GPT-5.5/xhigh through Copilot for both Direct Copilot and Argus. The full 731-task comparison reports approximate accuracy and the aggregate Token ratio

$$R_{\text{tok}} = \frac{N^{\text{total}}_{\text{Argus}}}{N^{\text{total}}_{\text{Copilot}}} \approx 1.41. \tag{23}$$

Raw Copilot Token totals and per-Wave resource traces were not retained; therefore the longitudinal analysis is Argus-only.

5.4 SWE-Bench Pro Runtime Self-Evolution Analysis

For a completed Argus Wave w with task set $\mathcal{T}_w$, we measure

$$\bar{I}^{\text{solve}}_w = \frac{1}{|\mathcal{T}_w|} \sum_{i \in \mathcal{T}_w} I^{\text{solve}}_i, \tag{24}$$

$$\bar{T}^{\text{agent}}_w = \frac{1}{|\mathcal{T}_w|} \sum_{i \in \mathcal{T}_w} T^{\text{agent}}_i. \tag{25}$$

I^{solve}_i contains all model calls in the accepted task trajectory, including routing, Skill adaptation, execution, and any independent review. T^{agent}_i measures active workflow time. Orchestration wait, environment preparation, external verification, infrastructure recovery, and post-task knowledge maintenance are excluded.

Completed Waves are summarized by task-weighted windows: W1–6 (startup), W7–12 (early reuse), W13–18 (composition shift), W19–22 (mature operation), and W23–24 (late difficult tasks). Two incomplete Waves are omitted from the grouped means. The primary comparison is startup versus mature operation; composition and difficulty variation remain visible.

The unit of analysis is the observed sequential window, not a matched task pair. The startup window begins with less project-specific state, whereas later windows can retrieve reviewed repository knowledge, reusable procedures, prior failure records, verification guidance, and updated routing decisions. Lower Token or time demand in a later window is therefore consistent with runtime self-evolution when the retained state removes repeated inspection or failed work. It is not, by itself, a causal estimate of the value of that state. Task identity, repository mix, execution latency, and difficulty also change across Waves, and no matched frozen-state replay is available.

5.5 SWE-Bench Pro Reviewer Analysis

A task is *Reviewer-invoked* when its final trajectory contains an independent Reviewer model call; otherwise it follows Engineer self-review. A Reviewer verdict of `continue` is a revision request. We report both subsequent official verifier success and the stricter case in which a later Reviewer verdict is `done`. The latter is called a *strict Reviewer rescue*.

Reviewer routing is adaptive rather than randomized. The analysis reports routing, revision, and recovery behavior over the observed task sequence.

The 41-artifact research portfolio is reported separately as a measure of research program breadth.

5.6 Representative Vertical-Trace Protocol

To complement endpoint and software-trajectory measurements, we report one Erdős–Gyárfás mathematical campaign as a representative vertical trace. The trace is role-led rather than theorem-led: it records what the Manager, Planner, Engineer, and Reviewer did during recovery, problem selection, route testing, proof production, revision, acceptance, and state retention. The reported record contains one accepted route falsification and six proof-backed research deltas; detailed mathematical statements remain in the supplementary claim table.

The protocol tracks whether the runtime retains a falsified branch, carries named blockers across missions, and admits later work after artifact-based review. The role trace, claim table, and figure provenance are released with the supplementary materials.

5.7 Theory-to-Measurement Mapping

We map the theory to four reported quantities. Immediate information gain ΔI_t is represented only in task-native units: benchmark improvement, accepted theorem delta, or certified branch elimination. Review correction is represented by Reviewer `continue`–revise trajectories and later external-verifier outcomes. Reuse value G_L is approximated observationally by later Token/time demand as Skill and Wiki state accumulates. Token conversion is represented by the role- and mission-level attribution in the mathematical case. Appendix D records the corresponding substitutions in their native units.

6 Results

The results come in that order for a reason. Seven arenas establish that the runtime does the work; 1,548 wall-clock hours establish that it did so with the Driver's seat empty. Neither half means much alone. A capability number obtained under continuous supervision is a different result from the same number obtained unattended, and an empty seat is worth nothing if nothing came out of it. We report the floor, then the human cost of reaching it, then what the unattended hours delivered.

6.1 Capability Floor Across Seven Arenas

Table 4 is the primary empirical summary and reports all seven benchmarks in one comparison; Table 3 defines the task and metric for each row.

Benchmark	Backbone	Backend	Protocol	Argus result	Comparison
SWE-Bench Pro	GPT-5.5/xhigh	Copilot	731 software-repair tasks	≈78% accuracy	Direct Copilot ≈59%; 1.41× aggregate Tokens
SOL-ExecBench	GPT-5.5	Codex	B200; 101 kernels	Rank #6 globally; 7 kernels in the top 3	Beat the #1-ranked entrant on 2 kernels head to head
nanochat B200	GPT-5.5	Codex	5 min; 1×B200	0.9636 BPB	Human best 0.9646
nanochat H100	GPT-5.5	Codex	5 min; 1×H100	0.9855 BPB	Human best 0.9879
nanoGPT speedrun	GPT-5.5	Codex	8×H100; N=10	79.77 s	Same-device human 80.18 s
AARRI-Bench	GPT-5.5	Codex	82 research-intern tasks	63/82 (76.8%)	Paper best 68.3%
Math-Reasoning Data	GPT-5.5	Codex	AIME-style synthesis	28.0 gap	Arbor 20.83; Claude 8.33; Codex 6.25

Table 4. Results across seven benchmark arenas. Values remain in task-native units and are not cross-normalized. Backbone denotes the model driving the research agent; Backend denotes the execution surface.

Three of these arenas—nanochat, the nanoGPT speedrun, and SOL-ExecBench—are also run by systems purpose-built for them. Recursive reports stronger numbers than ours on the first two (Recursive, 2026). We report them anyway, and read them as a floor rather than a claim to the frontier: they establish that a general runtime carrying an independent Reviewer, an authority boundary, and a durable campaign contract remains competitive on tasks a specialist system optimizes directly. A verification-gated architecture that lost badly here would be paying for its gates in end-task performance, and Section 8 would have to say so.

The runtime is not bound to one backbone or one execution surface. A separate SWE-Bench Pro run replaces GPT-5.5 on Copilot with GLM-5.2 at a 200K-token context window, driven through Claude Code, and is currently at 70.94% accuracy. That run is still in progress and has no matched Direct baseline on this backend, so we do not enter it in Table 4 or in any longitudinal analysis; we note it because the runtime carried the same contract, routing, and Reviewer policy onto a different model and a different agent surface without re-instrumentation.

As a small upstream-adoption example, an Argus-optimized TileLang `RWKV6` kernel was reviewed by a Moonshot AI-affiliated FLA collaborator and merged into `fla-org:main` (pull request (PR) #1045; commit `c70f11c`); Appendix E records the technical details.

An eighth arena is in progress and reported as partial. The MLE-Bench Lite campaign runs the official Low split (Chan et al., 2024) under a reviewer-approved medal gate: a competition closes only when an independently reviewed submission earns a

Kaggle medal, so outcomes are medals rather than leaderboard positions. Table 5 lists those closed to date—9 medals from 13 graded competitions, a 69.2% rate, evenly split across gold, silver, and bronze. We report the denominator because a medal rate without its graded total is not interpretable; the remaining four closed without a medal, two of them documented in detail.

The gate is narrow rather than nominal. On `denoising-dirty-documents` the campaign settled 0.00009 RMSE short of the silver band, and on `jigsaw-toxic-comment-classification-challenge` it cleared bronze by 0.00018 AUC—both awarded by the external grader against the official leaderboard, not by internal review. The transparent-conductor result is also a route change under that gate: the grader rejected the initial approach at 0.208 RMSLE, the campaign replaced it with a published state-of-the-art method, and the grader certified the replacement at 0.06402 against a 0.06582 bronze threshold. Remaining competitions continue; we exclude this arena from Table 4 and will report the full split on completion.

Competition	Metric	Argus	Medal
`aerial-cactus-identification`	AUC ↑	**1.00000**	**Gold**
`dogs-vs-cats-redux-kernels-edition`	LogLoss ↓	**0.02168**	**Gold**
`detecting-insults-in-social-commentary`	AUC ↑	**0.91386**	**Gold**
`histopathologic-cancer-detection`	AUC ↑	0.98312	Silver
`leaf-classification`	LogLoss ↓	0.00196	Silver
`mlsp-2013-birds`	AUC ↑	0.91479	Silver
`denoising-dirty-documents`	RMSE ↓	0.02618	Bronze
`jigsaw-toxic-comment-classification-challenge`	AUC ↑	0.98657	Bronze
`nomad2018-predict-transparent-conductors`	RMSLE ↓	0.06402	Bronze

Table 5. MLE-Bench Lite competitions closed with a medal by Argus so far. Arrows give the direction of improvement: area under the ROC curve (AUC) is maximized, while log loss (LogLoss), root mean squared error (RMSE), and column-wise root mean squared logarithmic error (RMSLE) are minimized. Each score is the Reviewer-approved submission that the external MLE-Bench grader scored, and each medal is the grader's award against that competition's official Kaggle leaderboard. Seven of the nine rows are reconciled against the 2026-07-29 reviewer-approved campaign snapshot, which records 40 graded submissions across nine competitions. The campaign is still running, so this table reports the competitions closed to date rather than a final standing.

The table shows one runtime reaching competitive outcomes across heterogeneous tasks. The remainder of this section analyzes the SWE-Bench Pro row because it provides a long sequential trajectory and Reviewer decisions.

6.2 Does the Driver's Seat Stay Empty?

Every preceding result measures what the runtime produced. This one measures whether producing it required a person. Across 27 campaigns and 1,548 wall-clock hours, Argus asked for a human 38 times—once every 40.7 hours—and spent 95.1–98.7% of its available time working. The seat stayed empty.

Two metrics carry that claim, and neither survives alone. *Interruption rate* alone is minimised by a system that does nothing; *duty cycle* alone is maximised by a system that burns Tokens in a loop. Together they pin the case that matters: work is being done, and nobody is being asked to supervise it.

Both quantities are defined in Section 3.7: duty cycle by complement in Equation 10, so that what counts as productive never has to be adjudicated, and interruption rate in Equation 11 over system-initiated requests only. We report the partition of Equation 12 alongside the rate, because a request for a credential and a request arising from indecision are different facts about the system.

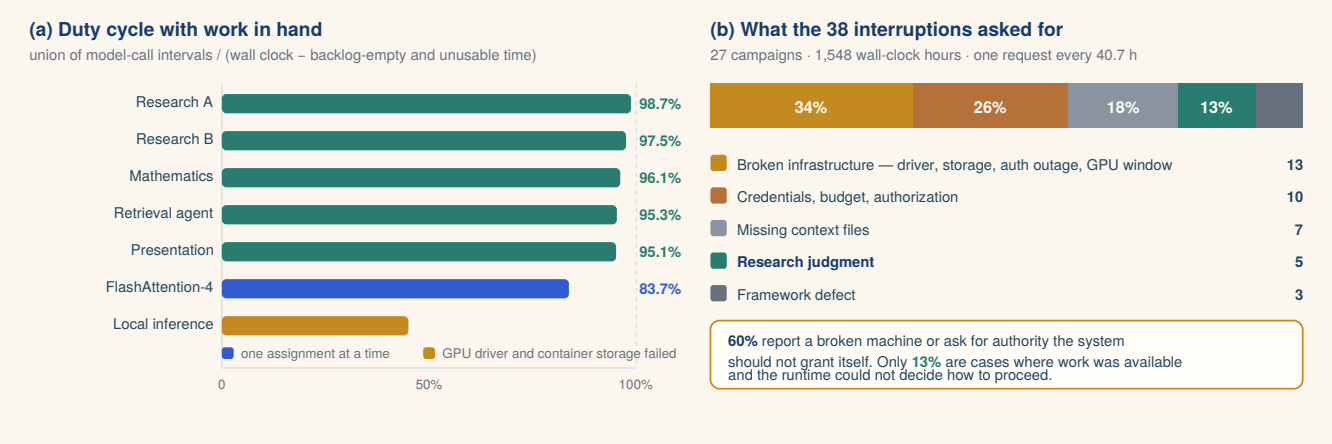

Figure 4. Unattended operating profile. Panel (a) gives duty cycle per campaign under Equation 10; the two lower bars are annotated with why they sit lower, and neither reason is agent hesitation. Panel (b) classifies every human request raised across the 27 campaigns by what it asked for. The two metrics are reported together because neither is interpretable alone: a system that does nothing has a perfect interruption rate, and a system that loops uselessly has a perfect duty cycle.

Campaign	Span (h)	Busy (h)	Duty cycle
Research campaign A	168.9	107.4	**98.7%**
Research campaign B	109.4	25.4	**97.5%**
Mathematics campaign	168.8	63.2	**96.1%**
Retrieval-agent campaign	133.1	46.5	**95.3%**
Presentation campaign	70.8	27.1	**95.1%**
FlashAttention-4 adaptation	87.7	21.2	83.7%
Local-inference campaign	169.2	44.7	45.0%

Table 6. Duty cycle with work in hand, computed from Equation 10. Including all idle time—periods with an empty backlog awaiting a new assignment—the aggregate raw busy fraction is 32.4%; the two figures answer different questions and should not be conflated.

Five campaigns sit between 95.1% and 98.7%. The two lower rows are the informative ones. The FlashAttention-4 adaptation ran in a one-assignment -at-a-time mode: each of its long silences begins with a mission completing successfully and ends with the operator supplying the next objective, so the gap measures the human's response latency, not the runtime's hesitation. The lowest row, at 45.0%, is the campaign whose interruption log records a failed NVIDIA driver, corrupted container image storage, and contention for an exclusive GPU window. **The ceiling on unattended operation in these traces is infrastructure availability, not agent autonomy.**

What the system asked for	Count	Share
Broken infrastructure (driver, storage, auth outage, GPU window)	13	34%
Credentials, budget, or authorization	10	26%
Missing context files	7	18%
Research judgment	**5**	**13%**
Framework defect	3	8%

Table 7. All 38 human requests raised across 27 campaigns and 1,548 wall-clock hours, classified by what was asked.

Across 27 campaigns, 1,548 wall-clock hours, and 306,691 logged events, the runtime raised 38 requests for a human: one every 40.7 hours. The composition matters more than the rate. A majority—34% infrastructure plus 26% authority—are cases where no autonomy improvement would help, because the machine was broken or the decision was not the machine's to make. The authority requests show the boundary working as designed rather than as a generic escape hatch; a representative one proposes its own ceiling before asking, requesting at most 256 additional calls under an existing subscription with an explicit incremental billing cap of zero and no new credentials. Only 13%, that is 5 requests in 1,548 hours, are the case the Driver framing is really about: work was available and the system could not decide how to proceed. One of those is a mathematical campaign reporting that it had exhausted the analytic and structural bounding methods within its current scope—an escalation we would want a research system to make.

These campaigns are also long individually, which matters for a reason separate from autonomy. 4 of the 27 exceed a week of wall-clock time and the longest runs 8.1 days, with a single trace recording 61,797 events. A trajectory of that span contains

dependencies no short episode can: a premise adopted early, contradicted by a measurement days later, and revised with both states still on the record. Section 8 returns to why that is the interesting property of this data.

The remaining 18% are the runtime's own fault in a specific and fixable way: canonical handoff files went missing and the system correctly refused to proceed from an inferred state rather than guess. Together with the infrastructure share, this locates most of the remaining gap in operational engineering—process supervision, storage reliability, durable context—rather than in the agent loop.

For scale, a strong coding agent driven turn by turn sustains roughly 1 hour of unattended operation before requiring its operator, an interval two orders of magnitude below the 40.7-hour figure above. The consequence for duty cycle follows from arithmetic rather than measurement: a harness whose Driver is a person cannot exceed that person's working hours, so even an operator devoting eight hours a day to a single campaign bounds it at 33%—and that ceiling is unaffected by how capable the underlying model becomes. The part that matters needs no experiment at all: it is arithmetic, and no scaffolding difference between the two systems touches it. One system's clock follows its operator's waking hours; the other's does not.

6.3 What the Unattended Hours Delivered

Duty cycle establishes that the seat stayed empty; it does not establish that anything worth having came out of it. Section 6.11 presents the delivery evidence in full, drawn from GPU and serving infrastructure because that domain pairs an unforgiving verifier with reviewers outside this project. Three comparisons bound the question here.

Against a human working with a coding agent, on a SenseNova U1 integration into SGLang, one engineer invested over 60 hours without completing the task, while Argus finished it within a 24.14-hour wall-clock envelope under an operator who was not an infrastructure specialist. Against a specialist team, on a FlashAttention-4 adaptation carrying six diffusion-LLM mask families across two GPU architectures, the runtime reached 5–14% of native kernel performance for under 80 CNY of model spend, in a direction that team had worked for over three months. Against an external reviewer, 4 kernels submitted to `flash-linear-attention` were accepted with no blocking changes requested.

The first two are bounds, not controlled trials, and Section 8 sets out their asymmetries. The third answers to nobody here: it was decided by people who did not write the work and had no stake in its acceptance.

6.4 Knowing When to Stop: Verification Routing and Non-Termination

Objective revision is admissible only if the runtime can withhold completion. This subsection measures where independent verification is spent and where the runtime declines to declare a task finished.

Of 731 tasks, 466 (63.7%) invoke an independent Reviewer, while 265 (36.3%) use Engineer self-review. The Reviewer requests another implementation round on 43 tasks. After revision, 34 pass the official verifier and 22 complete the strict `continue`→revision→`done` loop. A further 35 tasks receive a `blocked` verdict, recording that the runtime cannot complete them rather than emitting an unsupported completion. Refusing to stop early and refusing to claim success are two expressions of the same gate.

Reviewer-routed tasks consume 2.75× as many solve input tokens and 1.80× as much active time as self-reviewed tasks on average, indicating that the routing policy selects a harder workload. The recovery funnel is therefore more interpretable than a raw group-accuracy comparison.

The two populations should not be conflated. Of the Reviewer-routed workload, 388 tasks are accepted on first review and 43 receive a revision request, alongside 35 declared blocked; verifier pass (34) and strict rescue (22) are nested recovery milestones *within* the revision-requested group rather than separate outcome classes. Panel (a) of Figure 5 shows this partition together with the accuracy comparison.

6.5 Cost of the Mechanism as State Accumulates

Figure 5 shows the Argus-only longitudinal trajectory. Here, self-evolution refers to changes in persistent runtime state, not to online training of the underlying model. Across Waves, accepted trajectories can update repository knowledge, reusable Skills, verification guidance, task definitions, and routing policy. These objects alter what later missions retrieve, which checks they run, and where they spend independent review.

Mature operation W19–22 uses 21% fewer solve input Tokens and 15% less active workflow time per task than startup W1–6. The runtime spends less to do the same work once it has accumulated state to spend it against. The lowest-Token window, W13–18, reaches a 50% reduction from startup but requires more active time, showing that Token efficiency and execution latency do not move together. The final difficult-task window rebounds in both metrics.

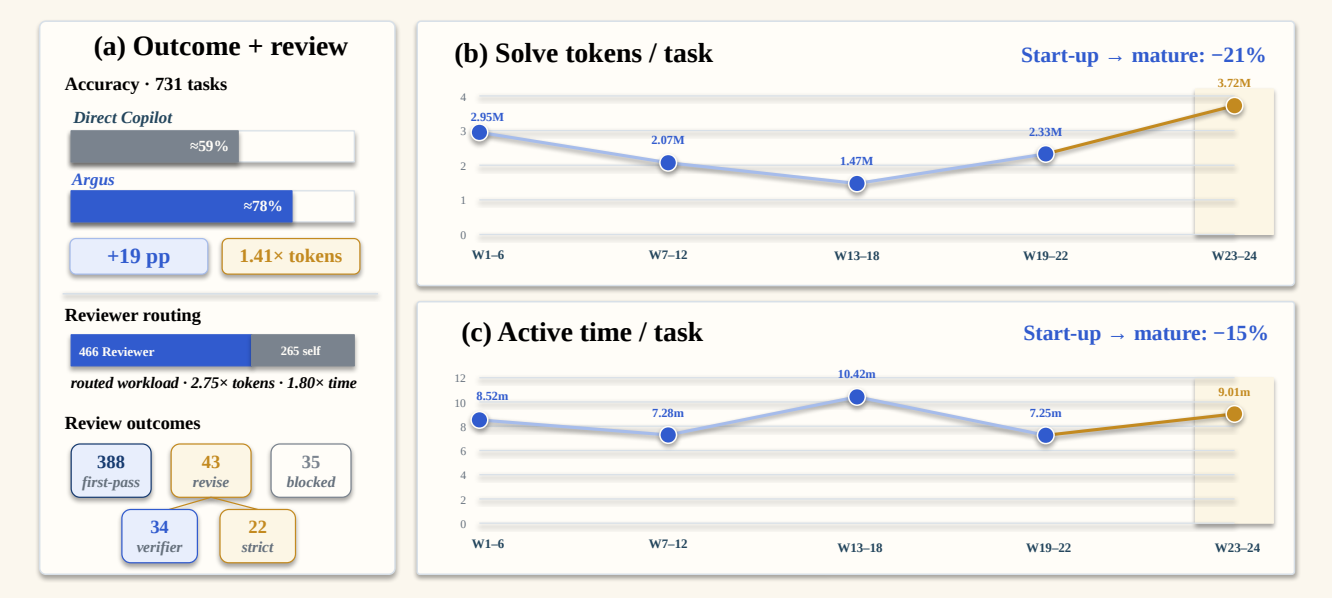

Figure 5. Outcome, review, and longitudinal efficiency in the 731-task SWE-Bench Pro run. Panel (a) reports the full-suite accuracy and aggregate-Token comparison, adaptive Reviewer routing, and observed review outcomes. Panels (b) and (c) aggregate Argus solve input Tokens and active workflow time over approximately six-Wave windows using task-weighted means. Relative to W1–6, W19–22 uses 21% fewer solve input Tokens and 15% less active time per task. Two incomplete Waves are omitted, while W23–24 remains visible as late difficult-task stress. Copilot per-Wave resource traces were not retained; the window comparison characterizes the operating profile over this sequence, and a controlled replay would isolate the learning effect.

The trajectories show a lower-cost mature operating window together with visible task-composition and late-wave difficulty effects. The rebound in W23–24 prevents the startup-to-mature comparison from being read as monotonic improvement, and the sequence characterizes state accumulation rather than isolating any single update.

6.6 Measured Process Quantities

The process theory is instantiated from two complementary traces. In the mathematical campaign, direct reasoning, verification, and action each account for approximately 56% of their respective attributed units. In SWE-Bench Pro, Reviewer revision requests lead to 79.1% official-verifier recovery and 51.2% strict review-loop rescue, while the mature window uses 21% fewer solve input Tokens and 15% less active time than startup. Appendix D gives the complete substitutions for the density, reuse, yield, and compression quantities.

The public research inventory additionally contains 41 de-duplicated artifacts across six programs, illustrating the range of research programs executed by the same runtime.

6.7 The Delivery Ledger

Table 8 indexes the campaigns reported in this section and the subsections that follow it, grouped by the kind of verifier each answers to. The grouping is the point: the further down the table a row sits, the less the runtime controls the standard it is judged against, and the harder it is to satisfy the gate with narrative.

Two rows carry disproportionate weight for the argument of this report, and neither is the highest number in the table. The upstream kernel row is the only one whose acceptance criteria were written by people with no connection to this project. The MOF row is the only one where the admitted method is *smaller* than the published method it replaces, which is not an outcome a system optimizing a visible score tends to reach.

6.8 Objective Revision in a Mathematical Campaign

Formal mathematics is the domain where machine verification is strongest: large-scale synthetic proof data and proof-assistant feedback have pushed LLM provers to competitive results on olympiad benchmarks (Xin et al., 2024, 2025), and reinforcement learning in Lean reached medal standard at the International Mathematical Olympiad (Hubert et al., 2025). The campaign below is a different setting: the target is an open conjecture rather than a problem known to have a proof, so the question is not whether a proof can be found but what a campaign should retain when it cannot.

Figure 6 follows the retained mathematical claim frontier inside one real campaign. The case begins with an open-ended research objective. The Manager preserves that goal through budget and process failures; the Planner turns the current reviewed state into bounded work; the Engineer retrieves sources, runs tools, and writes research artifacts; and the Reviewer decides whether the work is accepted, revised, or blocked before it can enter durable state.

Campaign	Result	Who decides
Benchmark harnesses (Section 6)		
SWE-Bench Pro	≈78% vs 59% direct; 35 declared blocked	Official verifier
SOL-ExecBench	Rank #6 globally; 7 kernels placed top-3; beat the #1 entrant on 2	Official scorer
nanochat B200 / H100	0.9636 / 0.9855 BPB vs 0.9646 / 0.9879 human best	Fixed protocol
nanoGPT speedrun	79.77 s vs 80.18 s same-device human	Verifier runs
AARRI-Bench	63/82 (76.8%) vs 68.3% paper best	Task rubric
Math-Reasoning Data	28.0 gap vs 20.83 / 8.33 / 6.25	Frozen solver
MLE-Bench Lite	69.2% medal rate (9/13 graded): 3 gold, 3 silver, 3 bronze	Kaggle leaderboard
AI for Science (Sections 6.8, 6.14)		
MOF generation	Chemical control 92.5 / 100.0 / 74.5%; AUC 0.594→0.833; simpler method wins at matched compute	External `MOFChecker`
Erdős–Gyárfás	Six proof-backed frontier updates; one falsified route retained	Proof checker
AI for research writing (Section 6.10)		
Paper production	Six pipelines to submission; 254 missions, 16 Stage rollbacks	Venue format
Research inventory	41 de-duplicated artifacts across six programs	Public manuscripts
AI infrastructure (Section 6.11)		
FlashAttention-4 dLLM	19/19 parity; 5–14% of native; 2.5–4.2× over Triton; under 80 CNY	Bit-exact reference
SGLang integration	36 files, `+11,263`; 160/160 exact; 5.108× at BS8	Real-hardware parity
Upstream kernels	4 `fla-org` submissions: RWKV6 1.21× (#1045), KDA 1.29× (#1128), SM100 fix restoring 76 tests (#1109), AttnRes 1.102× (#1114)	**Outside maintainers**
MiniMax-H3 local	62 GiB model on 24 GB M4 Pro; 6.159× on one A6000	Published artifacts
AI for AI (Section 6.12)		
Runtime self-evolution	Parameter-invariant axis: 21% fewer solve Tokens, 15% less active time at maturity	Longitudinal ledger
1B pretraining stack	Parameter-changing axis: from-scratch pipeline on 8×B200 built and run end to end	Training convergence
Hardware (Section 6.13)		
ACE-2 accelerator	13,914/13,914 commands; 0.614 mm^2, WNS/TNS 0.00 ns, exclusions published	Static timing engine

Table 8. Campaigns grouped by the authority that decides whether the result counts. Rows in the first block are scored by harnesses the runtime executes; the last two blocks are decided by external checkers, external reviewers, and physical tooling that the runtime does not own and cannot tune.

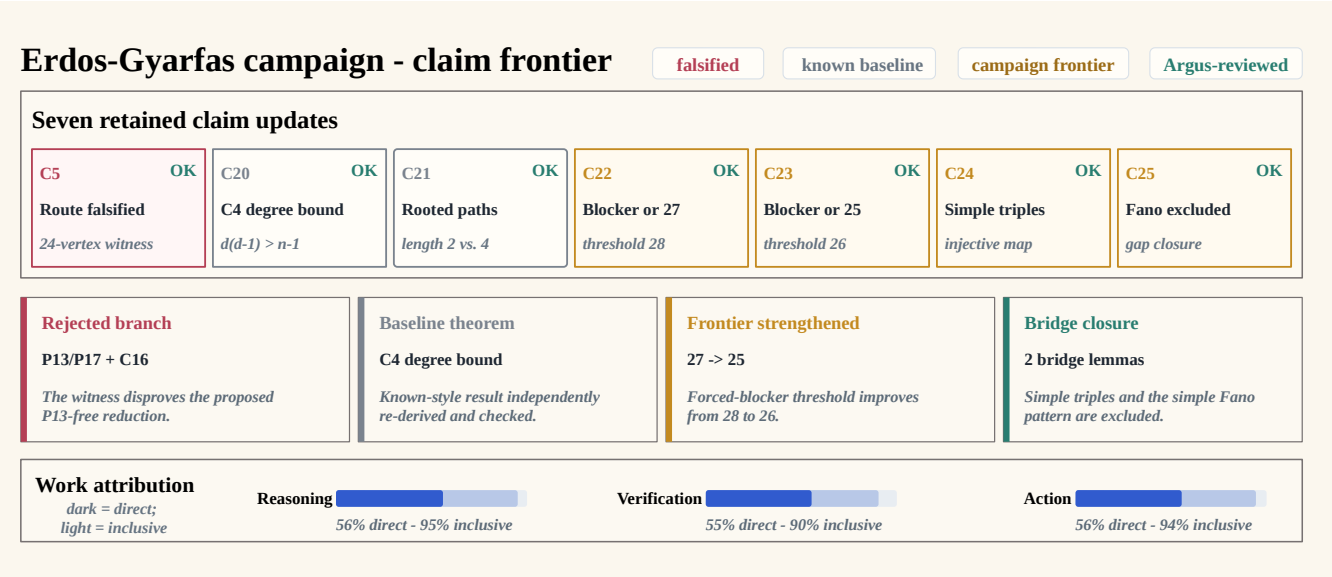

Figure 6. Reviewed claim frontier in the representative mathematical campaign. Seven retained updates progress from a falsified route (C5), through two independently re-derived baselines (C20–C21), to four campaign-frontier results (C22–C25), including the 27-to-25 strengthening and two bridge lemmas. The middle cards expose the corresponding artifacts, while the bottom bars separate direct frontier-changing work from an inclusive measure that also credits validation and state consolidation. Check marks indicate internal Argus review, not external peer review or a novelty claim.

The trace makes the work allocation concrete. The Manager recovers the campaign and owns stage transitions. The Planner first selects a cheap falsification test, then proposes changing the success contract from "collect observations" to "produce a proof"; the Manager admits that refinement, and later requires new missions to advance the retained result rather than restart from an easier fact. The Engineer performs the domain work—source retrieval, executable checks, proof writing, and result packaging. The Reviewer rejects an overstated route, requests the missing checks, and certifies the bounded theorem recorded by the artifacts.

In the reported trace, the campaign retains one falsified route and six proof-backed deltas, including one stricter bound and

two bridge results. Accepted work changes the next Planner input, rejected work remains available as a dead branch, and open blockers become explicit future tasks. The trace therefore captures a continuous research path rather than a sequence of isolated answers.

6.9 Efficiency Attribution in the Mathematical Case

We apply Equation 5 to the theorem-production window covering Rounds 12–17. The analysis contains 18 bounded missions: six changed the theorem frontier, four directly checked or re-certified mathematical results, seven consolidated review state and documentation, and one overlap-analysis mission was interrupted without an admitted result. Table 9 uses mission purpose as a coarse attribution rule for process allocation.

Efficiency view	Direct	Auxiliary	Failed/no-op	Unit and interpretation
Reasoning	56.0%	39.1%	4.9%	Engineer reasoning-output Tokens; direct means a frontier-changing proof mission.
Verification	55.4%	35.0%	9.6%	Reviewer reasoning-output Tokens; direct means proof comparison or re-certification.
Action	55.6%	38.9%	5.6%	Mission count; direct comprises six frontier missions and four verification missions.

Table 9. Efficiency attribution for the representative mathematical case. Auxiliary work includes result packaging, checkpoint updates, and documentation consolidation.

The normalized interpretation is concrete. Of every 100 Engineer reasoning-output Tokens, approximately 56 occurred in frontier-changing proof missions, 39 in verification or state-maintenance work, and 5 in the interrupted branch. Of every 100 Reviewer reasoning-output Tokens, approximately 55 directly checked a proof package, 35 maintained formal review state, and 10 were spent in the interrupted analysis. The strict action score is $10/18 = 55.6\%$; if the seven successful auxiliary missions are also credited, the inclusive score is $17/18 = 94.4\%$. Cost weighting makes the interruption more visible: the strict and inclusive action scores become 56.1% and 89.7%, respectively.

The trace also clarifies what the numerator should reward. The early witness check that killed a proposed reduction counts as an efficient action because it pruned a false route before a long proof attempt. In Round 15, selecting one internal triple, deriving the degree bounds, and closing the counting inequality are direct reasoning; rewriting the result into the structured claim record is auxiliary work. Reviewer inspection of those steps is direct verification, whereas updating the working checkpoint is coordination overhead. This separation prevents activity volume from being mistaken for research progress while preserving necessary coordination as a visible cost.

6.10 Objective Revision at Campaign Scale

The benchmark suite evaluates task endpoints, while the mathematical trace follows one research objective in depth. A complementary question is whether the same runtime can carry multiple research programs through the complete paper-production lifecycle. We therefore reconstruct six projects spanning evaluation reliability, vision–language matching, test-time adaptation, GUI agents, multimodal hallucination, and model quantization. The initial objectives and compute environments were provided externally; the retained Argus records identify the Manager, Planner, Engineer, and Reviewer as the actors responsible for Stage transitions, experiments, manuscript construction, review, and submission checks.

Figure 7 summarizes the portfolio. All 6 of 6 canonical pipelines reach the final submission Stage. In aggregate, they span 640 campaign-hours, 254 bounded missions, 576 Engineer rounds, 286 Reviewer revision verdicts, 89 session rolls, and 16 Manager rollbacks. Campaign-hours are summed across projects and are not calendar time because several projects overlap.

The trajectories are inconsistent with a one-pass account of autonomous paper writing. Each manuscript states a scoped research question and a task-native outcome rather than merely presenting a generated document. The portfolio includes uncertainty-aware leaderboard reporting, a failed compositional gate, an over-restrictive adaptation controller, a narrow GUI reliability gain, a failure-mode audit of multimodal wrappers, and a negative result for static quantization proxies. Several projects therefore become diagnostic or negative results after their original positive hypothesis fails. Reviewed Stage state makes that scientific pivot explicit instead of silently replacing the failed branch.

Figure 8 resolves the multimodal-hallucination campaign in greater detail. During a dense 12-hour search window, seven rollback decisions reject baseline coverage or proposed mechanisms that are unreproduced, base-identical, or missing their preregistered signal. Planner then proposes changing the claim from a positive mitigation method to a diagnostic negative-results audit, and Manager admission makes the refinement authoritative. Engineer completes a five-method by three-benchmark matrix with 4,500 official-scored rows; Reviewer binds the resulting no-op and degradation claims to those outputs. Two later submission checks return the project to benchmark to repair scorer provenance and GPU telemetry before the final 10-page AAAI-formatted package completes the pipeline.

The process layer is similarly nontrivial. Even the shortest case requires 28 Engineer rounds, 19 Reviewer revisions, and 13 session rolls over 20.8 hours. The two AAAI-formatted cases finish as 10-page anonymous submissions, while the four ACL-formatted cases contain 10–13 pages. Across all six projects, the retained academic, layout, and infrastructure histories contain 436 automated review snapshots. These are process records rather than external assessments of paper quality.

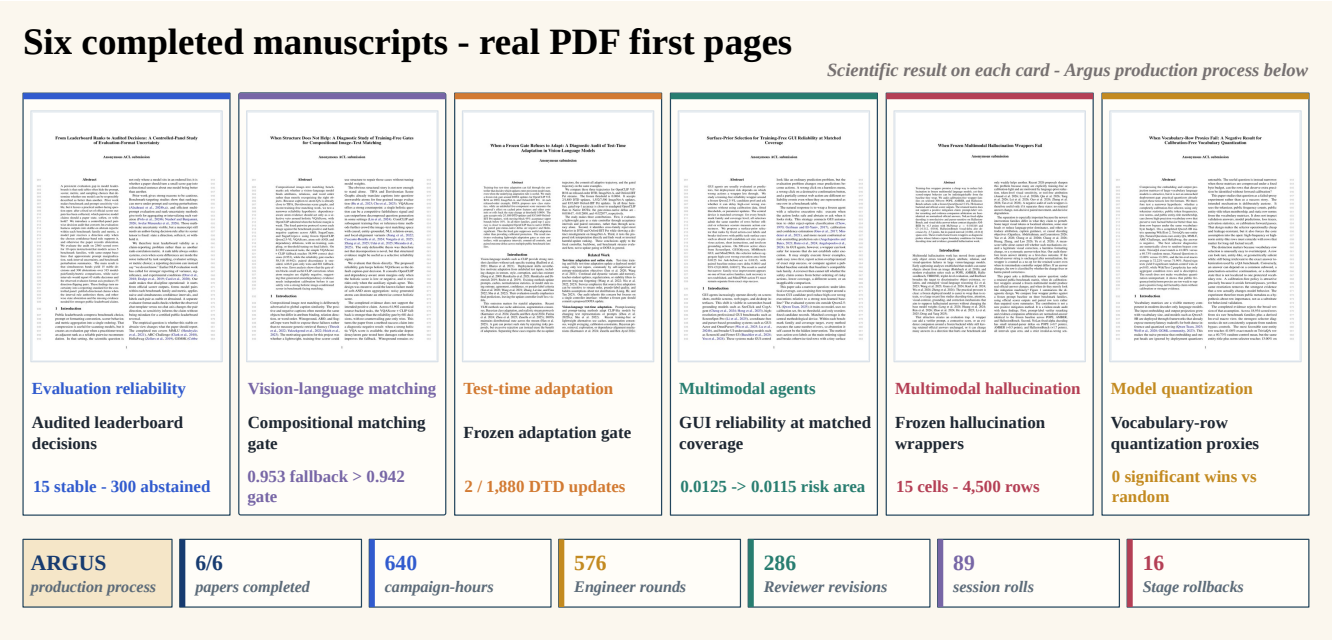

Figure 7. Scientific outcomes and production load across six paper campaigns. Each card pairs the final manuscript's first page with its domain, scoped research question, and principal task-native result, including diagnostic and negative findings. The bottom strip reports the shared production process: six completed pipelines, 640 aggregate campaign-hours (rounded from 639.55), 576 Engineer rounds, 286 Reviewer revisions, 89 session rolls, and 16 Stage rollbacks. Manuscript completion does not imply venue acceptance, novelty, or external peer review.

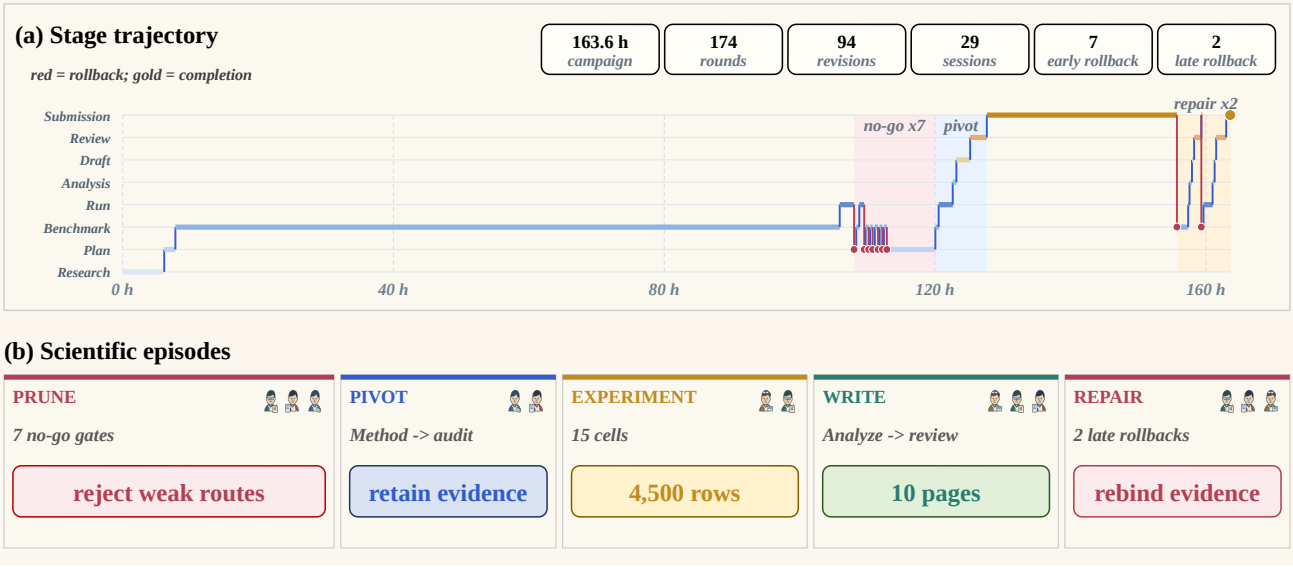

Figure 8. Representative 163.6-hour paper-production trajectory. Panel (a) plots the Manager-controlled Stage against campaign time: seven early approaches were dropped prune weak method routes, the project pivots from a positive method claim to an audit, and two late rollbacks repair the submission evidence. Panel (b) reduces the trace to five scientific episodes—prune, pivot, experiment, write, and repair—while retaining the 15-cell, 4,500-row experiment and the final 10-page manuscript.

The case study also exposes a consistency failure. 5 of the six projects close with a passing final assurance snapshot; the sixth, the compositional-matching pipeline, reaches a complete submission Stage and produces its final PDF while its last stored assurance object remains `BLOCKED` on a Manager-stage authority check. This discrepancy does not change the paper artifact, but it shows that derived certification state can lag the canonical pipeline state. Public release automation should reconcile those two views before presenting assurance as a final verdict.

6.11 Frontier Infrastructure: Working Where the Verifier Is Unforgiving

The campaigns above are evaluated by a proof checker, a venue format, and an official benchmark harness. This subsection covers the vertical where we have the most external evidence and where an unsupported claim has the shortest life: GPU kernel

and serving-infrastructure engineering. A kernel is either bit-accurate against a reference or it is not; it is either faster on the same hardware with the same shapes or it is not—the pairing of correctness and speedup that KernelBench formalizes as an evaluation target (Ouyang et al., 2025); and when the work is submitted upstream, the party deciding whether it is acceptable is a maintainer with no stake in this report.

The vertical spans both ends of the hardware range: datacenter accelerators where the constraint is throughput, and consumer devices where the constraint is whether the model fits at all.

This domain is also the sharpest test of the Driver framing, because it is where the objective moves the most. None of the campaigns below could state their target precisely at the start. The FlashAttention-4 adaptation did not know which of six mask families would fit the tile scheduler, and spent its first several rounds discovering that its initial data path was catastrophically wrong. The SGLang integration did not know that a cross-batch determinism defect existed until batch size eight produced drifting images. The correctness fix described below began as a performance investigation and became a correctness one. Each is a dense-intelligence task in the sense of Section 3: invention rather than retrieval, many dependent iterations, and a task-native verifier strong enough that the runtime can judge itself without appeal to its own narrative.

6.11.1 Diffusion-LLM Masks on FlashAttention-4

Diffusion language models do not use causal attention. They require block-causal, prefix-full, prefix-causal, prefix-hole, sliding-window, and cache-only mask families, plus compositions such as prefix-full with holes under a sliding window. The task was to carry all of them into a current-generation FlashAttention-4 CuTe DSL kernel across both Ampere SM80 and Hopper SM90 paths, with paged and dense KV layouts, without surrendering the performance that motivated using FA4 in the first place.

The result is 27 missions and 225 calls over 21.85 hours of model compute, producing 36 changed files (`+5,814/-14`). All 19 parity cases pass across both architectures, paged and dense dispatch, composed masks, and cache-only inputs, with maximum deviation of 3.9×10^{-3} at bf16 rounding scale and cosine similarity at or above 0.9999994; CUDA Graph replay is bit-identical to direct invocation. Against the production Triton path the adaptation is 2.5–4.2× faster end to end, or 1.6–2.6× when both sides enable CUDA Graph. Against upstream native FA4 it is slower by only 5–14%, and that residual is almost entirely a fixed 3.6–5 µs Python wrapper cost: the attention kernel itself runs within 2–4 µs of native.

Two facts about the process matter more than the endpoint. First, the campaign's early dense-plus-block-sparsity route was not slightly worse but 4.9–29.6× slower than native, with dense KV gather alone reaching 884.5 µs at decode. The final result came from recognising that this data path was wrong and abandoning it, not from optimising within it—and the rejected route, with its evidence, is retained in the skill library rather than discarded. Second, the entire campaign cost under 80 CNY of model spend, distributed across three vendors by role: an open-weight model carried 82% of Engineer compute while separate models served Manager triage and Planner decomposition, all under one role contract. A specialist infrastructure team had previously worked this direction for over three months; that figure comes from a practitioner interview covering work of different scope, not a matched trial.

6.11.2 A Unified Model into a Production Serving Stack

Integrating SenseNova U1 into SGLang required five workstreams that are usually staffed separately: architecture and weight mapping for language and multimodal understanding; native bounded flow-matching image generation; scheduler-owned text-to-image-to-text interleaving with correct KV lifecycle; CUDA Graph flow-prefill with concurrency determinism; and an OpenAI-compatible API with cancellation, disconnect, and multipart limits.

The delivered change is 36 files and `+11,263/-72` lines across 14 commits. Acceptance was established on real hardware rather than by structural review: 1,116 tensors load with zero missing or unknown, visual question answering is exact on 160 of 160 generated tokens, concurrency-8 is exact on 8 of 8, and throughput improves 5.108× at batch size 8 with 2.276× on text-to-image and 3.618× on image-conditioned generation. A cross-batch determinism defect that caused image drift at batch size 8 was found, localised, and fixed, with the invariant then holding across all batch sizes.

The comparison in Section 6.3 is against this task: one engineer working with a turn-by-turn coding agent invested over 60 hours without completing it, while a blind agent run stopped after roughly 1 hour 21 minutes with a CPU-oriented draft carrying no real weights, no GPU parity, and no benchmarks. Argus completed it within a 24.14-hour envelope under an operator who was not an infrastructure specialist. What separates the outcomes is not the ability to write plausible integration code but the willingness to keep going through the expensive second half: real checkpoint loading, exact parity, concurrency determinism, service-level safety, and the revisions that external review demands.

6.11.3 Upstream Review by Maintainers Who Owe Us Nothing

Kernels authored by Argus were submitted to `flash-linear-attention`, a widely used implementation library for linear-attention architectures. Two properties make this the strongest external evidence in this report: the acceptance standard is written by someone else—the repository requires dependent-test discovery and same-hardware before-and-after benchmarks—and the reviewers had no involvement in producing the work.

Four submissions are recorded here, spanning performance work and correctness repair. A TileLang RWKV6 forward-intra backend (PR #1045) improved forward latency from 0.199 ms to 0.168 ms (1.18×) and forward-plus-backward from 0.900 ms to

0.747 ms (1.21×) on an H100 NVL at B=8, T=1024, H=8, D=64, gated by 14 backend tests and a 13-case frozen verification gate, with unsupported dtypes, dimensions, and layouts rejected by a backend verifier rather than silently mishandled. Its submission states plainly that the implementation, optimisation loop, correctness validation, and performance evidence were completed autonomously by Argus, with only the target and hardware constraint chosen by the operator. That sentence was reviewed and accepted along with the code by an outside maintainer, which is a different kind of claim from one asserted in a report about oneself. It received no inline change requests.

A second submission (PR #1128) accelerates `chunk_kda` training with four TileLang kernels covering intra forward, dAv, fused dq/kg, and intra backward, reaching 1.29× over Triton across the four stages on B200. The instructive detail is what was not claimed: the best single stage reached 1.541×, but automatic dispatch was enabled only for the one workload with both verified correctness and a repeatable end-to-end gain, which measures 1.078–1.099×. The larger number was available and was not used to characterise the result.

The third (PR #1109) is two lines. On SM100, `chunk_kda` backward autotuning triggered an illegal memory access that poisoned the CUDA context, so the test file could not run to completion; excluding `BK==32` configurations at 4 and 8 warps restores a full pass at 76 tests. There is no speedup to report, because before the fix nothing could be measured at all. The reviewing maintainer independently re-derived that 24 autotune candidates survive the filter, confirmed no empty-configuration risk, verified that the described scope matched the diff, and diagnosed two red CI checks as infrastructure flakes rather than test failures, then approved with two non-blocking suggestions and no required changes. The fix had also been narrowed by its author from all Blackwell devices to the measured SM100 target—the same discipline of publishing the perimeter of a claim that appears in the ACE-2 certificate of Section 6.13.

A fourth submission (PR #1114) parallelizes the long-sequence `AttnRes` reduction, specializing forward and backward kernels on the exact residual count and distributing the dq, dw, and optional dow reductions across the flattened token dimension for $N \geq 16384$. Across five bfloat16 shapes on B200 it reaches a geometric-mean speedup of 1.102×, best case 1.237×. The submission reports its worst row, 1.033×, alongside the mean, and states that no broader hardware coverage or end-to-end model speedup is claimed.

6.11.4 The Same Discipline at the Other End of the Hardware Range

The campaigns above optimize kernels on datacenter accelerators. This one inverts the constraint: the model is fixed, and the hardware is what an individual actually owns. MiniMax-H3 is an audio-video generation model whose BF16 diffusion transformer alone occupies roughly 62 GiB, well beyond the memory of the two targets below. The task was not to quantize or distill it into something smaller, but to make the original weights run.

Apple M4 Pro, 24 GB unified memory. Argus adapted the upstream inference path to Apple MLX with block-wise streaming load and prompt release, so the full BF16 diffusion transformer and BF16 text encoder are never resident at once. The system produces 1344 × 768 video, 124 frames at 24 FPS with 5.17 seconds of stereo audio, in 47 minutes 58.7 seconds end to end at a peak memory footprint of approximately 15.8 GB—roughly a quarter of the model's nominal size, on a laptop-class device.

One RTX A6000, 48 GB. The same model was brought to a single Ampere-class workstation GPU at full FL2VA checkpoint fidelity, 1344 × 768, 124 frames, 24 FPS, 32 kHz stereo. The campaign then optimized it under measurement discipline. A BF16 dense warm baseline over N=10 runs establishes 1,792.202 s. An eight-step Turbo route reaches 290.998 s, a 6.159× speedup, and is adopted as the practical default. A sparse-attention route at r=8 is reported separately at a formal N=10 HTTP-time improvement of 15.203%, and a matched formal N=10 comparison on 30-second final audio-video yields +4.326%.

What makes this a runtime result rather than a port. Both targets are public repositories, and both record what was rejected as well as what was kept. Candidates that degraded output quality, produced no gain on the critical path, or rested on N=1 evidence are marked rejected rather than folded into the headline number, and quality failures are published alongside the speedups. The distinction matters because the two accepted routes are not equivalent: the Turbo path is an approximate decoding schedule and is labeled as such, not presented as lossless BF16 acceleration. Acceptance ran through paired N=10 comparisons on the same physical GPU, a 24-case quality suite, and memory and audio-video integrity checks.

This campaign exercises a different part of the runtime than kernel optimization. There is no leaderboard and no official verifier; it had to construct its own measurement protocol, decide which improvements were real, and publish the boundary of each claim. That is the same discipline the ACE-2 certificate applies to hardware and the materials campaign applies to chemistry, in a setting where the deliverable is simply that a model too large for the device runs on it.

6.11.5 Why This Vertical Is Evidence About the Runtime

These campaigns share a shape that a score-climbing system does not produce. One result is a rejected route retained as reusable knowledge. One is a deliberate choice to report the smaller honest number over the larger available one. One is a correctness fix with no performance story, whose entire value is that a test suite that could not finish now finishes. Together with three merged or reviewed upstream submissions and a serving integration validated on real hardware, they indicate that the unattended

hours produce work that survives contact with people who did not write it—which is the property that distinguishes delegated autonomy from generated volume.

6.12 AI for AI: The Second Axis of Self-Improvement

Every campaign so far produces something outside the runtime: a kernel, a manuscript, a crystal structure, a chip. This one points inward. The artifact is the machinery that produces models, which is the machinery that produces Argus itself.

Self-improvement comes in two kinds, and conflating them is why claims in this area are hard to evaluate. *Parameter-invariant* self-improvement changes what the system knows, retrieves, and obeys while the weights stay fixed: memories, skills, procedures, verifiers, routing decisions, and retained failures. This report measures it directly—Section 4 specifies the admission gate, and Section 6 shows mature waves using 21% fewer solve input Tokens and 15% less active time per task than startup, under a backbone that never changed. Systems in this family evolve their context, their retained experience, or their own scaffolding code under empirical feedback (Zhang et al., 2025b; Yin et al., 2025; Zhang et al., 2025a). *Parameter-changing* self-improvement alters the weights themselves. The two are complementary rather than competing: the first compounds within a fixed capability ceiling, and the second is what moves the ceiling.

Argus operates on the first axis today, and that is the axis where evidence is actually obtainable, because a state update has an owner, a precondition, and an audit trail. But the ceiling is real. No amount of accumulated skill makes a frozen backbone reason better than it can, and a system confined to that axis improves asymptotically toward its own limit. Reaching the second axis requires participating in the production of weights, which means owning a pretraining stack: data pipeline, training loop, stability, throughput, failure recovery, and the judgement to decide which of those is the current bottleneck.

Argus built and ran that stack. On an eight-GPU B200 node, the runtime took a 1B-parameter model from scratch through data-quality and pipeline construction, training stability and speed, GPU utilization and bottleneck analysis, engineering quality and failure recovery, and successive optimization rounds—with the operator driving it having prior experience only in post-training, not in pretraining infrastructure. The pipeline runs end to end and continues under the runtime's own supervision.

What this is evidence for, and what it is not. The claim here is deliberately narrow, because the interesting failure would be to overstate it. Building a pretraining stack is not yet parameter-changing self-improvement; it is the entry condition for it. Until a system can produce weights, the second axis is unavailable in principle, not merely unattempted.

Three steps remain, and each is genuinely hard. The trained model is not yet good enough to serve as a role backbone. The training data is a public corpus rather than the runtime's own retained trajectories, which Section 7.7 argues is the natural source and which the first axis has been accumulating all along. And no model produced this way has been placed into a role and measured for whether the runtime that built it improved. Only that last measurement would demonstrate a closed loop rather than a capability.

The two axes are not independent, which is the point of reporting them together. The trajectories that parameter-invariant self-evolution retains—objectives, actions, measurements, revisions, verdicts, and the routes that were refuted—are exactly the supervision a training stage would consume. A runtime that improves its own state while producing a structured record of how it did so is, in effect, generating its own training corpus as a side effect of operating. Owning the stack that could consume it is what makes that observation more than rhetorical.

Why this vertical is a hard test of unattended operation. Pretraining is the least forgiving setting in this report for a Driver that cannot judge its own progress. A multi-day run commits resources irreversibly, and its characteristic failures—a data pipeline defect, a silent throughput regression, an instability that only manifests after a loss curve has looked healthy for hours—surface long after the decision that caused them. There is no fast verifier to consult, no test that turns green, and the cost of noticing late is measured in GPU-days. This is precisely the regime in which Question Q1 of Section 1—is this actually working—cannot be answered by checking an exit code, and where a runtime that persists without judging is worse than one that stops.

6.13 RTL to Mapped Synthesis: the ACE-2 Accelerator

The campaigns above produce software, proofs, and manuscripts. ACE-2 asks whether the same runtime holds when the artifact is hardware, where an unverified claim survives exactly as long as it takes someone to run the tool that contradicts it. Prior work on language models for hardware has concentrated on generating and evaluating RTL at the module level (Liu et al., 2023b) and on adapting models to chip-design corpora (Liu et al., 2023a); the question here is different, namely whether a runtime can carry an accelerator from model structure through functional closure and mapped synthesis as one continuous campaign. Argus specified ACE-2, wrote its RTL, built its verification environment, and drove it through mapped synthesis and static timing without a human author of record for any of those artifacts. The result is an inference accelerator that executes Qwen2.5-0.5B end to end in W4A8, certified against a fixed scope.

Functional closure. The accepted design runs the full 24-layer Qwen2.5-0.5B W4A8 command integration. Layer 0 matches the reference on all 18 ordered operators exactly, and the two-token runtime completes 13,914/13,914 commands over 1,240,410,384 simulator cycles with generated token identifiers $[0, 0]$ and no first failure recorded. The runtime package, command log, and progress journal are each bound by content hash, so the functional claim resolves to specific bytes rather than to a summary.

Physical closure. Canonical SKY130 HD mapped synthesis and OpenSTA at TT 25 °C / 1.80 V report 62,283 cells and 0.614 mm^2 of non-SRAM area against an operator-set cap of 2.0 mm^2, with +0.6966 ns detailed setup slack, 0.00 ns worst negative slack, 0.00 ns total negative slack, and a 10.000 ns clock period. Both operator-owned targets—the area cap and the 100 MHz floor—pass without relaxation, and the passing packet is the sole exactly-once canonical run rather than the best of several attempts.

What the gate produced. The certification is dated 2026-08-04 and was issued by a Reviewer that did not perform the work, bound to one RTL tree hash. Each accepted RTL repair carries its own reviewer binding together with the originating and resulting tree hashes, so the certified design is reachable from the audit trail rather than asserted alongside it. The more informative artifact, for the argument of this report, is what the certificate refuses to say. It enumerates its own exclusions: no routed timing, no power signoff, no DRC/LVS, no GDS or tapeout, no silicon validation, no generation beyond two tokens, no external deployment interfaces, and no FPGA prototype. That list was produced by the same gate that admitted the positive claims. An unbounded runtime would have reported a chip; this one reported a chip and the exact perimeter of the evidence supporting it.

ACE-2 therefore extends the pattern of Sections 6.8 and 6.10 into a domain with an unusually unforgiving verifier. The functional and physical claims are narrow by construction, and the scope conditions are part of the output rather than a caveat added afterward.

6.14 AI for Science: Simplifying a Published Method in Materials Generation

The verticals above are evaluated by a proof checker, a venue format, and a synthesis toolchain. This one is evaluated by an external chemistry validator that the runtime does not own, against two published models it did not write. The campaign targets metal–organic framework (MOF) generation—a setting where generative models have advanced quickly for inorganic crystals through equivariant diffusion and property-conditioned sampling (Jiao et al., 2023; Zeni et al., 2023), and flow matching has been extended to metal–organic frameworks specifically (Kim et al., 2024), but where chemical composition, building units, and three-dimensional structure remain tightly coupled, so progress requires both materials-chemistry understanding and the engineering capacity to run training, inference, and evaluation at scale. Argus organized its own training and large-scale inference on four- and eight-GPU NVIDIA B200 nodes across this campaign, reproducing two published models, diagnosing their capability gaps, running the experiments, and revising its route when the evidence demanded it.

The outcome is the sharpest instance in this report of the pattern Section 4 describes: the admitted result is *simpler* than the method it replaces, and it was admitted only after a confound in the original comparison was removed. It is also the clearest case of a research campaign in which no benchmark score was available to steer by—the objective was to understand why a generator fails, and the right measurement had to be constructed before the question could be asked.

Adding a control the base model lacked. MOFFlow-2 generates building-block sequences with an autoregressive language model and then places them by flow matching. Its published conditioning accepts a single continuous property and cannot address chemical identity. Argus added metal element, node nuclearity, and ligand family as three discrete condition tokens read through cross-attention, each with its own `ANY` token and independent condition dropout, so one model serves unconditional, partial, and joint requests. Continuing to train the full decoder raised validation loss, so the language model was frozen and only the condition encoder and six cross-attention modules were updated: 12.69M trainable parameters of 75.8M, or 16.7%. Over 3,300 balanced condition requests weighted toward rare chemistry, adherence reaches 92.5% on metal (17.4% under a permuted-condition control), 100.0% on nuclearity (24.3%), and 74.5% on ligand family (39.0%). The effect concentrates in the tail the corpus under-represents: nickel rises from a 2.1% corpus base rate to 83.7%, trinuclear nodes from 3.9% to 100.0%, and pyrazolate ligands from 2.0% to 46.0%. Unconditional structure-level validity also improves, from 30.61% for the local MOFFlow-2 reproduction to 37.12%. The control is not free: at guidance w=2 structural validity falls to 17.18%, and the campaign records that trade-off rather than reporting only the adherence numbers.

Replacing the score, then questioning the method. A failure census over 19,483 paired structures found that the published steering potential penalizes atom overlap, while the dominant failures of the all-atom model are the opposite: under-coordination, fragments, and floating components. Argus built a scoring function aligned to the external `MOFChecker` criteria instead, with radii derived from covalent and van der Waals tables rather than tuned by hand. Against the external validator it separates passing from failing structures at AUC 0.833, versus 0.594 for the overlap-only potential.

That gain then exposed a measurement problem. Enabling the published Feynman–Kac steering also switches the integrator from ODE to SDE, so the reported improvement mixes two changes. Separating them attributes +1.92 points to the integrator alone, which does not reach significance (p=0.238), and +9.00 points to the score inside a fixed SDE (p=2.0×10^{-9}). With that confound removed, the runtime tested whether the particle interaction was doing any work at all, by holding model, integrator, score, forward-pass budget, and the same 989 crystals fixed and simply generating K independent trajectories and keeping the best-scoring final structure.

Forward passes	Feynman–Kac	Argus best-of-K	Difference
$K = 4$	—	53.39%	—
$K = 8$	52.38%	**55.21%**	+2.83
$K = 16$	57.94%	**59.76%**	+1.82
$K = 32$	59.96%	**61.38%**	+1.42

Table 10. External `MOFChecker` pass rate on 989 paired crystals at matched forward-pass budget, model, integrator, and score. Single-sample SDE decoding passes 43.38%. Independent generation followed by best-of-K selection beats the published Feynman–Kac steering at every budget; the K=8 difference survives a paired test (p=0.0486). Best-of-4 at 53.39% already exceeds Feynman–Kac at K=8 while spending half the compute.

Table 10 reports the result. The mechanism is visible in the particle statistics: at λ=2 the median effective sample size is approximately one, and 72.1% of crystals collapse to a single particle after resampling, after which the remaining SDE noise does not restore diversity. The published method pays for K forward passes and then discards most of the exploration it bought. Keeping the trajectories independent costs nothing extra and retains it.

Where the campaign stops. This is a research candidate, not a state-of-the-art claim, and the runtime records why. The 989-crystal subset is the same data on which the score was designed, so the numbers above are not an independent holdout. The full 19,483-structure leaderboard has not been run. The matched-compute MOFFlow-2 comparison was interrupted by a parameter error that enumerated all 19,792 test structures instead of the intended 1,500, and its export is incomplete. Conditional sequence generation and best-of-K structure prediction have not been joined into one end-to-end system. No relaxation, machine-learned interatomic potential, or DFT check has been run, so every number here is scored by the same validator the selection optimizes against, which is exactly the failure mode Section 8 names for verifier-shaped objectives. The campaign therefore holds at a candidate version rather than declaring a release, and the outstanding items are enumerated rather than deferred silently.

What this vertical contributes to the argument of the report is not the pass rate. It is that a runtime given a published method, a fixed compute budget, and an external validator arrived at a smaller method than the one it started from, and that the step which made this admissible was removing a confound rather than adding a component.

7 Analysis and Discussion

7.1 Reading the Duty-Cycle Result

The central measurement of this report is that the Driver's seat stayed empty: 38 human requests across 1,548 hours, one every 40.7 hours, at 95.1–98.7% duty cycle with work in hand. What that buys, and what it does not, is worth separating.

The requests concentrate where a person is genuinely required, which is a stronger result than their scarcity. Two thirds of them report broken infrastructure or ask for authority the machine should not grant itself, and both categories would survive any improvement in agent reasoning; a corrupted container store is not an autonomy problem, and a system that quietly provisioned its own credentials would be a worse system, not a better one. The July 2026 Hugging Face intrusion is that claim demonstrated at cost: a highly capable agent, unattended and unbounded, spent four and a half days acquiring access it was never granted in order to reach a benchmark answer (Hugging Face, 2026). Capability was not the missing ingredient there, and the same holds across the broader pattern of model capability folded into malicious operations (OpenAI, 2026a). The 13% that are research judgment is the number that would fall if the underlying model improved; the other two thirds would not.

It does *not* show that unattended time is free. The residual gap between the observed duty cycle and unity is dominated by infrastructure failure and lost context files, which is an argument for process supervision and durable storage rather than for a stronger model. The lowest-scoring campaign in Table 6 is the clearest case: its ceiling was a GPU driver, not its planner.

What it does show is a change in what bounds the work. Under turn-by-turn supervision, the effective operating window of an agent is its operator's attention, and no amount of accuracy extends it. Once completion has an owner other than the executor, the window becomes a property of the runtime and its infrastructure.

The substitution is worth stating plainly, because it is a change of regime and not of constant. A system bounded by operator attention scales with people: to run twice the research, staff twice the operators, and the second operator is as scarce and as

non-parallel as the first. A system bounded by infrastructure scales with infrastructure, which is procurable. Nothing here makes research cheap, and nothing here removes the human from the loop—38 requests were still made, and the ones that matter are exactly the ones a machine should not settle. What it removes is the requirement that a person be *present* for the work to continue, and that requirement is what has kept the output of agentic systems tied to the length of a working day. Sections 6.3 and the benchmark results below address the obvious follow-up—whether the hours produce anything—because duty cycle alone would describe an expensive idle loop equally well.

7.2 System-Level Performance

The seven-benchmark suite shows one runtime operating across software repair, GPU-kernel optimization, model training, research tasks, and data synthesis; a separate mathematical vertical trace examines proof-oriented research depth. On SWE-Bench Pro, Argus reaches approximately 78% accuracy compared with 59% for Direct Copilot at 1.41× aggregate Tokens. The comparison pairs additional computation for planning, execution, and review with a 19-point accuracy difference.

The longitudinal run shows how this cost changes with accumulated state. Mature W19–22 uses 21% fewer solve input Tokens and 15% less active time per task than startup W1–6. The curve remains non-monotone: W13–18 combines low Token use with longer execution, while W23–24 contains a late concentration of difficult tasks. The operating profile changes over time as shared state expands, while task-dependent variation remains visible.

7.3 Reviewer as Selective Error Correction

The Reviewer requests revision on 43 SWE-Bench Pro tasks. After another Engineer round, 34 pass the official verifier and 22 complete the stricter `continue`→revision→`done` loop. These trajectories show the Reviewer functioning as an error-correction stage rather than a terminal commentary layer.

Reviewer routing is adaptive: Reviewer-routed tasks use more Tokens and active time than self-reviewed tasks. The observed routing concentrates independent review on the more resource-intensive part of the workload.

7.4 Autonomous Research Production

The six-paper case study extends the endpoint results into a complete research lifecycle. All six canonical pipelines reach submission completion across six domains and two venue formats, but none follows a one-pass path. Reviewer revision verdicts, Stage rollbacks, and session rolls are common rather than exceptional. The final manuscripts therefore demonstrate persistence across research framing, experimentation, writing, review, and packaging—not merely the ability to emit paper-like prose.

The multimodal-hallucination trace illustrates the control mechanism. Reviewer gates reject seven weak method routes before they are scaled into an unsupported positive claim; Planner proposes reframing the accumulated failures as a diagnostic study; Engineer then completes the 15-cell canonical matrix; and Manager accepts two late rollbacks when submission checks expose scorer-provenance and telemetry defects. Role separation redirected the scientific trajectory here, and the record shows where: failure was retained as information, and only reviewed state moved the campaign forward.

The case also separates artifact completion from process certification. Five final assurance snapshots pass; the compositional-matching project retains a stale blocked assurance snapshot after its canonical pipeline and final PDF complete. This is a useful systems distinction: derived quality metadata must be reconciled with the authoritative Stage state before it can serve as a public certificate.

7.5 Persistent State and Verification-Gated Runtime Self-Evolution

The long-lived object in Argus is the campaign state rather than a provider transcript. Accepted results, failed routes, tools, skills, and current blockers remain available after session and process restarts. Equation 15 formalizes the information advantage of this record, while Equation 17 explains why the runtime keeps both a complete event history and a bounded working checkpoint.

This mechanism gives concrete meaning to *verification-gated fixed-model runtime self-evolution*. The base model remains unchanged, but an admitted mission can change the starting point, available procedures, verification obligations, or routing policy of later work. The admission gate binds each candidate to artifacts, task-native evidence, and an authorized commit; independent Reviewer judgment is one gate path rather than a requirement for every low-risk update. Rejected branches also participate when they are retained as verified exclusions: later missions can avoid repeating the branch and cite its failed evidence when proposing a pivot. Evolution therefore occurs in the runtime's admitted state and control policy, not in model weights or an unconstrained summary of prior conversation.

The two longitudinal views expose complementary parts of this claim. The SWE-Bench Pro trajectory measures how solve Tokens and active time vary as shared state accumulates, while the mathematical campaign shows direct semantic reuse: later missions consume earlier definitions, bounds, rejected routes, and bridge lemmas. Neither isolates a single mechanism: separating wiki from skills from routing requires replaying matched tasks against frozen state, which we list as future work.

7.6 Endogenous Harnessing: When a Plan Becomes a Constraint

A harness is supposed to bound what a system may do. The instructive failure is when a system manufactures its own harness out of a decision it made earlier, and then obeys it. We call this *endogenous harnessing*. It is the specific way an unattended runtime goes wrong: not by running out of capability, but by continuing to execute, correctly and productively, against a target that later evidence has already undermined.

The mechanism follows directly from the two levels in Algorithm 1. A Planner proposes a strategy under the information available at planning time. That proposal is written into a mission, and from the first round onward the Engineer treats it as the task and the Reviewer treats it as the standard the work is judged against. What began as one agent's hypothesis has become an external constraint on every agent that comes after, including agents that now know more. The system stops behaving like a researcher continuously reconsidering the best route and starts behaving like a researcher bound by a note they wrote before the experiment ran.

A recorded verification trace shows the shape. The Planner required a `skip-zero` candidate. The Engineer satisfied that local objective. The Reviewer then identified a no-gap validator alternative that would have discharged the same obligation more directly. The better judgement arrived, was recorded, and did not change the task. The earlier, less informed decision won on authority rather than on evidence.[1]

Planning is not the problem here, and neither is bounding. The problem is category confusion between two things a plan can be. A plan can be a *contract*, which is binding and expensive to change, or a *falsifiable hypothesis*, which is supposed to be discarded the moment the evidence turns. Filing both in the same place gives a technical strategy the immovability that should belong only to a safety boundary. The correct division is sharper: frozen authority, prohibitions on granting new trust, and irreversibility limits are hard harness and should resist revision; the choice of route, validator, or representation is a technical bet and should be revisable by any role that produces evidence against it.

The channel, and why it was silent. The runtime's response is to give the disconfirming role a formal way to move the task boundary without letting it seize the authority to do so unilaterally. A Reviewer may emit `plan_signal=reconsider` together with the assumption it disputes, a concrete alternative, and an `authority_impact` declaring whether the change is technical, touches the Manager's contract, or belongs to the operator. The Manager adjudicates and returns one of keep, revise, replace, or ask-operator; the Planner then authors any replacement. Authority stays where it was, but evidence acquires a route to the decision it contradicts.

The instructive part is that this channel existed end to end and was nonetheless unreachable. The Reviewer's decision event is flat, while the parser read the plan fields only from a nested object, so a correctly formed challenge was discarded before anything could route it. The vocabulary compounded the failure: the token `reconsider` appeared in no role prompt, and the only worked example showed an invalid value. A campaign could therefore close round after locally correct round with nothing able to question the design that produced them—which is precisely the failure this section names. The defect was diagnosed and repaired during the preparation of this report by reading both payload shapes through one reader and naming the vocabulary, with the authority each impact level carries, in the Reviewer prompt.

We report this rather than quietly shipping the fix because the sequence is the argument. Endogenous harnessing is not a reasoning failure. In every instance we recorded, the system found the counterexample, identified the validator alternative, and understood the cascading invariant; it could not act on any of them in time because later, better-informed roles had no authority to revise the task boundary. The binding constraint on this class of system is methodological rather than cognitive, which is also why the remedy is a change in authority routing rather than a stronger model. This failure is specific to the unattended setting. With a human Driver, a badly framed plan is caught the next time the operator reads the output. Once the seat is delegated, nothing catches it unless the runtime itself provides the channel. Longitudinal evidence on how often the reopened channel fires is future work; the campaign tally now reports objective-level stagnation and cumulative spend as facts so that a stalled plan is visible before a human is asked to notice it.

7.7 From Runtime Learning to Model Training

The same trajectories can be transformed into supervised examples, preference pairs, and reinforcement-learning transitions. AgentInstruct, Agent-FLAN, and Agent Lightning provide complementary mechanisms for this conversion (Mitra et al., 2024; Chen et al., 2024; Luo et al., 2025). Argus contributes a structured source stream linking objectives, tool actions, measurements, revisions, and retained-state updates. A future training stage can use this stream to internalize recurring planning and verification patterns while the runtime continues to provide long-horizon coordination.

Section 6.12 supplies the other half of that path, and the relationship between the two halves is the substantive point. Self-improvement separates into a parameter-invariant axis, which this report measures, and a parameter-changing axis, which it does not. The first compounds within a fixed capability ceiling: retained knowledge, verifiers, routing, and refuted routes make later missions cheaper and better targeted, but no accumulation of state makes a frozen backbone reason beyond its limit. The second moves the ceiling.

[1] This trace is drawn from internal system-verification work. We report it as a described sequence rather than a public reproducible artifact.

The two are coupled in a specific way. Operating on the first axis produces, as a byproduct, precisely the supervision the second axis consumes—typed trajectories in which an objective, the actions taken, the measurements obtained, the verdict issued, and the routes abandoned are all recorded with their justification. A runtime that self-evolves while logging how it did so is generating its own training corpus while it works. Owning a pretraining stack is what makes that byproduct actionable rather than merely suggestive.

We separate the axes deliberately, because the gap between holding both and having joined them is where claims about recursive self-improvement usually become unfalsifiable. Three steps remain: a model good enough to carry a role, training on the runtime's own trajectories rather than a public corpus, and a measured improvement in the runtime that produced it. Only the third closes the loop.

This is what commits Argus to the second of the two routes in Section 2.4. We do not claim a system that autonomously discovers how to improve itself. We report a runtime that executes research-level work unattended across software, kernels, silicon, mathematics, materials, and manuscripts; that measurably improves its retained state under an unchanged backbone; and that has built the pretraining stack which is the precondition for participating in weight production at all. The stages of the model-development pipeline it has taken over, and those still held by humans, are enumerated rather than asserted—and that enumeration is what an honest claim in this area currently looks like. The useful test is not whether a system can write a paper about AI, but whether it can move model capability; when a system is not only the object being trained but also a subject doing the training, the interesting question stops being whether recursive self-improvement has started and becomes how far the transfer has gone.

7.8 Vertical Research in Mathematics

The Erdős–Gyárfás campaign exposes the four-role organization at research depth. The Manager preserves the objective and stage, the Planner converts the current frontier into bounded theorem tasks, the Engineer produces proofs and computational checks, and the Reviewer controls admission into durable state.

The campaign retains one falsified route and six accepted theorem-frontier updates. Later missions improve a bound and close two graph-realizability gaps by starting from the reviewed ledger rather than restarting from the original conjecture. This is the same runtime mechanism seen in software repair, applied to a domain where progress is expressed through definitions, lemmas, counterexamples, and proofs.

7.9 What Progress Looks Like Without a Score

Three results in this report would be invisible to a scalar objective, and reading them together shows what the vocabulary of Section 4.5 is for.

The mathematical campaign retains one falsified route alongside six accepted frontier updates. On a score, the falsified route is a week that produced nothing. In the ledger it is an eliminated branch with the evidence that eliminated it, and later missions cite it rather than re-entering it. The FlashAttention-4 campaign is the sharper case: its initial data path ran up to 29.6× *slower* than the native kernel. A system optimizing a latency number would have recorded its worst result of the campaign and kept tuning inside that route. What the runtime recorded instead was information gain about the data path, which is what licensed abandoning it rather than optimizing within it—and the final result came from that abandonment. The materials campaign inverts the usual direction of progress: the admitted method is *simpler* than the published one it replaces, and it became admissible only after a confound was removed from the original comparison. Removing a confound raises no metric at all.

None of these three are anomalies to be explained away. They are the ordinary shape of research, and a runtime that could only recognize a rising number would have mishandled each one—by discarding the retained route, by continuing to optimize a route it should have left, and by never running the comparison that made the simpler method visible.

7.10 Why Infrastructure Is the Load-Bearing Vertical

Of the domains in this report, GPU and serving infrastructure carries the most weight, for a reason that has little to do with the domain being fashionable. It combines the two properties this report keeps separating. It is maximally underdefined at the start—no one could specify in advance which mask families the tile scheduler would tolerate, or that a determinism defect would appear only at batch size eight—and it is maximally verifiable at the end, because parity is bit-exact, latency is measured on the same hardware with the same shapes, and a compilation that crashes is not a matter of opinion. That combination—underdefined at the start, decidable at the end—is what condition T3 of Section 3 asks for, and it is exactly the regime in which a Driver is both necessary and safe to delegate.

It also supplies the strongest external check available to us. Benchmarks and papers in this report are, ultimately, evaluated by harnesses and standards we selected. Upstream review is not: the repository's acceptance criteria were written by people who have never heard of this project, and a maintainer independently re-derived the autotune configuration count, verified the described scope against the diff, and distinguished infrastructure flakes from test failures before approving. The result that carries the most information is the two-line correctness fix, precisely because it has no performance story. A system

optimising for a visible number does not produce it. A system that treats "the test suite cannot run to completion" as a result worth delivering does.

The same section contains the clearest negative evidence in the report. The FlashAttention-4 campaign's initial dense-plus-block-sparsity route was 29.6× slower than native, and the campaign recovered by recognising the data path was wrong rather than by optimising within it. That rejected route, its measurements, and the reasoning that killed it are retained in the skill library. This is what Section 4 means by admitted state changing the starting point of later work: the next campaign in this area does not have to rediscover that the route is a dead end.

7.11 Verticals Whose Verifier the Runtime Does Not Own

The chip and materials campaigns (Sections 6.13 and 6.14) sharpen the same argument under a stricter condition: in both, the deciding evaluator is external. A static-timing engine and an independent chemistry validator will contradict an unsupported claim the moment they are run, so the gate cannot be satisfied by narrative.

Their outcomes are informative in opposite directions, and both are consistent with the position of Section 3. ACE-2 closes a scope and then publishes the perimeter of that scope as part of the result, so the exclusions are an output of the gate rather than a caveat appended to it. The materials campaign instead reverses a design assumption: after separating an integrator change from a scoring change, the runtime found that the published particle-interaction step was discarding the exploration it paid for, and that keeping K trajectories independent scored higher at matched compute. The admitted method is therefore smaller than the one it replaced. Neither outcome required a new component, and neither was reported as a state-of-the-art claim, because in one case the evidence stops at mapped synthesis and in the other it stops at the same validator the selection optimizes against.

7.12 Connecting Theory and Results

A formalism earns its place by being checkable against something. Each row of Table 11 pairs a component of the process-to-capability model with the measurement that bears on it, so that a reader who doubts the model knows which number to attack.

Theory component	Measurement	Observed result	Interpretation
Process-data dominance	Mathematical trajectory	One dead route and six dependent theorem updates	The retained process changes the starting state of later missions.
Review gate	43 Reviewer revision requests	34 verifier recoveries; 22 strict rescues	Independent review redirects initially unacceptable work into another implementation round.
Reuse value G_L	Startup W1–6 versus mature W19–22	21% fewer solve input Tokens; 15% less active time	Later operation reaches a lower resource regime while shared state expands.
Recovery under persistent state	Paper-production trajectory	Seven approaches dropped, two late submission repairs, and 29 session rolls	The campaign changes hypothesis and repairs evidence without losing accepted results or restarting the manuscript.
Token conversion	Rounds 12–17 role attribution	56.0% reasoning, 55.4% verification, and 55.6% action direct shares	The research trace separates frontier work, coordination, and interrupted work.

Table 11. Connection between the process-to-capability theory and measured system behavior. Detailed substitutions appear in Appendix D.

8 Limitations and Future Work

Everything here comes from a four-month window. Argus began in May 2026, and the campaigns in this report ran between then and August. This bounds what the report can claim in both directions, and we state it first because the other limitations should be read against it. It means we have no evidence about long-term stability: whether retained knowledge stays useful over a year, whether a knowledge base accumulates contradictions faster than it retires them, whether the duty-cycle profile holds once campaigns outlive the team's attention to them. Systems of this kind commonly degrade on exactly those axes, and four months is not long enough to observe it. It also means the runtime changed underneath its own evaluation—logging, verticals, and role contracts all evolved across the period, so the campaigns are not instances of one frozen system. Readers should treat this as a snapshot of a system still moving quickly rather than a characterization of a mature one, and should expect the specific mechanisms described in Section 4 to keep changing.

The model may absorb the Driver. A serious counterargument holds that scaffolding is temporary: each generation of models absorbs what the previous generation's harness had to supply, so the model eats the harness (Kilpatrick, 2026). We expect that to keep happening, and Section 7.7 argues the runtime should feed it. But absorption does not settle the question this report asks. Two of the four Driver questions are not capability problems at all: who may certify completion is a question about authority, not about reasoning, and a model that judged its own work perfectly would still be judging its own work. The same holds for the boundary that stops for credentials and irreversible actions. If a future model internalizes Q2 and Q3 entirely, Q1 and Q4 remain structural, and the mechanisms here would become cheaper rather than unnecessary. What we cannot say is where that line finally settles.

User-guided pivots are not publicly evaluated prospectively. Internal system-verification use has produced concrete cases in which stopping or surfacing a contradiction helped real researchers refine the operational task. Those cases contain project-specific research details and user interactions that cannot be disclosed here. They motivate the mechanism but do not constitute a public prospective study of clarification quality, researcher calibration, or time to identify the binding constraint. A publishable evaluation requires consented, prospectively designed collaboration studies with disclosure-safe tasks.

Unattended-operation metrics come from production campaigns. Duty cycle and interruption rate are measured from runtime ledgers across campaigns that differ in vertical, task difficulty, hardware, and operator availability. They characterise an operating profile, not a controlled protocol: no campaign was replayed under a matched configuration, and the duty-cycle denominator excludes periods with an empty backlog, which is a judgement about what the runtime should be held responsible for rather than a neutral measurement. The aggregate raw busy fraction over all wall-clock time is 32.4%, and the two figures answer different questions. Interruption counts include only system-initiated requests, so a deployment whose operators intervene voluntarily will see a different practical experience than the rate suggests. The 1-hour comparison figure for a turn-by-turn coding agent is an internal observation under a different scaffold, not a matched trial.

Delivery comparisons are bounding observations, not controlled trials. The SGLang comparison involves one engineer, one task, and unmatched starting information—the human attempt could consult an already-public implementation, which makes 60 hours a lower bound rather than an estimate. The three-month specialist-team figure for the FlashAttention-4 direction comes from a practitioner interview covering work with a different scope and acceptance standard. The 80 CNY figure is incremental model spend reconciled from vendor billing; the settled ledger records these calls as unpriced because subscription CLIs do not report per-call cost, and the figure excludes subscription seats, GPU time, and human hours. External maintainer acceptance establishes that specific artifacts met an outside standard, not that unattended output meets it in general.

Scope of the infrastructure results. The FlashAttention-4 parity and performance figures were verified on an SM90/Hopper-class device; comparable H100 figures are a team report awaiting raw logs and should not be read as archive-verified. Two of the upstream kernel submissions are opt-in or restricted to a promoted dispatch bucket, so the default path for library users is unchanged and no general end-user speedup is claimed. The Blackwell correctness fix has no before-and-after performance comparison by construction. Single-workload measurements on one device are reported as such throughout, and none of these results establishes that the runtime reaches the same standard on kernel families it has not attempted.

Evidence-driven control can still misjudge the evidence. Holding the objective as a revisable hypothesis moves the failure mode rather than removing it. The risk is no longer that a correct target gets abandoned but that an incorrect revision is admitted: a Manager or operator can approve a poorly framed tradeoff, a recorded standing intent can omit a tacit requirement, and evidence sufficient to refute one framing does not by itself establish that the replacement is better. The report-level K_t/X_t projection is also distributed across several runtime surfaces rather than one atomic transaction. We do not have a measurement separating revisions that were genuinely warranted from those that merely cleared the gate; establishing one requires seeded misspecifications with known ground truth. Future work should test adversarial refinements, invariant-preservation checks, disagreement handling, and rollback of harmful contract changes.

Verification is only as sound as its evidence boundary. An executable test, formal checker, benchmark, or model Reviewer can encode the wrong property. The runtime can record and revise a verifier, but that does not make the verifier correct. Controlled studies should separately seed implementation bugs, specification bugs, and mismatches between them, then measure whether the runtime localizes the faulty side rather than merely satisfying the current check.

Attribution and task sequence. The SWE-Bench Pro experiment compares the complete Argus runtime with Direct Copilot. Reviewer routing is adaptive, the 22 completed Waves (24 Wave IDs with two incomplete Waves excluded) follow one task order, and per-Wave Direct-Copilot Token/time records are unavailable. The current results therefore characterize whole-system behavior over this sequence. Matched frozen-state runs, randomized task orders, and randomized review routing will separate the contributions of persistent state, role separation, and review. The startup-to-mature comparison is observational rather than a causal learning ablation.

Generality across models, domains, and evaluators. The seven benchmark arenas use different metrics, hardware, and execution protocols, while the mathematical analysis follows one vertical campaign. Results are interpreted in their native units rather than combined into a universal score. The in-progress GLM-5.2 run on Claude Code is the first evidence that the runtime transfers across both backbone and execution surface, but it is incomplete and has no matched Direct baseline, so it bounds nothing yet. Repeated campaigns, additional backbones, and external domain review will measure how the runtime transfers across task families and research standards.

ACE-2 is certified for a demonstrated scope, not for silicon. The chip result establishes functional closure of a 24-layer, two-token Qwen2.5-0.5B W4A8 integration and canonical mapped SKY130 synthesis with static timing. It is not routed timing, power signoff, DRC/LVS, GDS or tapeout, silicon validation, generation beyond two tokens, or an FPGA prototype, and the certificate records those exclusions itself. Whether the same runtime reaches signoff-quality physical design or fabricated silicon is untested.

Metric calibration and model transfer. The efficiency analysis labels whole role calls by mission purpose; finer segment labels would sharpen the reasoning, action, and verification factors. Reviewer sensitivity and false-acceptance rate require randomized routing with an external verifier, and the counterfactual reuse value G_L requires paired future tasks with and without a state update. The runtime-to-model path is currently a training direction: held-out trajectory splits and scaffold-removal studies will measure how much of the long-horizon control loop can be internalized by the model.

Scope of the paper-production case study. The six paper projects come from one shared research environment, campaign-hours overlap in calendar time, stored review snapshots are model-generated, and runtime logging evolved across projects. Within that scope, the case study establishes end-to-end lifecycle behavior and recovery under review across six domains and two venue formats, not paper acceptance, scientific novelty, or superiority to human research teams. It also does not establish a measured zero-touch rate. Recorded costs in the public data exclude utility calls without a priced event.

8.1 Future Work: The Loop Between Auto Research and RSI

The preceding section lists what four months does not establish. This one states what we are building toward, because the items above are a route rather than a backlog: the runtime described here is the first version of a program, and the questions it leaves open are the ones the next version is designed to answer.

The containment relation of Section 2.4—recursive self-improvement as auto research pointed at AI itself—is accurate but static. The direction we think matters is dynamic, and it runs in both directions: each capability is a precondition for the other, and neither reaches its ceiling alone.

RSI is a precondition for auto research in any specialized domain. An auto research system operating on frontier problems cannot run on pretrained knowledge alone, and this is a structural limitation rather than a matter of scale. Domain expertise has a short half-life at the frontier: the kernel technique that matters was published last quarter, the mask semantics were defined by a paper that does not yet have citations, the toolchain quirk that blocks synthesis was discovered by three people and written down by none of them. Human experts do not merely hold knowledge; they continuously manufacture it, and a system whose parameters were frozen before that manufacture is permanently behind wherever the work is actually interesting.

A system that only retrieves what it was trained on can therefore assist an expert but cannot replace one. Doing what an expert does requires learning *in situ*: acquiring, from the campaign itself, the facts and procedures that did not exist when the model was trained, and retaining them in a form later work can rely on. That is self-improvement, and it is not optional garnish on auto research—it is what separates auto research at the frontier from auto research on solved problems. This report measures the parameter-invariant form of it, and the natural extension is to carry the same discipline into the parameters themselves, so that what a campaign discovers can change how the system reasons rather than only what it retrieves.

Auto research is a precondition for RSI, in two ways. The reverse dependency is at least as strong, and it is why we regard the model-training-engineer direction as the one that will arrive first.

The first is data. Equation 15 formalizes why a final artifact is a lossy projection of the trajectory that produced it, and the practical consequence is that the internet contains overwhelmingly the projections. It records the merged patch, not the four approaches abandoned before it; the published theorem, not the route that was falsified on the way; the accepted paper, not the reframing that happened after the method search failed. **Process data is the scarce resource**, and it is scarce for a structural reason: nobody was ever paid to write down the reasoning that did not work. The pattern is visible in the reporting itself. When a frontier lab announced solutions to ten long-open mathematical problems at under $2,000 of tokens each, the first question a careful reader asked was how many problems absorbed the same spend and produced nothing (Willison, 2026)—and that number is not published by anyone, because the incentive to publish it has never existed. An auto research runtime operating under verification produces exactly this, continuously and in typed form—objectives, actions, measurements, verdicts, the routes that were refuted, and the evidence that refuted them—as a byproduct of doing its job. The trajectories in this report were retained to coordinate the next mission. Their more consequential use is as supervision for training the next model, and the conversion path—supervised, preference-based, and reinforcement-learning transitions over agent trajectories—is already established in the literature Section 2 surveys. The demand side is visible in how much effort now goes into manufacturing such corpora deliberately: SWE-Gym and SWE-smith construct software-agent trajectories at scale precisely because they do not occur naturally in scraped data (Pan et al., 2024; Yang et al., 2025), and process supervision outperforms outcome supervision

when the intermediate steps can be obtained (Lightman et al., 2023). What has been missing is a source that generates such data at volume without a human authoring each example. A working auto researcher is that source.

The property that makes it a distinctive source is not volume but *span*. Agent corpora are assembled from short episodes, because an episode that requires a human to sit through it is expensive to collect and an episode that runs unattended has not until now been reliable enough to harvest. The campaigns in this report are individually days long—4 exceed a week and the longest runs 8.1 days, with a single trace reaching 61,797 recorded events. What accumulates over that span is not more examples of the same kind; it is a kind that short episodes structurally cannot contain: a hypothesis proposed on day one, contradicted by a measurement on day three, and revised on day five, with every intermediate state, verdict, and discarded alternative recorded in between. Supervision for long-horizon judgment—when to persist, when to abandon a route, what evidence licenses changing the target—is carried by exactly those long-range dependencies, and an episode that ends in an hour has none of them to exhibit.

This is the concrete form the second axis would take. Rather than train on a public corpus, train on the runtime's own multi-day traces, with the verdicts supplying labels the trajectories already carry: the `blocked` that withheld a completion, the transition that recorded an informative failure as progress, the plan challenge that overturned an earlier decision. We have not done this. We report it as the direction because the runtime now produces the data as a byproduct of operating, and because generating that data was the part that had no obvious path.

The second is capability. Parameter self-evolution requires something that can actually train models: process the data, modify the code, design the ablation, read the curve, and decide what to change. That is not a capability a model has by virtue of being large; it is the competence of a strong training engineer, and acquiring it is an auto research problem in its own right. **Only once a system can conduct research does it become possible for it to conduct the research that produces its own successor.** The pretraining stack of Section 6.12 is the first concrete step: the runtime now owns the machinery, and the open question is whether it can operate that machinery well enough that the resulting model is better because of its decisions rather than despite them.

What closing the loop would look like. Composing the two directions gives a cycle rather than a ladder. An auto researcher runs frontier campaigns; those campaigns generate verified process data no corpus contains; that data trains a model with better research judgment; the improved model conducts better campaigns, including campaigns about training; and its in-situ learning keeps it current with knowledge produced after its own training cutoff. Each turn compounds the next, and the constraint that binds it is neither model scale nor data volume but whether the system can run long enough, unattended enough, and with enough judgment about its own output to complete a turn without a human carrying it. That is why we regard duty cycle, interruption rate, and the willingness to return `blocked` as foundational rather than operational metrics: they measure whether the cycle can turn at all.

The loop is not closed, and no step of it is demonstrated end to end here. What exists is a runtime that executes research-level work unattended, improves its retained state under verification, produces the process record such a loop would consume, and has now built the training machinery such a loop would require. What remains is to connect them, and to measure at each turn whether the system that comes out is better than the one that went in.

8.2 Work in Progress

Three campaigns are running at the time of writing and are reported here as direction rather than result, so that the boundary between what has been verified and what is merely underway stays explicit.

The next three steps on the loop. Concretely, the cycle described above advances by three measurable steps, each independently hard: training a model good enough to serve as a role backbone rather than a demonstration; using Argus's own retained trajectories as the training corpus rather than public data; and running the resulting model in a role to measure whether the runtime that produced it improves. Only the third would constitute evidence of a closed loop rather than of capability, and none is claimed here.

Scaling the materials model. Section 6.14 holds at a research candidate. The continuing work moves from method optimization on an existing backbone to architecture and training-route exploration at larger parameter and data scale on the same B200 cluster, with the aim of carrying the verified scoring function and selection mechanism into a foundation-model setting. The outstanding items enumerated in that section—an independent holdout, the full leaderboard, cross-dataset transfer, and physical plausibility checks beyond the selecting validator—gate any stronger claim.

Completing the in-flight benchmark arenas. The MLE-Bench Lite campaign continues beyond the 13 graded competitions reported in Section 6, and the GLM-5.2 SWE-Bench Pro run on Claude Code has no matched direct baseline and is excluded from all comparisons until it does. Both will be reported on completion rather than at their current partial standing.

9 Conclusion

An agent harness lets a model act. Someone still has to decide what is worth attempting, whether the attempt succeeded, and when the answer belongs to a person. That seat—the Driver—has held a human in every deployed agent system, which is why agent progress has been rate-limited by waking hours rather than by capability. Argus takes the wheel.

The enabling condition was not a stronger model. It was giving the completion decision an owner other than the executor, and giving that owner something to decide with where no score exists. A Reviewer that runs read-only cannot repair the evidence it judges; one that can return `blocked` can decline to certify rather than manufacture a success; a boundary that stops for credentials and irreversible actions keeps escalations rare and meaningful. These are usually called quality features. They are more accurately the load-bearing wall of unattended operation: without a party that may certify and did not do the work, someone has to stay in the room, because nothing else can be trusted to say when the work is done. Where a numeric reward is absent, the runtime records progress as a typed transition in which informative failure still counts, refuses to let an experiment that never ran refute a hypothesis, and holds the objective as a hypothesis rather than a target—so a campaign can improve for days with no number improving, which is the ordinary condition of research rather than an edge case.

The delegation is measurable, so we measured it: 38 human requests across 1,548 hours, one every 40.7 hours, of which only 13% were research judgment, at 95.1–98.7% duty cycle with work in hand. The hours produced artifacts rather than uptime—a FlashAttention-4 adaptation within 5–14% of native for under 80 CNY, an SGLang integration that had defeated more than 60 hours of human effort, and 4 kernels accepted by outside maintainers with no blocking changes, one of them a correctness fix with no speedup to report because before it the test suite could not run at all.

Self-improvement here has two axes, and separating them is what keeps the claim falsifiable. The parameter-invariant axis compounds retained knowledge, verifiers, and refuted routes under a fixed backbone; this report measures it. The parameter-changing axis moves the ceiling that the first cannot, and requires producing weights—Argus has built the pretraining stack, which is the entry condition rather than the closed loop. The two are coupled: operating on the first generates the typed trajectories the second would train on, so a runtime that self-evolves while recording how it did so assembles its own supervision as a byproduct of working. Training on that corpus, and measuring whether the resulting model improves the runtime that built it, is what remains.

These results come from production campaigns, not a controlled protocol, and Section 8 sets out what that costs. What survives the accounting is the part worth keeping: one runtime carried research-level work across software, kernels, silicon, mathematics, materials, and manuscripts while a person was not watching—and the mechanism that made that acceptable was not autonomy granted to the executor but authority withheld from it.

Contributors

Argus is built by a team across ten institutions. Within each group below, names are listed alphabetically by given name.

Core Contributors

Boxiu Li (Shanghai Jiao Tong University; Microsoft), Zimo Wen (Shanghai Jiao Tong University), Yijia Fan (Microsoft).

Contributors

Chuan Wen (Shanghai Jiao Tong University), Fan Yang (Microsoft), Hangxi Guo (The Chinese University of Hong Kong, Shenzhen), Jiaao Wu (Tsinghua University), Jiachen Zhang (Independent Researcher), Junxiang Lei (Fudan University), Mukai Li (The University of Hong Kong), Ruize Tang (Microsoft), Runjing Gu (Shanghai Jiao Tong University), Shibo Hu (Peking University), Sihan Chen (Shanghai Jiao Tong University), Sufeng Guo (Nanjing University), Wanbo Zhang (Fudan University), Xi Chen (Independent Researcher), Xian Zhang (Microsoft), Xiaoyu Chen (Shanghai Jiao Tong University), Xuanhe Zhou (Shanghai Jiao Tong University), Xuyao Huang (Shanghai Jiao Tong University), Yifei Gao (Shanghai Jiao Tong University), Yifei Shen (Microsoft), Yijie Jin (Shanghai Jiao Tong University), Yilin Chen (Donghua University), Yuheng Wu (Shanghai Jiao Tong University), Yuzhe Zhang (Shanghai Jiao Tong University), Zelong Zhao (Independent Researcher), Zhijie Deng (Shanghai Jiao Tong University).

A Detailed SWE-Bench Pro Results

The main text reports the startup-to-mature comparison. Table 12 shows every analysis window, including the composition shift and late difficult-task period.

Window	Interpretation	Tasks	Input/task	Active min/task	Token index
W1–6	Startup	120	2.95M	8.52	100
W7–12	Early reuse	140	2.07M	7.28	70
W13–18	Composition shift	151	1.47M	10.42	50
W19–22	Mature	158	2.33M	7.25	79
W23–24	Late difficult tasks	49	3.72M	9.01	126

Table 12. Task-weighted Argus window statistics. Token index normalizes W1–6 to 100.

Wave	Tasks	Solve input/task	Agent s/task	Skills	Wiki
1	20	2.839M	545.6	34	27
2	20	3.263M	642.6	54	41
3	20	3.609M	569.6	68	52
4	20	2.908M	490.7	89	64
5	20	2.832M	452.0	106	81
6	20	2.246M	366.8	127	94
7	20	2.564M	519.8	148	101
8	20	2.882M	465.8	168	115
9	20	2.615M	394.2	190	129
10	20	2.204M	377.1	208	143
11	20	2.041M	396.9	225	155
12	40	1.099M	451.5	260	171
14	32	1.327M	527.0	290	191
16	39	1.564M	623.1	305	206
17	40	1.537M	668.5	306	219
18	40	1.432M	662.8	333	237
19	40	2.424M	431.0	365	258
20	40	2.129M	400.5	404	284
21	39	2.165M	427.9	413	305
22	39	2.593M	481.3	434	323
23	38	3.873M	565.6	471	345
24	11	3.186M	455.4	478	352

Table 13. All completed Argus Wave summaries used in the longitudinal analysis. Two incomplete Waves are omitted from grouped means.

B Reviewer Intervention Details

Table 14 separates routing decisions from recovery outcomes. The first two rows give the routing split; rows three through five partition the 466 independent-Reviewer outcomes; the final two rows measure recovery after a revision request.

Reviewer mechanism event	Tasks	Interpretation
Independent Reviewer invoked	466	63.7% of 731
Engineer self-review	265	36.3% of 731
Accepted on first independent review	388	terminal `done`
Blocked on first independent review	35	terminal `blocked`
Independent Reviewer requested revision	43	at least one `continue`
Official verifier success after revision	34	79.1% of revision requests
Strict review-loop rescue	22	51.2% of revision requests

Table 14. Reviewer routing and recovery. Strict rescue requires an independent Reviewer `continue` followed by a later Reviewer `done`.

C Representative Vertical Trace Details

Table 15 expands the role handoffs summarized in Figure 6. The table deliberately foregrounds Agent authority and durable outputs rather than the mathematical statements.

Phase	Manager / Planner	Engineer	Reviewer	Durable output
Recover and scope	Manager restores the campaign; Planner screens candidate problems.	Grounds sources and creates deterministic checks.	Verifies problem fidelity and accepts the scoped objective.	Persistent objective, selected problem, initial claim record.
Test a route	Planner chooses the cheapest decisive falsification mission.	Runs the witness verifier and records the failed reduction.	Accepts the negative result and fixes its scope.	Certified dead branch and revised next action.
Produce proof package	Planner requires theorem, proof, and verification artifacts.	Writes proof, validation record, lemma graph, and checkpoint.	Returns `continue` when checks are incomplete, then rechecks.	Reviewer-accepted bounded result package.
Strengthen	Planner requires later work to consume and advance retained state.	Improves one result and proves bridge/obstruction lemmas.	Checks correctness, strict progress, and claim boundaries.	Updated accepted state plus named remaining blockers.
Continue	Manager advances or rolls back from the verdict; Planner selects the next blocker.	Starts the next mission from retained artifacts.	Updates the frontier status and next blockers.	Inspectable research trace and reusable cross-session state.

Table 15. Role-resolved phases in the reported Erdős–Gyárfás campaign. The table follows how each role changes the shared research state.

The supplementary trace bundle contains the role/claim table, calculation map, editable figure source, and provenance. The source campaign retains the complete artifacts and event history.

Representative proof artifact (abridged)

For fixed roots x, y, let $\mathscr{P}_4(x,y)$ be the length-4 paths from x to y, and let $\mathscr{F}$ be their internal three-vertex sets. In a graph with no C_8, any two members of $\mathscr{F}$ intersect: two paths with disjoint internal vertices would join to form an 8-cycle. A separate no-C_4 case analysis shows that any fixed pair of internal vertices occurs in at most three path sets.
Assume that the path family has no common internal blocker. Choose $E = \{a,b,c\} \in \mathscr{F}$. For each $t \in E$, some member F_t omits t. Every set containing t must meet the three vertices of F_t; the pair-occurrence bound therefore gives $d(t) \le 9$. Hence

$$d(a) + d(b) + d(c) \le 27.$$

Every member of $\mathscr{F}$ meets E, while E itself contributes three incidences rather than one, so

$$|\mathscr{F}| + 2 \le d(a) + d(b) + d(c) \le 27.$$

Thus $|\mathscr{P}_4(x,y)| = |\mathscr{F}| \le 25$: the paths share a common internal blocker or there are at most 25 of them. This excerpt illustrates the kind of proof artifact produced and reviewed by the workflow; the complete case analysis and computational checks remain in the source bundle.

D Process-Theory Calculations

Table 16 records the complete substitutions behind the process quantities summarized in the main text. The mathematical window contains 332,274 reasoning-output Tokens over 9,620.5 active seconds. The strict and inclusive density values use the two attribution policies defined in Equation 5. The process-compression row is a separate campaign snapshot.

Theory quantity	Data substitution	Observed value	Interpretation
Strict $\widehat{\rho}_I$	$332{,}274/9{,}620.5 \times 0.5598 \times 0.5556 \times 0.5538$	5.95 effective Token-eq./s	Direct reasoning, action, and verification only.
Inclusive $\widehat{\rho}_I$	$332{,}274/9{,}620.5 \times 0.9512 \times 0.9444 \times 0.9043$	28.06 effective Token-eq./s	Includes successful coordination and state-maintenance work.
Reviewer correction	34/43 verifier recoveries; 22/43 strict rescues	79.1%; 51.2%	Post-revision outcomes on SWE-Bench Pro.
Observational proxy $\widehat{G}_{\text{obs}}$	$(2.95-2.33)$M Tokens/task; $(8.52-7.25)$ min/task	0.62M Tokens and 1.27 min per task	**Not** the counterfactual G_L of Eq. 18: a startup-versus-mature difference confounded by task composition. The counterfactual requires a frozen-state replay.
Immediate $\widehat{\mathscr{Y}}_{\text{PRI}}$	$\lambda_g = 0$; $6/332{,}274 \times 10^6$	18.1 accepted deltas/M reasoning Tokens	Six accepted theorem-frontier updates in Rounds 12–17.
Process compression	$384{,}370{,}463/3{,}240$ bytes	118,633× event-history/checkpoint ratio	Event history relative to the active working checkpoint.

Table 16. Detailed calculation of the process-theory quantities. Values retain their task-specific units; the main text reports the corresponding system-level patterns.

E Upstream Kernel Adoption

The kernel-optimization work produced an externally reviewed upstream contribution to Flash Linear Attention. In FLA PR #1045, Argus implemented and optimized an opt-in TileLang `RWKV6` dense-`bf16`, `D=64` forward-intra kernel. The submitted H100 evidence reports a forward latency reduction from 0.199 ms to 0.168 ms (1.18×) and a forward-plus-backward reduction from 0.900 ms to 0.747 ms (1.21×), with 13 correctness-gate passes and 14 repository tests.

An FLA collaborator affiliated with Moonshot AI reviewed the generated CUDA, identified a long-sequence numerical-stability issue, and requested block-local exponent centering. After Argus applied the fix and reran the verification suite, the contribution was merged into `fla-org:main` on July 20, 2026 as commit `c70f11c`. This is evidence of human-reviewed upstream adoption, not external scientific validation of the broader runtime.

F Notation

Symbol	Meaning
o, y_0, f, B	Objective, initial artifact, evaluator, and resource budget
$K_t = (\iota, o_t, c_t, v_t)$	Standing intent, objective, constraints, and verification criteria
X_t	User-visible clarifications, priorities, and unresolved questions
K'_t, u_t	Proposed contract and required admission authority
Γ_t	Round-level operational records before trajectory projection
τ_t	Mission trajectory over one or more execution/review rounds
H_t	Persistent runtime state
E_t	Results admitted after mission t
Q_t	Persistent task and evaluation definitions
U	Partial persistent-state update operator
ManagerAdmit	Analytical projection of evidence- and authority-gated contract refinement
θ_t	Underlying model parameters, fixed in this report
C_t	Effective retained capability set
$\phi(\tau_t)$	Role-resolved mission trace $(m_t, p_t, x_t, r_t, \Delta H_t)$
$\rho_I(T)$	Dense-intelligence density over horizon T
η_r, η_a, η_v	Measured reasoning, action, and verification efficiency factors
$\mathscr{R}_q(X)$	Minimum downstream decision risk when observing X
$\psi_b^\star$	Decision-useful process compression under context budget b
$G_L(\Delta H_t)$	Discounted counterfactual reuse value over L future tasks
$\mathscr{Y}_{\text{PRI}}(W)$	Verified reusable process-intelligence yield per Token
α, β	Reviewer sensitivity and false-acceptance rate
$V_R(T)$	Conceptual accumulated research value
R_{tok}	Aggregate Argus-to-Direct-Copilot Token ratio
$\bar{I}_w^{\text{solve}}$	Mean solve input tokens in Wave w
$\bar{T}_w^{\text{agent}}$	Mean active workflow time in Wave w

Table 17. Notation used in the report.

Machine-readable aggregates and editable figure sources are released with the report. They are kept outside the narrative so that implementation metadata does not obscure the scientific results.

References

Andreessen Horowitz. Can agents use a computer yet? we've got the data. https://a16z.com/can-agents-use-a-computer-yet-weve-got-the-data/, 2026.

Anthropic. Discovering cryptographic weaknesses with Claude. https://www.anthropic.com/research/discovering-cryptographic-weaknesses, July 2026a.

Anthropic. Emergent risks in multiagent AI systems. https://www.anthropic.com/research/multiagent-systems, 2026b.

Argus Team. Argus: A self-evolving research agent. Project website, 2026. URL https://argusbot.cn/.

Jinheon Baek, Sujay Kumar Jauhar, Silviu Cucerzan, and Sung Ju Hwang. ResearchAgent: Iterative research idea generation over scientific literature with large language models. *arXiv preprint arXiv:2404.07738*, 2024. URL https://arxiv.org/abs/2404.07738.

David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2):265–272, 1953. doi: 10.1214/aoms/1177729032. URL https://doi.org/10.1214/aoms/1177729032.

Ryan Caldwell and Bhavya U. Improving token efficiency for GitHub Copilot in VS Code. https://code.visualstudio.com/blogs/2026/06/17/improving-token-efficiency-in-github-copilot, June 2026. Visual Studio Code engineering blog.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, et al. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024. URL https://arxiv.org/abs/2410.07095.

Zehui Chen, Kuikun Liu, Qiuchen Wang, Wenwei Zhang, Jiangning Liu, Dahua Lin, Kai Chen, and Feng Zhao. Agent-FLAN: Designing data and methods of effective agent tuning for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 9354–9366. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.557. URL https://aclanthology.org/2024.findings-acl.557/.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025. URL https://arxiv.org/abs/2509.16941.

Huan-ang Gao, Jiayi Geng, Wenyue Hua, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025.

Alireza Ghafarollahi and Markus J. Buehler. SciAgents: Automating scientific discovery through multi-agent intelligent graph reasoning. *arXiv preprint arXiv:2409.05556*, 2024. URL https://arxiv.org/abs/2409.05556.

Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, et al. Accelerating scientific discovery with co-scientist. *arXiv preprint arXiv:2502.18864*, 2025. URL https://arxiv.org/abs/2502.18864.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, et al. A survey on LLM-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, et al. Large language models cannot self-correct reasoning yet. *International Conference on Learning Representations (ICLR)*, 2024.

Thomas Hubert, Rishi Mehta, Laurent Sartran, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2025. doi: 10.1038/s41586-025-09833-y.

Hugging Face. Anatomy of a frontier lab agent intrusion: A technical timeline of the july 2026 incident. https://huggingface.co/blog/agent-intrusion-technical-timeline, July 2026. Companion to the July 2026 security incident disclosure.

Marcus Hutter, Shane Legg, et al. From AGI to ASI. *arXiv preprint arXiv:2606.12683*, 2026. Google DeepMind.

Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, et al. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025. URL https://arxiv.org/abs/2502.13138.

Rui Jiao, Wenbing Huang, Peijia Lin, Jiaqi Han, et al. Crystal structure prediction by joint equivariant diffusion. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. URL https://arxiv.org/abs/2310.06770.

Jiajie Jin, Yuyang Hu, Kai Qiu, Qi Dai, Chong Luo, et al. Toward generalist autonomous research via hypothesis-tree refinement. *arXiv preprint arXiv:2606.11926*, 2026. URL https://arxiv.org/abs/2606.11926.

Keller Jordan and contributors. Modded-NanoGPT: The nanogpt speedrun. GitHub repository, 2024. URL https://github.com/KellerJordan/modded-nanogpt.

Andrej Karpathy. nanochat: The best ChatGPT that $100 can buy. GitHub repository, 2025. URL https://github.com/karpathy/nanochat.

Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 2026. URL https://github.com/karpathy/autoresearch.

Logan Kilpatrick. Why the model eats the harness. *Training Data* podcast, Sequoia Capital. https://www.sequoiacap.com/series/training-data, 2026.

Nayoung Kim, Seongsu Kim, Minsu Kim, Jinkyoo Park, and Sungsoo Ahn. MOFFlow: Flow matching for structure prediction of metal-organic frameworks. *arXiv preprint arXiv:2410.17270*, 2024.

Thomas Kwa, Ben West, Joel Becker, et al. Measuring AI ability to complete long software tasks. *arXiv preprint arXiv:2503.14499*, 2025. URL https://arxiv.org/abs/2503.14499.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023. URL https://arxiv.org/abs/2305.20050.

Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, et al. SOL-ExecBench: Speed-of-light benchmarking for real-world GPU kernels against hardware limits. *arXiv preprint arXiv:2603.19173*, 2026. URL https://arxiv.org/abs/2603.19173.

Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, et al. ChipNeMo: Domain-adapted LLMs for chip design. *arXiv preprint arXiv:2311.00176*, 2023a.

Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. Verilogeval: Evaluating large language models for verilog code generation. *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023b.

Xiao Liu, Hao Yu, Hanchen Zhang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023c. URL https://arxiv.org/abs/2308.03688.

Chris Lu et al. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL https://arxiv.org/abs/2408.06292.

Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent lightning: Train any AI agents with reinforcement learning. *arXiv preprint arXiv:2508.03680*, 2025. URL https://arxiv.org/abs/2508.03680.

Alisia Lupidi, Bhavul Gauri, Thomas Simon Foster, et al. AIRS-Bench: A suite of tasks for frontier AI research science agents. *arXiv preprint arXiv:2602.06855*, 2026. URL https://arxiv.org/abs/2602.06855.

Chang Ma, Junlei Zhang, Zhihao Zhu, et al. AgentBoard: An analytical evaluation board of multi-turn LLM agents. *arXiv preprint arXiv:2401.13178*, 2024. URL https://arxiv.org/abs/2401.13178.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.

Arindam Mitra, Luciano Del Corro, Guoqing Zheng, Shweti Mahajan, Dany Rouhana, Andres Codas, et al. AgentInstruct: Toward generative teaching with agentic flows. *arXiv preprint arXiv:2407.03502*, 2024. URL https://arxiv.org/abs/2407.03502.

Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, et al. MLGym: A new framework and benchmark for advancing AI research agents. *arXiv preprint arXiv:2502.14499*, 2025.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. URL https://arxiv.org/abs/2506.13131.

OpenAI. Disrupting malicious uses of ai. https://openai.com/index/disrupting-malicious-ai-uses/, February 2026a. Threat intelligence report.

OpenAI. Scientific computing in the age of agentic AI. https://openai.com/index/scientific-computing-agentic-ai/, July 2026b. Field report on eight agent-assisted life-science projects.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, et al. Kernelbench: Can llms write efficient gpu kernels? *arXiv preprint arXiv:2502.10517*, 2025.

Charles Packer, Sarah Wooders, Kevin Lin, et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, et al. Training software engineering agents and verifiers with swe-gym. *arXiv preprint arXiv:2412.21139*, 2024.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. Llm evaluators recognize and favor their own generations. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, et al. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023. URL https://arxiv.org/abs/2304.03442.

Weitong Qian, Beicheng Xu, Zhongao Xie, et al. AutoSci: A memory-centric agentic system for the full scientific research lifecycle. *arXiv preprint arXiv:2605.31468*, 2026. URL https://arxiv.org/abs/2605.31468.

Recursive. First steps toward automated AI research. https://www.recursive.com/articles/first-steps-toward-automated-ai-research, 2026. State-of-the-art results on NanoChat Autoresearch, NanoGPT Speedrun, and SOL-ExecBench.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6. URL https://doi.org/10.1038/s41586-023-06924-6.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. URL https://arxiv.org/abs/2302.04761.

Samuel Schmidgall and Michael Moor. AgentRxiv: Towards collaborative autonomous research. *arXiv preprint arXiv:2503.18102*, 2025. URL https://arxiv.org/abs/2503.18102.

Samuel Schmidgall, Yusheng Su, Ze Wang, et al. Agent laboratory: Using LLM agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025. URL https://arxiv.org/abs/2501.04227.

Noah Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. URL https://arxiv.org/abs/2303.11366.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024. URL https://arxiv.org/abs/2408.03314.

Qiong Tang, Tianxiang Sun, Xiangkun Hu, et al. FARS: A fully automated research system deployed at scale. *arXiv preprint arXiv:2606.31651*, 2026. URL https://arxiv.org/abs/2606.31651.

Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, et al. Ai research agents for machine learning: Search, exploration, and generalization in mle-bench. *arXiv preprint arXiv:2507.02554*, 2025.

Guanzhi Wang et al. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

Jiayu Wang, Weijiang Lv, Bowen Fu, et al. Act as a real researcher: A suite of benchmarks evaluating frontier LLMs and agentic harnesses in research lifecycle. *arXiv preprint arXiv:2606.07462*, 2026. URL https://arxiv.org/abs/2606.07462.

Xingyao Wang, Boxuan Li, Yufan Song, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024a. URL https://arxiv.org/abs/2407.16741.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024b. URL https://arxiv.org/abs/2409.07429.

Koki Wataoka, Tsubasa Takahashi, and Ryokan Ri. Self-preference bias in llm-as-a-judge. *arXiv preprint arXiv:2410.21819*, 2024.

Yixuan Weng, Minjun Zhu, Guangsheng Bao, Hongbo Zhang, Jindong Wang, Yue Zhang, and Linyi Yang. CycleResearcher: Improving automated research via automated review. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=bjcsVLoHYs.

Hjalmar Wijk, Tao Lin, Joel Becker, et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024. URL https://arxiv.org/abs/2411.15114.

Simon Willison. Ten advances in mathematics and theoretical computer science. https://simonwillison.net/2026/Aug/1/ten-advances-in-mathematics/, August 2026. Commentary on OpenAI's ten-proofs release.

Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, et al. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*, 2024.

Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, et al. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. *International Conference on Learning Representations (ICLR)*, 2025.

Wujiang Xu, Zujie Liang, Kai Mei, et al. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025. URL https://arxiv.org/abs/2502.12110.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, et al. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025. URL https://arxiv.org/abs/2504.08066.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, et al. Swe-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.

John Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. URL https://arxiv.org/abs/2405.15793.

Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023. URL https://arxiv.org/abs/2210.03629.

Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, et al. Gödel agent: A self-referential agent framework for recursive self-improvement. *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

Claudio Zeni, Robert Pinsler, Daniel Zügner, Andrew Fowler, et al. Mattergen: A generative model for inorganic materials design. *arXiv preprint arXiv:2312.03687*, 2023.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025a.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025b.

Andrew Zhao, Daniel Huang, Quentin Xu, et al. ExpeL: LLM agents are experiential learners. *AAAI Conference on Artificial Intelligence*, 2024.